

AELION — Deep Learning IA (TP B6)

Notebooks 01 à 10 — export complet

Étape 1 — Chargement et préparation des données (MVTec AD — `pil`)

TP B6, partie 1. Plan du notebook :

1. Analyse préliminaire du jeu de données
2. Caractéristiques des images
3. Normalisation
4. Livrable
5. Recommandations

In [1]:

```
from pathlib import Path import matplotlib.pyplot as plt import numpy as np from
IPython.display import Markdown, display from PIL import Image from indusense.vision.dataset
import ( DATA_ROOT, DEFAULT_IMAGE_SIZE, list_defects, load_defect_images, load_good_images,
train_val_split, ) IMAGE_SIZE = DEFAULT_IMAGE_SIZE SEED = 42 FIGURES_DIR =
Path("../reports/figures") FIGURES_DIR.mkdir(parents=True, exist_ok=True)
```

1. Analyse préliminaire du jeu de données

Aperçu de la structure du dataset avant tout traitement : catégories de défauts disponibles et effectifs par split.

In [2]:

```
defects = list_defects() def count_images(folder): return len(list(folder.glob("*.png")))
descriptions = { "color": "variation de teinte ou de pigmentation de la pilule", "combined":
"plusieurs défauts combinés sur la même pièce", "contamination": "présence d'un corps
étranger sur la pilule", "crack": "fissure visible sur la surface", "faulty_imprint":
"gravure d'impression (« FF ») déficiente ou effacée", "pill_type": "pilule d'un mauvais type
mélangée par erreur", "scratch": "rayure de surface", } n_train_good = count_images(DATA_ROOT
/ "train" / "good") n_test_good = count_images(DATA_ROOT / "test" / "good") lines = [ f"-
**Images saines** : {n_train_good} en entraînement (`train/good`), " f"{n_test_good} en test
(`test/good`)." , ] lines += [ f"- **{defect}** : {count_images(DATA_ROOT / 'test' / defect)}
images de test - " f"{descriptions[defect]}." for defect in defects ]
display(Markdown(chr(10).join(lines)))
```

- **Images saines** : 267 en entraînement (`train/good`), 26 en test (`test/good`).
- **color** : 25 images de test — variation de teinte ou de pigmentation de la pilule.
- **combined** : 17 images de test — plusieurs défauts combinés sur la même pièce.
- **contamination** : 21 images de test — présence d'un corps étranger sur la pilule.
- **crack** : 26 images de test — fissure visible sur la surface.
- **faulty_imprint** : 19 images de test — gravure d'impression (« FF ») déficiente ou effacée.
- **pill_type** : 9 images de test — pilule d'un mauvais type mélangée par erreur.
- **scratch** : 24 images de test — rayure de surface.

2. Caractéristiques des images¶

Analyse d'un échantillon d'images brutes (avant redimensionnement/normalisation) : résolution native, contraste fond/objet, centrage et variabilité d'orientation — utile pour calibrer les augmentations de l'étape 2.

In [3]:

```
raw_sample = sorted((DATA_ROOT / "train" / "good").glob("*.png"))[0] with
Image.open(raw_sample) as raw_img: resolution = raw_img.size mode = raw_img.mode raw_rgb =
np.asarray(raw_img.convert("RGB")) raw_gray = np.asarray(raw_img.convert("L")) h, w =
raw_gray.shape corner = raw_rgb[0:20, 0:20].reshape(-1, 3).mean(axis=0) center = raw_rgb[h //
2 - 20 : h // 2 + 20, w // 2 - 20 : w // 2 + 20].reshape(-1, 3).mean(axis=0) raw_paths =
sorted((DATA_ROOT / "train" / "good").glob("*.png"))[:20] centroids, ratios = [], [] for p in
raw_paths: with Image.open(p) as img: gray = np.asarray(img.convert("L")) mask = gray > 60
ys, xs = np.where(mask) centroids.append((ys.mean(), xs.mean())) ratios.append((ys.max() -
ys.min()) / (xs.max() - xs.min())) centroids = np.array(centroids) lines = [ f"- **Résolution
native** : {resolution[0]}x{resolution[1]} px, mode {mode}." , f"- **Contraste fond/objet** :
fond (coin) RGB moyen {corner.round(1)}, " f"objet (centre) RGB moyen {center.round(1)} -
séparation nette, segmentation triviale." , f"- **Centrage** : centroïde moyen (y, x) =
{centroids.mean(axis=0).round(1)} " f"(écart-type {centroids.std(axis=0).round(2)}) - quasi
parfait, dispositif très contrôlé." , f"- **Variabilité d'orientation** : ratio bbox
hauteur/largeur de {min(ratios):.3f} " f"à {max(ratios):.3f} - léger tilt naturellement
présent dans les pièces saines." , ] display(Markdown(chr(10).join(lines)))
```

- **Résolution native** : 800×800 px, mode RGB.
- **Contraste fond/objet** : fond (coin) RGB moyen [12.4 12.1 21.2], objet (centre) RGB moyen [182.5 182.3 182.1] — séparation nette, segmentation triviale.
- **Centrage** : centroïde moyen (y, x) = [400.6 400.3] (écart-type [0.3 0.25]) — quasi parfait, dispositif très contrôlé.
- **Variabilité d'orientation** : ratio bbox hauteur/largeur de 0.683 à 0.917 — léger tilt naturellement présent dans les pièces saines.

3. Normalisation¶

Chargement des images saines : conversion RGB, redimensionnement (rééchantillonnage bicubique), normalisation en float32 dans [0, 1]. Une fraction de train/good est réservée pour la validation (graine fixe, `train_val_split`), qui servira à calibrer le seuil de détection en partie 2.

In [4]:

```
good_train = load_good_images("train", IMAGE_SIZE) good_train, good_val =
train_val_split(good_train, val_fraction=0.15, seed=SEED) good_test =
load_good_images("test", IMAGE_SIZE) lines = [ f"- **Saines train** : {good_train.shape[0]}
images, forme {good_train.shape[1:]} , " f"dtype {good_train.dtype}." , f"- **Saines
validation** : {good_val.shape[0]} images - réservées pour calibrer " f"le seuil de détection
(partie 2)." , f"- **Saines test** : {good_test.shape[0]} images." , f"- **Plage de valeurs
normalisées** : min={good_train.min():.3f}, " f"max={good_train.max():.3f}." , ]
display(Markdown(chr(10).join(lines)))
```

- **Saines train** : 227 images, forme (256, 256, 3), dtype float32.

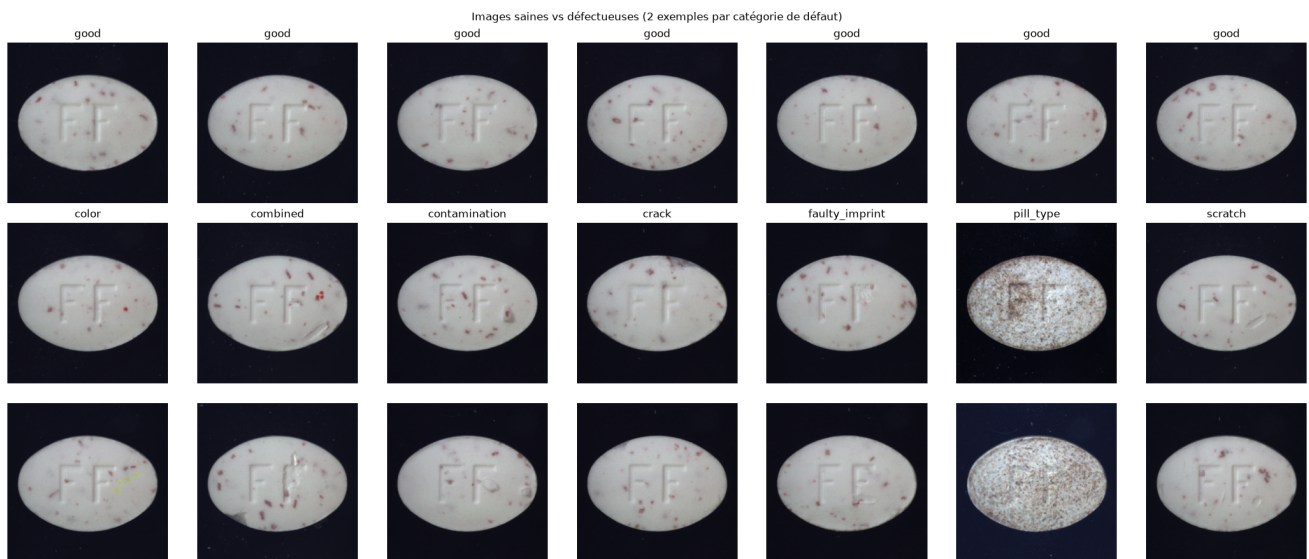
- **Saines validation** : 40 images — réservées pour calibrer le seuil de détection (partie 2).
- **Saines test** : 26 images.
- **Plage de valeurs normalisées** : min=0.016, max=0.969.

4. Livrable — saines vs défauts après chargement/normalisation¶

La figure ci-dessous illustre le résultat concret des sections 1 à 3 : pour chacune des 7 catégories de défauts, une image saine de référence (good) est présentée au-dessus de deux exemples défectueux, toutes redimensionnées et normalisées de façon identique. Elle permet de vérifier visuellement que le chargement préserve bien les couleurs et proportions de la pièce, et sert de référence visuelle pour juger, à l'étape 2, si une augmentation donnée reste plausible ou dénature l'image.

In [5]:

```
n_examples = 2
defect_samples = {defect: load_defect_images(defect, IMAGE_SIZE)[:n_examples]
for defect in defects}
fig, axes = plt.subplots(1 + n_examples, len(defects), figsize=(3 *
len(defects), 3 * (1 + n_examples)))
for col, defect in enumerate(defects):
    axes[0, col].imshow(good_test[col])
    axes[0, col].set_title("good")
    axes[0, col].axis("off")
    for row in range(n_examples):
        axes[1 + row, col].imshow(defect_samples[defect][row])
        axes[1 + row, col].set_title(defect if row == 0 else None)
        axes[1 + row, col].axis("off")
fig.suptitle("Images saines vs défectueuses (2 exemples par catégorie de défaut)")
fig.tight_layout()
fig.savefig(FIGURES_DIR / "01_sain_vs_defaut.png", dpi=150)
plt.show()
```



5. Recommandations pour les augmentations (étape 2)¶

À partir des constats des sections 1 à 3 :

- Le centrage quasi parfait et la largeur fixe (section 2) justifient des amplitudes de translation/échelle modérées (~5 %) plutôt que fortes.
- La variabilité de tilt déjà observée dans les données saines (section 2) rend plausible une rotation légère ($\pm 15^\circ$) — elle ne fait qu'amplifier une variation déjà réelle.
- La catégorie de défaut `color` (section 1) interdit toute augmentation de teinte/saturation forte (seul un ajustement modéré de luminosité/contraste est sûr), sous peine de brouiller un signal de défaut

réel.

- La gravure fine « FF » et la catégorie `faulty_imprint` interdisent flou fort et déformations élastiques, qui effaceraient ou simuleraient ce défaut.
- La gravure « FF » n'est **pas symétrique** : un flip horizontal ou vertical en inverserait la lecture en miroir — une configuration qu'aucune caméra fixe ne peut observer en réalité (la face imprimée ne change pas par une simple rotation 2D). Les flips doivent donc être **exclus** du pipeline, pas seulement limités.
- Les défauts `crack/scratch` étant des textures linéaires fines, tout bruit géométrique (grid distortion) risquerait de générer des artefacts ressemblant à de faux défauts.
- Le fond uniforme (section 2) ne nécessite aucune augmentation dédiée : la variabilité à couvrir concerne uniquement l'objet.

Étape 2 — Augmentation des données (Albumentations — `pill`)¶

TP B6, partie 1. Pipeline appliqué uniquement aux images **saines d'entraînement**.

In [1]:

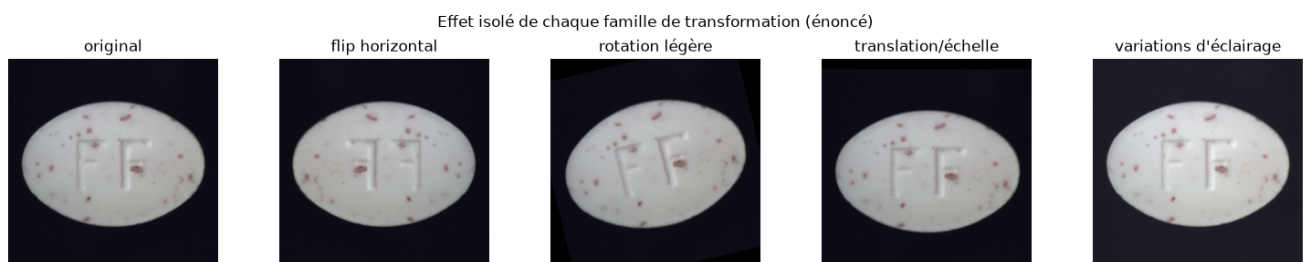
```
from pathlib import Path import albumentations as A import matplotlib.pyplot as plt from
IPython.display import Markdown, display from indusense.vision.augment import augment_image,
build_augmentation_pipeline, describe_replay from indusense.vision.dataset import
DEFAULT_IMAGE_SIZE, load_good_images FIGURES_DIR = Path("../reports/figures")
FIGURES_DIR.mkdir(parents=True, exist_ok=True) good_train = load_good_images("train",
DEFAULT_IMAGE_SIZE) pipeline = build_augmentation_pipeline() original = good_train[0]
```

Démonstration isolée des transformations de l'énoncé¶

L'énoncé du TP liste quatre familles de transformations : **flips, rotations légères, translation/échelle, variations d'éclairage**. Chacune est forcée ici individuellement ($p=1$, une seule transformation à la fois) pour observer son effet propre sur la gravure « FF » — y compris le **flip**, volontairement **exclu du pipeline réel** (`augment.py`), montré ici comme preuve visuelle du problème de miroir (cf. justification plus bas).

In [2]:

```
demo_transforms = { "flip horizontal": A.ReplayCompose([A.HorizontalFlip(p=1.0)]), "rotation
légère": A.ReplayCompose([A.Rotate(limit=15, border_mode=0, p=1.0)]), "translation/échelle":
A.ReplayCompose( [A.Affine(translate_percent=(-0.05, 0.05), scale=(0.95, 1.05),
border_mode=0, p=1.0)] ), "variations d'éclairage": A.ReplayCompose(
[A.RandomBrightnessContrast(brightness_limit=0.15, contrast_limit=0.15, p=1.0)] ), }
demo_results = {name: tf(image=original) for name, tf in demo_transforms.items()} fig, axes =
plt.subplots(1, len(demo_results) + 1, figsize=(3 * (len(demo_results) + 1), 3))
axes[0].imshow(original) axes[0].set_title("original") axes[0].axis("off") for i, (name,
result) in enumerate(demo_results.items(), start=1): axes[i].imshow(result["image"])
axes[i].set_title(name) axes[i].axis("off") fig.suptitle("Effet isolé de chaque famille de
transformation (énoncé)") fig.tight_layout() fig.savefig(FIGURES_DIR /
"02_transformations_isolees.png", dpi=150) plt.show() lines = [f"- **{name}** :
{describe_replay(result['replay'])}" for name, result in demo_results.items()]
display(Markdown(chr(10).join(lines)))
```



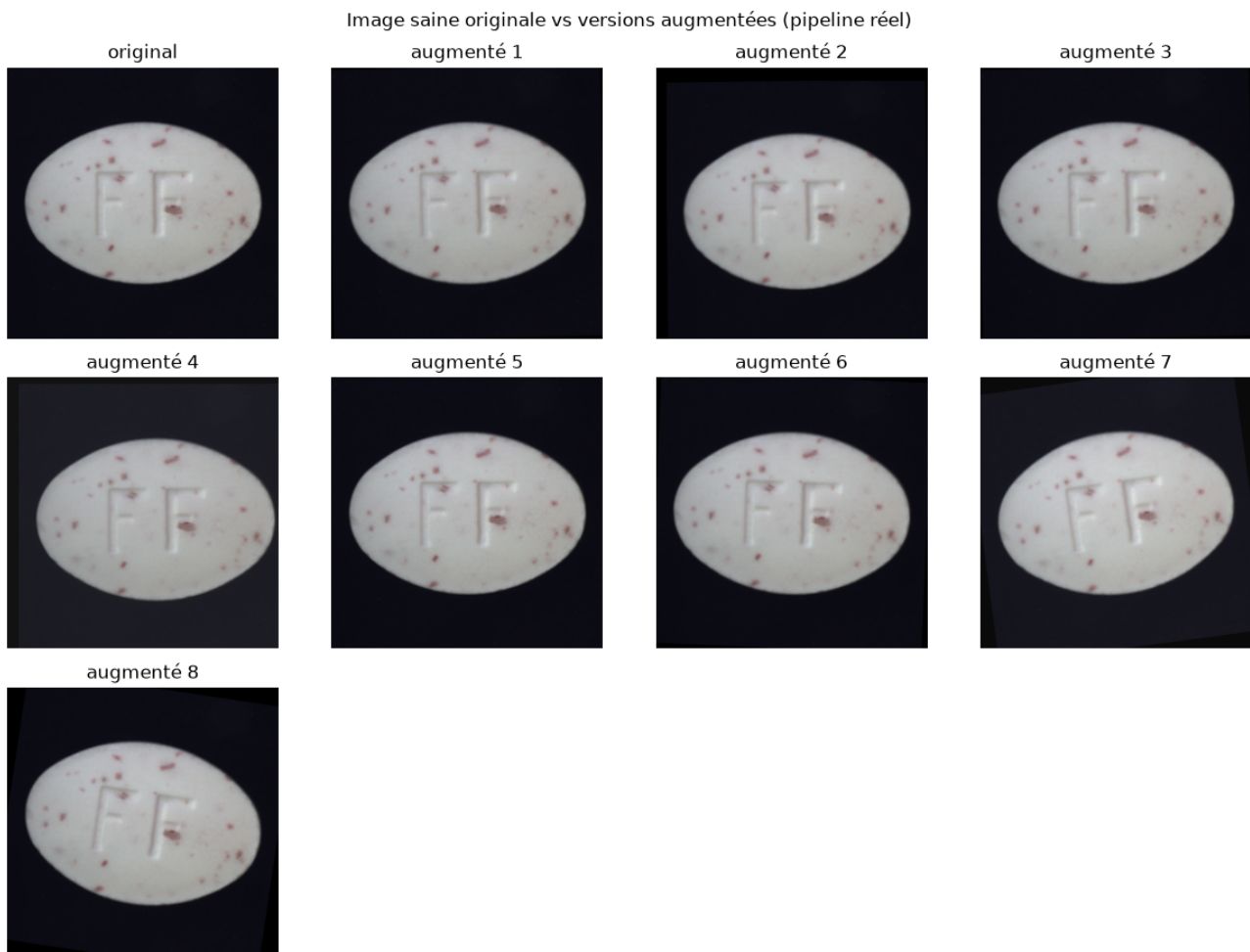
- **flip horizontal** : flip horizontal (miroir gauche/droite)
- **rotation légère** : rotation +13.6°
- **translation/échelle** : translation (-0.0 %, +4.8 %), échelle (×1.01, ×0.97)

- **variations d'éclairage** : contraste $\times 1.09$, luminosité $+0.06$

Livrable — original vs versions augmentées (pipeline réel, 8 tirages aléatoires)¶

In [3]:

```
n_variants = 8 results = augment_image(original, pipeline, n=n_variants) variants = [r["image"] for r in results] descriptions = [describe_replay(r["replay"]) for r in results] n_cols = 4 n_rows = -(n_variants + 1) // n_cols fig, axes = plt.subplots(n_rows, n_cols, figsize=(3 * n_cols, 3 * n_rows)) axes = axes.flatten() axes[0].imshow(original) axes[0].set_title("original") axes[0].axis("off") for i, variant in enumerate(variants, start=1): axes[i].imshow(variant) axes[i].set_title(f"augmenté {i}") axes[i].axis("off") for ax in axes[n_variants + 1 :]: ax.axis("off") fig.suptitle("Image saine originale vs versions augmentées (pipeline réel)") fig.tight_layout() fig.savefig(FIGURES_DIR / "02_augmentations.png", dpi=150) plt.show() lines = [f"- **augmentation {i}** : {desc}" for i, desc in enumerate(descriptions, start=1)] display(Markdown(chr(10).join(lines)))
```



- **augmentation 1** : rotation $+1.2^\circ$; contraste $\times 0.98$, luminosité -0.00
- **augmentation 2** : rotation $+0.7^\circ$; translation ($+4.0\%$, $+5.0\%$), échelle ($\times 0.96$, $\times 0.96$)
- **augmentation 3** : rotation $+1.2^\circ$
- **augmentation 4** : translation ($+4.2\%$, $+2.4\%$), échelle ($\times 1.01$, $\times 1.00$); contraste $\times 0.87$, luminosité $+0.07$
- **augmentation 5** : aucune transformation tirée (probabilités défavorables)

- **augmentation 6** : rotation -2.4°
- **augmentation 7** : rotation $+7.8^\circ$; contraste $\times 1.02$, luminosité $+0.04$
- **augmentation 8** : rotation -8.5°

Justification des transformations retenues (et écartées)¶

`pill` est un objet **centré et aligné** (comme `bottle`), pas une texture : chaque image montre une pilule unique, cadrée de façon quasi identique par la caméra industrielle, avec une gravure « FF » embossée sur la surface.

Retenues, amplitudes modérées :

- **Rotation légère ($\pm 15^\circ$) et translation/échelle modérées ($\pm 5\%$)** : imite les petites variations de positionnement de la pièce sur la ligne, sans dépasser ce qu'une caméra fixe observerait réellement. Une rotation, contrairement à un flip, ne miroir pas le texte : la gravure reste lisible, seulement inclinée.
- **Variations de luminosité/contraste modérées** : reproduit les variations d'éclairage industriel (heure, usure des lampes) sans changer la texture de la pilule elle-même.

Écartées :

- **Flips horizontal/vertical** : *écartés après relecture des caractéristiques de la section 2 du notebook 01*. Une première version du pipeline les incluait, en supposant qu'une pilule vue de dessus reste plausible retournée. C'est faux : la gravure « FF » n'est pas symétrique, et un flip en inverse la lecture en miroir — la démonstration isolée plus haut rend le problème directement visible sur la gravure. Une configuration qu'aucune caméra fixe ne peut observer en réalité (retourner l'image 2D n'équivaut pas à retourner la pièce physique, qui montrerait alors une autre face, pas un miroir de la face imprimée). Un flip produit donc une image saine **impossible**, pas une variation plausible.
- **Rotation forte (proche de 180° ou aléatoire sur 360°)** : sur un objet aligné, une rotation trop forte élargit artificiellement la notion de « normal » que l'auto-encodeur doit apprendre. Le risque direct : un défaut réel (ex. `crack`), une fois « caché » dans une orientation inhabituelle mais couverte par l'augmentation, serait bien reconstruit par le modèle — donc **non détecté**. C'est l'inverse de l'effet recherché.
- **Distorsions élastiques / déformations fortes** : pertinentes pour des textures (tissus, surfaces irrégulières) où la déformation locale ne change pas la nature de l'objet, mais inadaptées ici : la forme et les bords nets de la pilule sont eux-mêmes des indices de défaut (`crack`, `scratch`) qu'il ne faut pas brouiller.

Point de réflexion (objet vs texture) : sur une catégorie *texture* (ex. `carpet`, `wood`), une augmentation géométrique forte (rotation libre, flip, crop aléatoire) est généralement acceptable — la texture est localement invariante par rotation/réflexion. Sur un objet *centré et aligné* portant un repère directionnel (texte, logo) comme `pill`, toute transformation qui inverse la chiralité de l'image (flip) est à proscrire, et l'augmentation géométrique restante doit rester modérée sous peine d'élargir la notion de « normal » au point de masquer les défauts recherchés.

Point de réflexion (rotation forte, 180°) : le pipeline retenu limite la rotation à $\pm 15^\circ$ et exclut délibérément les rotations proches de 180° (cf. ci-dessus, risque d'élargir excessivement la notion de « normal »). Cette exclusion repose sur une hypothèse implicite — que la pilule sort toujours de la chaîne

de fabrication dans une orientation à peu près constante — qui n'a pas été vérifiée. **À valider avec le métier** : si le poste d'inspection peut réellement recevoir des pilules présentées à 180° (convoyeur ou système d'alimentation qui ne garantit pas une orientation unique), alors ce ne serait pas une aberration mais une variation réelle du process. Une rotation à 180°, contrairement à un flip, ne miroir pas la gravure (elle reste lisible, juste tête en bas) : elle n'a donc pas le défaut rédhibitoire identifié plus haut pour les flips. Si le métier confirme que cette orientation existe en sortie de chaîne, il faudrait reconsidérer la limite de $\pm 15^\circ$ — quitte à accepter le compromis avec le risque de masquage de défauts déjà discuté, à arbitrer sur la base de la réalité du poste d'inspection plutôt que sur un critère statistique seul.

Étape 3 — Conception de l'auto-encodeur (pill)

TP B6, partie 2. Construire un auto-encodeur CNN (3 conv encodeur + 3 déconv décodeur, sortie sigmoid), entraîné uniquement sur les pièces saines : ce qu'il reconstruit mal devient suspect.

In [1]:

```
from pathlib import Path from IPython.display import Markdown, display from
indusense.vision.model import build_autoencoder, compression_ratio IMAGE_SIZE = (256, 256)
FIGURES_DIR = Path("../reports/figures") FIGURES_DIR.mkdir(parents=True, exist_ok=True)
```

Décision — taille du goulot d'étranglement

Décision : un goulot à **16 canaux** sur une grille spatiale 32×32 (après 3 convolutions stride-2 sur une entrée 256×256), soit un ratio de compression visé d'environ **×12**.

Pourquoi : l'énoncé identifie un piège central — si le ratio de compression est proche de 1, l'auto-encodeur peut apprendre une quasi-**identité** et reconstruire aussi bien les défauts que les pièces saines, ce qui annule le signal d'anomalie (l'erreur de reconstruction ne distingue plus rien). À l'inverse, un goulot trop serré perd trop d'information et dégrade la reconstruction même des pièces saines. On vise ici un ratio nettement supérieur à 1 (×12) pour donner à l'auto-encodeur une réelle contrainte de compression, en pariant sur une meilleure séparation sain/défaut — au prix d'un compromis à vérifier empiriquement dans les notebooks suivants (l'AUROC de l'étape 6 tranchera si ce choix était le bon).

In [2]:

```
model = build_autoencoder(image_size=IMAGE_SIZE, latent_channels=16) model.summary()
```

Model: "pill_autoencoder"

Layer (type)	Output Shape	Param #
image (InputLayer)	(None, 256, 256, 3)	0
enc_conv1 (Conv2D)	(None, 128, 128, 32)	896
enc_conv2 (Conv2D)	(None, 64, 64, 64)	18,496
latent (Conv2D)	(None, 32, 32, 16)	9,232
dec_deconv1 (Conv2DTranspose)	(None, 64, 64, 64)	9,280
dec_deconv2 (Conv2DTranspose)	(None, 128, 128, 32)	18,464
reconstruction (Conv2DTranspose)	(None, 256, 256, 3)	867

Total params: 57,235 (223.57 KB)

Trainable params: 57,235 (223.57 KB)

Non-trainable params: 0 (0.00 B)

Livrable — nombre de paramètres, goulot et ratio de compression¶

In [3]:

```
ratio = compression_ratio(model) latent_shape =  
tuple(model.get_layer("latent").output.shape[1:]) n_input = IMAGE_SIZE[0] * IMAGE_SIZE[1] * 3  
n_latent = latent_shape[0] * latent_shape[1] * latent_shape[2] n_params =  
model.count_params() lines = [ f"- **Paramètres totaux** : {n_params:,} — modèle  
volontairement petit (227 images " f"saines d'entraînement, pas de quoi nourrir un réseau  
profond).", f"- **Entrée** : {IMAGE_SIZE[0]}×{IMAGE_SIZE[1]}×3 = {n_input:,} valeurs.", f"-  
**Goulot d'étranglement (`latent`)** : {latent_shape[0]}×{latent_shape[1]}×"  
f"{latent_shape[2]} = {n_latent:,} valeurs.", f"- **Ratio de compression** : entrée ÷ latent  
= ×{ratio:.1f}." , ] display(Markdown(chr(10).join(lines)))
```

- **Paramètres totaux** : 57,235 — modèle volontairement petit (227 images saines d'entraînement, pas de quoi nourrir un réseau profond).
- **Entrée** : $256 \times 256 \times 3 = 196,608$ valeurs.
- **Goulot d'étranglement (**latent**)** : $32 \times 32 \times 16 = 16,384$ valeurs.
- **Ratio de compression** : $\text{entrée} \div \text{latent} = \times 12.0$.

Point de réflexion — le piège de la quasi-identité¶

Un ratio proche de **×1** signifierait que le goulot contient presque autant d'information que l'image d'origine : l'auto-encodeur n'a alors aucune raison d'apprendre une représentation compacte du « normal », il peut se contenter de recopier l'entrée (à la précision des convolutions près) — y compris les défauts, qui seraient alors reconstruits presque aussi bien que le reste. L'erreur de reconstruction cesserait d'être un signal utile.

Avec **×12**, le modèle est forcé de résumer chaque image en une fraction de son information — mais la grille latente reste spatialement large (32×32), donc une partie non négligeable de la structure locale est préservée malgré tout. Ce n'est donc pas une garantie absolue contre le piège, seulement un choix qui le rend moins probable qu'un ratio proche de 1. Deux leviers restent disponibles si l'évaluation (étape 6) montre une séparation sain/défaut insuffisante : resserrer encore le goulot (moins de canaux latents), ou passer à une perte **SSIM** plus sensible aux altérations de texture qu'une simple erreur pixel à pixel (MSE).

Étape 4 — Entraînement de l'auto-encodeur (pill)

TP B6, partie 2. Entraîner sur les seules pièces saines (cible = entrée), suivi MLflow.

In [1]:

```
from pathlib import Path import matplotlib.pyplot as plt from IPython.display import
Markdown, display from indusense.vision.model import build_autoencoder from
indusense.vision.train import MODEL_PATH, build_training_set, save_model, train_autoencoder
IMAGE_SIZE = (256, 256) FIGURES_DIR = Path("../reports/figures")
FIGURES_DIR.mkdir(parents=True, exist_ok=True)
```

Décision — hyperparamètres d'entraînement

Décision : perte **MSE** pour le run retenu (celui sauvegardé et utilisé dans les notebooks suivants), un second run en perte **SSIM** à titre de comparaison. Train augmenté ×5 (image originale + 4 versions augmentées par le pipeline de la partie 1), validation saine non augmentée. **EarlyStopping** sur `val_loss` plutôt qu'un nombre d'epochs fixe.

Pourquoi MSE plutôt que SSIM comme run retenu

L'argument décisif — cohérence entre ce que le modèle optimise et ce qu'on va mesurer ensuite.

Le score d'anomalie qui sert dans tout le reste du TP (notebook 05, `anomaly.py`) est l'erreur de reconstruction **MSE** moyennée sur l'image. Ce score n'existe pas encore ici — seule la perte d'entraînement (MSE ou SSIM selon le run) est calculée dans ce notebook — mais on sait déjà qu'il sera basé sur la MSE, donc on peut anticiper quel modèle lui sera cohérent :

- un modèle **entraîné avec la perte MSE** est *directement* optimisé pour minimiser cette même quantité sur les pièces saines : la fonction utilisée ensuite pour distinguer sain/défaut est exactement celle que l'entraînement a cherché à réduire — pas de décalage ;
- un modèle **entraîné avec la perte SSIM** optimise un critère différent (similarité structurelle, volontairement insensible aux décalages de luminance/contraste) — rien ne garantit qu'un tel modèle ait, en prime, une MSE de reconstruction basse et bien calibrée sur les saines. Le retenir comme modèle final introduirait un vrai décalage : entraîné pour un objectif, mesuré sur un autre.

Le run MSE n'a structurellement pas ce problème, puisqu'il optimise exactement ce qu'on mesurera après.

Deux arguments secondaires, qui affaiblissent l'intérêt de SSIM ici sans l'invalider en principe :

1. **L'avantage théorique de SSIM se dilue dans notre pipeline** : SSIM est en principe plus sensible aux altérations de texture locale (pertinent pour des défauts comme crack/scratch) — mais le score d'anomalie (`image_scores`, calculé au notebook 05) moyenne l'erreur sur l'image **entière** (256×256), pas sur des fenêtres locales. À ce niveau d'agrégation globale, un défaut ponctuel pèse peu dans une moyenne sur 65 536 pixels, quelle que soit la perte utilisée à l'entraînement — le notebook 05 le confirme : le chevauchement sain/défaut observé montre que beaucoup de petits défauts ne remontent pas au niveau du score global. SSIM ne montrerait tout son intérêt exploité en carte locale par fenêtre, pas comme simple substitut de la perte globale.

2. **Coût mesuré, pas seulement anticipé** : une fois les deux runs exécutés, le run SSIM a pris 1658 s contre 1085 s pour MSE (+53 %) pour un nombre d'epochs comparable — un différentiel réel qui pèse en faveur de MSE comme run par défaut si l'on doit réentraîner régulièrement.

Conclusion : SSIM n'est pas rejeté — l'énoncé demande explicitement de l'essayer, et le run est conservé comme point de comparaison documenté (utile pour juger si son coût supplémentaire se justifierait avec un score exploitant des cartes d'erreur locales plutôt qu'une moyenne globale). Mais ce n'est pas le modèle retenu, précisément parce que MSE est le seul choix qui garantit l'alignement entre ce que le modèle apprend et ce qu'on mesure ensuite pour détecter une anomalie.

Pourquoi les autres choix¶

- **Augmentation x5** : le jeu de saines est minuscule (227 images après réservation de la validation) — l'augmenter artificiellement limite le risque que le modèle apprenne par cœur ces 227 exemples plutôt qu'une notion générale de « normal ».
- **EarlyStopping** : avec un jeu aussi petit, le nombre d'epochs optimal n'est pas connu à l'avance ; s'arrêter sur `val_loss` évite de choisir arbitrairement.

In [2]:

```
x_train, x_val = build_training_set(image_size=IMAGE_SIZE, n_aug=4) model =
build_autoencoder(image_size=IMAGE_SIZE, latent_channels=16) lines = [ f"- **Train (avec
augmentation)** : {x_train.shape[0]} images.", f"- **Validation (saines, non augmentées)** :
{x_val.shape[0]} images.", ] display(Markdown(chr(10).join(lines)))
```

- **Train (avec augmentation)** : 1135 images.
- **Validation (saines, non augmentées)** : 40 images.

In [3]:

```
model, history, duration = train_autoencoder( model, x_train, x_val, loss="mse", epochs=50,
batch_size=32, learning_rate=1e-3, run_name="autoencoder-mse", log_mlflow=True, )
display(Markdown(f"- **Durée d'entraînement (run MSE)** : {duration:.1f} s."))
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow
>= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the
TensorFlow-DirectML plugin.
```

Epoch 1/50

36/36 - 22s - 619ms/step - loss: 0.1127 - val_loss: 0.0738

Epoch 2/50

36/36 - 22s - 599ms/step - loss: 0.0278 - val_loss: 0.0063

Epoch 3/50

36/36 - 21s - 571ms/step - loss: 0.0047 - val_loss: 0.0034

Epoch 4/50

36/36 - 21s - 589ms/step - loss: 0.0031 - val_loss: 0.0024

Epoch 5/50

36/36 - 21s - 575ms/step - loss: 0.0021 - val_loss: 0.0015

Epoch 6/50

36/36 - 22s - 600ms/step - loss: 0.0015 - val_loss: 0.0011

Epoch 7/50

36/36 - 22s - 615ms/step - loss: 0.0011 - val_loss: 9.0232e-04

Epoch 8/50

36/36 - 23s - 629ms/step - loss: 9.2788e-04 - val_loss: 8.1481e-04

Epoch 9/50

36/36 - 20s - 563ms/step - loss: 8.2232e-04 - val_loss: 7.0998e-04

Epoch 10/50

36/36 - 21s - 590ms/step - loss: 7.8199e-04 - val_loss: 6.5263e-04

Epoch 11/50

36/36 - 21s - 578ms/step - loss: 7.2469e-04 - val_loss: 6.6711e-04

Epoch 12/50

36/36 - 21s - 578ms/step - loss: 7.2205e-04 - val_loss: 5.7390e-04

Epoch 13/50

36/36 - 21s - 576ms/step - loss: 6.5574e-04 - val_loss: 6.2777e-04

Epoch 14/50

36/36 - 21s - 593ms/step - loss: 6.2778e-04 - val_loss: 5.3307e-04

Epoch 15/50

36/36 - 20s - 550ms/step - loss: 6.0137e-04 - val_loss: 5.1706e-04

Epoch 16/50

36/36 - 20s - 566ms/step - loss: 5.9959e-04 - val_loss: 5.1003e-04

Epoch 17/50

36/36 - 23s - 629ms/step - loss: 5.4459e-04 - val_loss: 5.3887e-04

Epoch 18/50

36/36 - 24s - 664ms/step - loss: 5.3953e-04 - val_loss: 4.3881e-04

Epoch 19/50

36/36 - 21s - 577ms/step - loss: 5.2100e-04 - val_loss: 4.3345e-04

Epoch 20/50

36/36 - 21s - 591ms/step - loss: 5.0845e-04 - val_loss: 4.6953e-04

Epoch 21/50

36/36 - 21s - 577ms/step - loss: 4.9136e-04 - val_loss: 4.0200e-04

Epoch 22/50

36/36 - 23s - 626ms/step - loss: 4.7604e-04 - val_loss: 5.4112e-04

Epoch 23/50

36/36 - 25s - 700ms/step - loss: 5.3888e-04 - val_loss: 3.7549e-04

Epoch 24/50

36/36 - 23s - 649ms/step - loss: 4.3687e-04 - val_loss: 3.6560e-04

Epoch 25/50

36/36 - 25s - 702ms/step - loss: 4.1385e-04 - val_loss: 4.1896e-04

Epoch 26/50

36/36 - 22s - 609ms/step - loss: 5.2886e-04 - val_loss: 3.9072e-04

Epoch 27/50

36/36 - 27s - 745ms/step - loss: 4.1311e-04 - val_loss: 3.4328e-04

Epoch 28/50

36/36 - 25s - 685ms/step - loss: 4.0561e-04 - val_loss: 4.0303e-04

Epoch 29/50

36/36 - 25s - 681ms/step - loss: 4.0172e-04 - val_loss: 3.2548e-04

Epoch 30/50

36/36 - 26s - 716ms/step - loss: 4.0564e-04 - val_loss: 3.6536e-04

Epoch 31/50

36/36 - 24s - 678ms/step - loss: 3.8010e-04 - val_loss: 3.4617e-04

Epoch 32/50

36/36 - 23s - 649ms/step - loss: 3.6689e-04 - val_loss: 4.1190e-04

Epoch 33/50

36/36 - 24s - 667ms/step - loss: 3.8558e-04 - val_loss: 3.5830e-04

Epoch 34/50

36/36 - 23s - 651ms/step - loss: 3.7051e-04 - val_loss: 2.9463e-04

Epoch 35/50

36/36 - 20s - 563ms/step - loss: 3.6673e-04 - val_loss: 3.1865e-04

Epoch 36/50

36/36 - 20s - 556ms/step - loss: 3.7824e-04 - val_loss: 3.9913e-04

Epoch 37/50

36/36 - 19s - 536ms/step - loss: 3.4384e-04 - val_loss: 2.8310e-04

Epoch 38/50

36/36 - 20s - 562ms/step - loss: 3.6797e-04 - val_loss: 2.7453e-04

Epoch 39/50

36/36 - 20s - 543ms/step - loss: 3.1394e-04 - val_loss: 2.6927e-04

Epoch 40/50

36/36 - 20s - 559ms/step - loss: 3.9826e-04 - val_loss: 3.6531e-04

Epoch 41/50

36/36 - 20s - 553ms/step - loss: 3.3368e-04 - val_loss: 2.6146e-04

Epoch 42/50

36/36 - 19s - 527ms/step - loss: 3.1244e-04 - val_loss: 2.8130e-04

Epoch 43/50

36/36 - 20s - 545ms/step - loss: 3.1045e-04 - val_loss: 3.0515e-04

Epoch 44/50

36/36 - 21s - 595ms/step - loss: 3.1328e-04 - val_loss: 4.3984e-04

Epoch 45/50

36/36 - 20s - 547ms/step - loss: 3.6009e-04 - val_loss: 2.7253e-04

Epoch 46/50

36/36 - 20s - 565ms/step - loss: 3.0742e-04 - val_loss: 2.5446e-04

Epoch 47/50

36/36 - 19s - 541ms/step - loss: 2.8610e-04 - val_loss: 2.4130e-04

Epoch 48/50

36/36 - 20s - 543ms/step - loss: 3.0454e-04 - val_loss: 2.7681e-04

Epoch 49/50

36/36 - 20s - 558ms/step - loss: 3.0367e-04 - val_loss: 2.5277e-04

Epoch 50/50

36/36 - 23s - 635ms/step - loss: 3.0842e-04 - val_loss: 2.6205e-04

2026/07/21 14:51:43 WARNING mlflow.tensorflow: You are saving a TensorFlow Core model or Keras model without a signature. Inference with mlflow.pyfunc.spark_udf() will not work unless the model's pyfunc representation accepts pandas DataFrames as inference inputs.

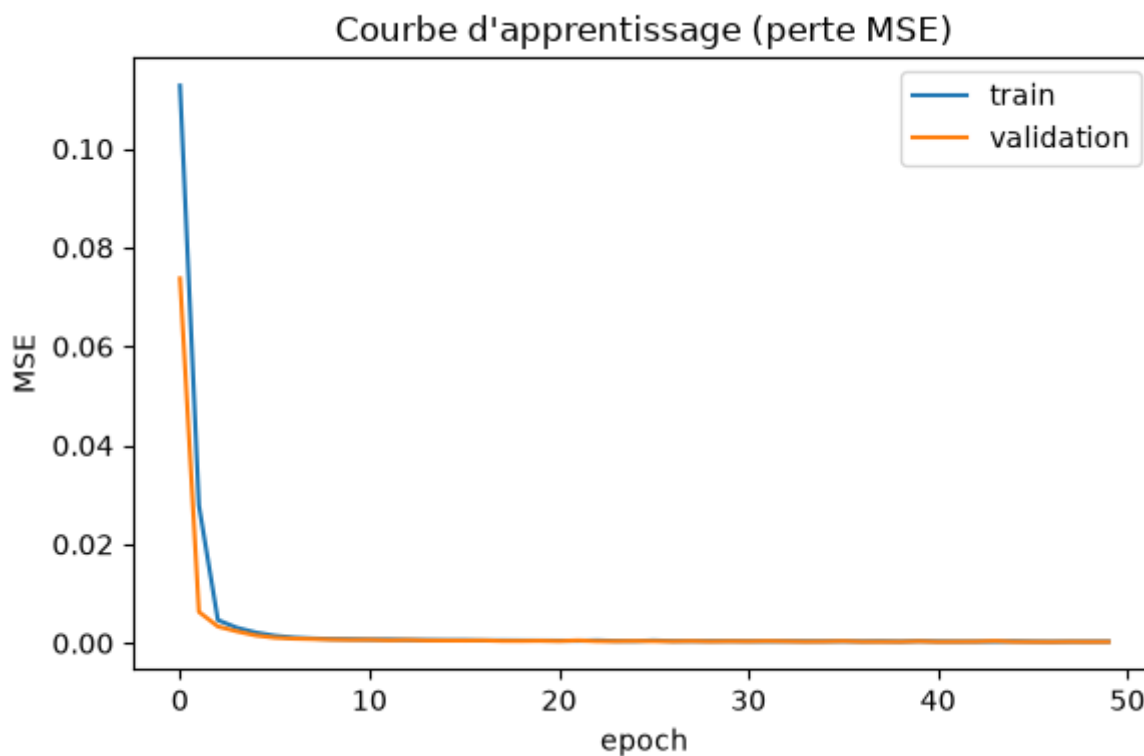
2026/07/21 14:52:19 WARNING mlflow.utils.environment: Failed to resolve installed pip version. ``pip`` will be added to conda.yaml environment spec without a version specifier.

- **Durée d'entraînement (run MSE) : 1085.0 s.**

Livrable — courbe d'apprentissage¶

In [4]:

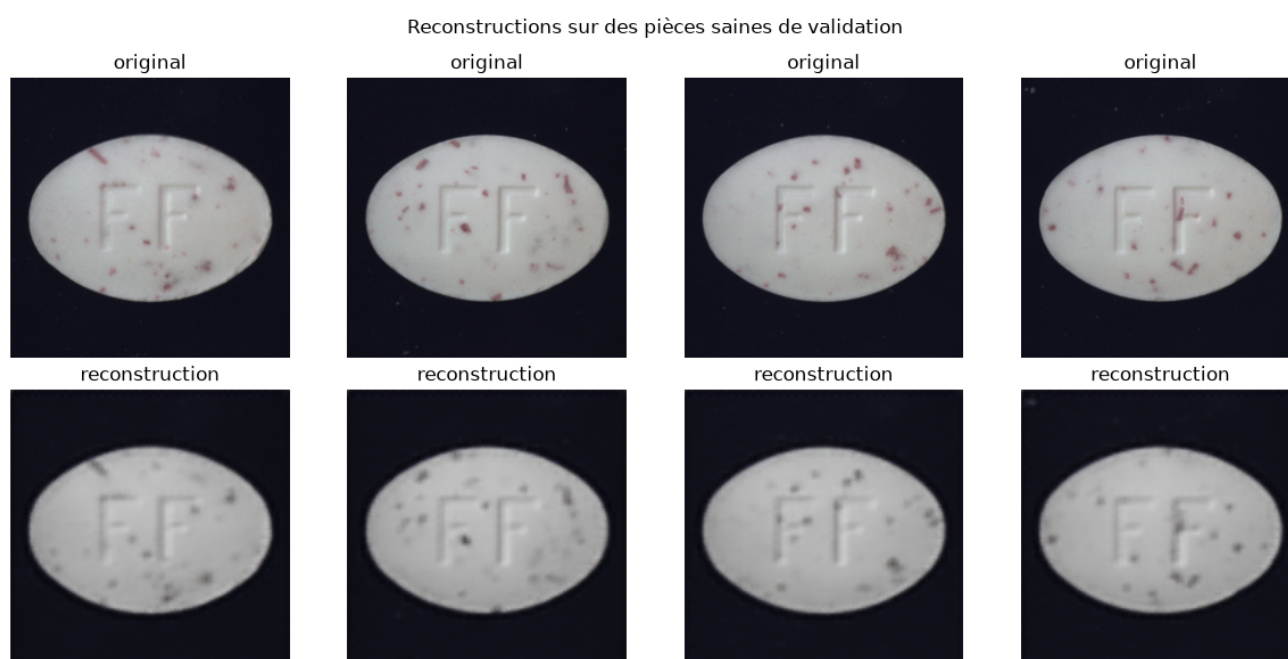
```
fig, ax = plt.subplots(figsize=(6, 4)) ax.plot(history.history["loss"], label="train")
ax.plot(history.history["val_loss"], label="validation") ax.set_xlabel("epoch")
ax.set_ylabel("MSE") ax.set_title("Courbe d'apprentissage (perte MSE)") ax.legend()
fig.tight_layout() fig.savefig(FIGURES_DIR / "04_courbes_apprentissage.png", dpi=150)
plt.show()
```



Livrable — exemples de reconstruction (saines de validation)¶

In [5]:

```
n_examples = 4 sample = x_val[:n_examples] reconstructions = model.predict(sample, verbose=0)
fig, axes = plt.subplots(2, n_examples, figsize=(3 * n_examples, 6)) for i in
range(n_examples): axes[0, i].imshow(sample[i]) axes[0, i].set_title("original") axes[0,
i].axis("off") axes[1, i].imshow(reconstructions[i]) axes[1, i].set_title("reconstruction")
axes[1, i].axis("off") fig.suptitle("Reconstructions sur des pièces saines de validation")
fig.tight_layout() fig.savefig(FIGURES_DIR / "04_reconstructions.png", dpi=150) plt.show()
```



Décision — comparaison avec une perte SSIM¶

Décision : entraîner un second modèle, architecture identique, perte SSIM au lieu de MSE, journalisé dans un run MLflow séparé (`autoencoder-ssim`) — sans remplacer le modèle retenu pour la suite du TP.

Pourquoi : l'énoncé invite explicitement à « essayer aussi SSIM ». Les valeurs de perte MSE et `1 - SSIM` ne vivent pas sur la même échelle : les comparer directement n'a pas de sens. Ce qui est interprétable ici, c'est l'écart train/val **au sein** de chaque run (indice de sur-apprentissage), pas une comparaison de valeur absolue entre les deux pertes. Un arbitrage définitif MSE vs SSIM se ferait sur la capacité de détection réelle (AUROC, étape 6), pas sur la courbe de perte.

In [6]:

```
model_ssim = build_autoencoder(image_size=IMAGE_SIZE, latent_channels=16) model_ssim,
history_ssim, duration_ssim = train_autoencoder( model_ssim, x_train, x_val, loss="ssim",
epochs=50, batch_size=32, learning_rate=1e-3, run_name="autoencoder-ssim", log_mlflow=True, )
lines = [ f"- **Run MSE** : perte train={history.history['loss'][-1]:.4f}, "
f"val={history.history['val_loss'][-1]:.4f}, durée={duration:.1f} s.", f"- **Run SSIM** :
perte (`1 - SSIM`) train={history_ssim.history['loss'][-1]:.4f}, "
f"val={history_ssim.history['val_loss'][-1]:.4f}, durée={duration_ssim:.1f} s.", "- Ces deux
valeurs de perte ne sont **pas comparables entre elles** (échelles " "différentes) – seul
l'écart train/val de chaque run est interprétable ici. La " "***durée**", elle, est directement
comparable entre les deux runs.", ] display(Markdown(chr(10).join(lines)))
```

Epoch 1/50

36/36 - 34s - 941ms/step - loss: 0.5479 - val_loss: 0.5489

Epoch 2/50

36/36 - 32s - 876ms/step - loss: 0.5167 - val_loss: 0.4814

Epoch 3/50

36/36 - 34s - 945ms/step - loss: 0.4120 - val_loss: 0.1754

Epoch 4/50

36/36 - 34s - 947ms/step - loss: 0.1550 - val_loss: 0.0767

Epoch 5/50

36/36 - 32s - 878ms/step - loss: 0.0762 - val_loss: 0.0599

Epoch 6/50

36/36 - 33s - 904ms/step - loss: 0.0641 - val_loss: 0.0531

Epoch 7/50

36/36 - 32s - 887ms/step - loss: 0.0593 - val_loss: 0.0502

Epoch 8/50

36/36 - 32s - 887ms/step - loss: 0.0548 - val_loss: 0.0474

Epoch 9/50

36/36 - 33s - 904ms/step - loss: 0.0517 - val_loss: 0.0453

Epoch 10/50

36/36 - 35s - 959ms/step - loss: 0.0486 - val_loss: 0.0435

Epoch 11/50

36/36 - 35s - 967ms/step - loss: 0.0462 - val_loss: 0.0445

Epoch 12/50

36/36 - 35s - 971ms/step - loss: 0.0440 - val_loss: 0.0411

Epoch 13/50

36/36 - 35s - 964ms/step - loss: 0.0422 - val_loss: 0.0405

Epoch 14/50

36/36 - 32s - 891ms/step - loss: 0.0406 - val_loss: 0.0393

Epoch 15/50

36/36 - 37s - 1s/step - loss: 0.0394 - val_loss: 0.0390

Epoch 16/50

36/36 - 40s - 1s/step - loss: 0.0378 - val_loss: 0.0401

Epoch 17/50

36/36 - 39s - 1s/step - loss: 0.0368 - val_loss: 0.0374

Epoch 18/50

36/36 - 36s - 1s/step - loss: 0.0358 - val_loss: 0.0363

Epoch 19/50

36/36 - 37s - 1s/step - loss: 0.0350 - val_loss: 0.0362

Epoch 20/50

36/36 - 39s - 1s/step - loss: 0.0340 - val_loss: 0.0357

Epoch 21/50

36/36 - 38s - 1s/step - loss: 0.0331 - val_loss: 0.0350

Epoch 22/50

36/36 - 35s - 970ms/step - loss: 0.0326 - val_loss: 0.0358

Epoch 23/50

36/36 - 33s - 923ms/step - loss: 0.0323 - val_loss: 0.0344

Epoch 24/50

36/36 - 35s - 961ms/step - loss: 0.0316 - val_loss: 0.0340

Epoch 25/50

36/36 - 38s - 1s/step - loss: 0.0309 - val_loss: 0.0357

Epoch 26/50

36/36 - 34s - 958ms/step - loss: 0.0307 - val_loss: 0.0332

Epoch 27/50

36/36 - 33s - 912ms/step - loss: 0.0302 - val_loss: 0.0332

Epoch 28/50

36/36 - 38s - 1s/step - loss: 0.0300 - val_loss: 0.0351

Epoch 29/50

36/36 - 35s - 984ms/step - loss: 0.0293 - val_loss: 0.0322

Epoch 30/50

36/36 - 39s - 1s/step - loss: 0.0289 - val_loss: 0.0326

Epoch 31/50

36/36 - 38s - 1s/step - loss: 0.0287 - val_loss: 0.0326

Epoch 32/50

36/36 - 37s - 1s/step - loss: 0.0286 - val_loss: 0.0319

Epoch 33/50

36/36 - 35s - 963ms/step - loss: 0.0283 - val_loss: 0.0314

Epoch 34/50

36/36 - 31s - 874ms/step - loss: 0.0277 - val_loss: 0.0315

Epoch 35/50

36/36 - 32s - 897ms/step - loss: 0.0273 - val_loss: 0.0337

Epoch 36/50

36/36 - 33s - 920ms/step - loss: 0.0273 - val_loss: 0.0306

Epoch 37/50

36/36 - 32s - 878ms/step - loss: 0.0267 - val_loss: 0.0314

Epoch 38/50

36/36 - 30s - 834ms/step - loss: 0.0269 - val_loss: 0.0337

Epoch 39/50

36/36 - 27s - 762ms/step - loss: 0.0267 - val_loss: 0.0303

Epoch 40/50

36/36 - 28s - 779ms/step - loss: 0.0260 - val_loss: 0.0314

Epoch 41/50

36/36 - 30s - 833ms/step - loss: 0.0261 - val_loss: 0.0297

Epoch 42/50

36/36 - 28s - 771ms/step - loss: 0.0256 - val_loss: 0.0305

Epoch 43/50

36/36 - 28s - 776ms/step - loss: 0.0257 - val_loss: 0.0298

Epoch 44/50

36/36 - 27s - 762ms/step - loss: 0.0254 - val_loss: 0.0294

Epoch 45/50

36/36 - 27s - 760ms/step - loss: 0.0253 - val_loss: 0.0294

Epoch 46/50

36/36 - 27s - 753ms/step - loss: 0.0251 - val_loss: 0.0292

Epoch 47/50

36/36 - 29s - 794ms/step - loss: 0.0249 - val_loss: 0.0293

Epoch 48/50

```
36/36 - 27s - 760ms/step - loss: 0.0246 - val_loss: 0.0289
```

Epoch 49/50

```
36/36 - 30s - 825ms/step - loss: 0.0246 - val_loss: 0.0287
```

Epoch 50/50

```
36/36 - 29s - 813ms/step - loss: 0.0245 - val_loss: 0.0294
```

```
2026/07/21 15:20:12 WARNING mlflow.tensorflow: You are saving a TensorFlow Core model or Keras model without a signature. Inference with mlflow.pyfunc.spark_udf() will not work unless the model's pyfunc representation accepts pandas DataFrames as inference inputs.
```

```
2026/07/21 15:20:56 WARNING mlflow.utils.environment: Encountered an unexpected error while inferring pip requirements (model URI: C:\Users\patri\AppData\Local\Temp\tmpbfetvn0l\model, flavor: tensorflow). Fall back to return ['tensorflow==2.21.0', 'cloudpickle==3.1.2']. Set logging level to DEBUG to see the full traceback.
```

```
2026/07/21 15:20:56 WARNING mlflow.utils.environment: Failed to resolve installed pip version. ``pip`` will be added to conda.yaml environment spec without a version specifier.
```

- **Run MSE** : perte train=0.0003, val=0.0003, durée=1085.0 s.
- **Run SSIM** : perte (1 - SSIM) train=0.0245, val=0.0294, durée=1658.0 s.
- Ces deux valeurs de perte ne sont **pas comparables entre elles** (échelles différentes) — seul l'écart train/val de chaque run est interprétable ici. La **durée**, elle, est directement comparable entre les deux runs.

Livraison — sauvegarde du modèle retenu (perte MSE)¶

In [7]:

```
save_model(model) display(Markdown(f"- **Modèle sauvegardé** : `{MODEL_PATH}`"))
```

- **Modèle sauvegardé** :
D:\ProjetsIA\AELION_Deep_Learning_IA\artifacts\ml\models\autoencoder_pill.keras

Étape 5 — Score d'anomalie et calibration du seuil (pill)

TP B6, partie 2. Utilise le modèle entraîné (notebook 04) pour calculer un score d'anomalie par image, calibrer un seuil de détection sur les pièces saines de validation, puis évaluer ce score sur l'ensemble du jeu de test (saines + tous les défauts).

In [1]:

```
from pathlib import Path import matplotlib.pyplot as plt import numpy as np from
IPython.display import Markdown, display from indusense.vision.anomaly import (
calibrate_threshold, confusion, error_maps, image_auroc, image_scores, reconstruct, ) from
indusense.vision.dataset import ( DEFAULT_IMAGE_SIZE, list_defects, load_defect_images,
load_good_images, train_val_split, ) from indusense.vision.train import load_trained_model
IMAGE_SIZE = DEFAULT_IMAGE_SIZE FIGURES_DIR = Path("../reports/figures")
FIGURES_DIR.mkdir(parents=True, exist_ok=True) model = load_trained_model()
display(Markdown(f"- **Modèle chargé** : {model.count_params()} paramètres."))
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow
>= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the
TensorFlow-DirectML plugin.
```

- **Modèle chargé** : 57235 paramètres.

Décision — reconstituer le split de validation

Décision : recharger les pièces saines d'entraînement et refaire le split train/validation avec exactement les mêmes paramètres qu'au notebook 04 (`val_fraction=0.15`, `seed=42`), sans régénérer les versions augmentées.

Pourquoi : le seuil doit être calibré sur des images que le modèle n'a **jamais** vues à l'entraînement — c'est le rôle de `val_good` défini en partie 1 et repris par `build_training_set`. Réutiliser directement `build_training_set` ici régénérerait aussi les variantes augmentées de chaque image d'entraînement (coûteux et inutile ici seul `val_good` sert dans ce notebook). Reproduire le split avec la même graine donne exactement le même ensemble de validation, sans ce coût.

In [2]:

```
good_train = load_good_images("train", IMAGE_SIZE) _train_raw, val_good =
train_val_split(good_train, val_fraction=0.15, seed=42) display( Markdown( f"- **Saines de
validation** : {val_good.shape[0]} images " "(jamais vues à l'entraînement)." ) )
```

- **Saines de validation** : 40 images (jamais vues à l'entraînement).

Définition — le score d'anomalie

C'est ici, pour la première fois dans le TP, que le score d'anomalie est calculé (il n'existe pas encore au notebook 04 — seule la perte d'entraînement, MSE ou SSIM selon le run, y est calculée). Il est défini dans `anomaly.py` en deux temps :

- **`error_maps(images, reconstructions)`** : erreur quadratique (MSE) pixel à pixel entre l'image originale et sa reconstruction, moyennée sur les 3 canaux couleur → une **carte d'erreur** par image (une valeur par pixel).
- **`image_scores(maps)`** : moyenne de cette carte sur tous les pixels de l'image → un **score scalaire unique** par image, comparable d'une image à l'autre.

Pourquoi l'erreur de reconstruction sert de score d'anomalie (rappel du principe posé au notebook 03) : le modèle n'a été entraîné que sur des pièces **saines** (notebook 04) — il a appris à bien reconstruire ce qui est normal, et seulement cela. Une pièce saine de test, jamais vue mais similaire à ce qui a été appris, devrait donc produire une erreur de reconstruction faible ; une pièce défectueuse, qui contient une zone que le modèle n'a jamais appris à reconstruire, devrait produire une erreur plus élevée localement — et donc, une fois moyennée, un score plus élevé. Ce raisonnement est la prémisse même de toute l'approche par auto-encodeur : le score n'est **pas** une prédiction directe (le modèle ne classe rien), c'est une mesure indirecte de « à quel point cette image ressemble à ce que le modèle a appris être normal ».

Décision — calibrer le seuil sur les saines de validation uniquement¶

Décision : calculer le score d'anomalie (erreur de reconstruction moyenne) sur `val_good`, puis calibrer le seuil avec les deux méthodes proposées par l'énoncé : percentile 99 (retenu comme seuil principal) et moyenne + 3 écarts-types (calculée à titre de comparaison).

Pourquoi : calibrer sur le test reviendrait à choisir un seuil avec les données qu'on évalue ensuite (fuite). Le percentile 99 est retenu comme seuil principal car il ne suppose aucune forme particulière pour la distribution des scores sains — contrairement à moyenne $\pm$ k-écart-type, qui suppose implicitement une distribution proche de la normale, une hypothèse peu justifiée pour une erreur de reconstruction (positive par construction, potentiellement asymétrique).

In [3]:

```
reconstructions_val = reconstruct(model, val_good) maps_val = error_maps(val_good,
reconstructions_val) scores_val = image_scores(maps_val) threshold_percentile =
calibrate_threshold(scores_val, method="percentile", q=99.0) threshold_mean_std =
calibrate_threshold(scores_val, method="mean_std", k=3.0) lines = [ f"- **Score moyen (saines
de validation)** : {scores_val.mean():.5f} " f"(écart-type {scores_val.std():.5f}).", f"-
**Seuil percentile 99** (retenu) : {threshold_percentile:.5f}.", f"- **Seuil moyenne +
3-écart-type** (comparaison) : {threshold_mean_std:.5f}.", ]
display(Markdown(chr(10).join(lines)))
```

- **Score moyen (saines de validation)** : 0.00024 (écart-type 0.00002).
- **Seuil percentile 99** (retenu) : 0.00029.
- **Seuil moyenne + 3-écart-type** (comparaison) : 0.00032.

Décision — jeu de test complet (saines + tous les défauts)¶

Décision : évaluer le score sur `test/good` et sur **toutes** les catégories de défauts (`list_defects()`), avec un label binaire par image (0 = saine, 1 = défaut).

Pourquoi : l'histogramme et l'AUROC n'ont de sens que si l'évaluation couvre l'ensemble des cas rencontrés en production, pas une seule catégorie de défaut choisie arbitrairement.

In [4]:

```
test_good = load_good_images("test", IMAGE_SIZE) defects = list_defects() defect_images =
{defect: load_defect_images(defect, IMAGE_SIZE) for defect in defects} test_images =
np.concatenate([test_good, *defect_images.values()], axis=0) labels = np.concatenate(
[np.zeros(len(test_good))] + [np.ones(len(images)) for images in defect_images.values()] )
lines = [f"- **Saines de test** : {len(test_good)} images."] lines += [f"- **{defect}** :
{len(images)} images." for defect, images in defect_images.items()] lines.append(f"- **Total
test** : {len(test_images)} images ({int(labels.sum())} défauts).")
display(Markdown(chr(10).join(lines)))
```

- **Saines de test** : 26 images.
- **color** : 25 images.
- **combined** : 17 images.
- **contamination** : 21 images.
- **crack** : 26 images.
- **faulty_imprint** : 19 images.
- **pill_type** : 9 images.
- **scratch** : 24 images.
- **Total test** : 167 images (141 défauts).

Livraison — histogramme des scores (saines vs défauts) et seuil

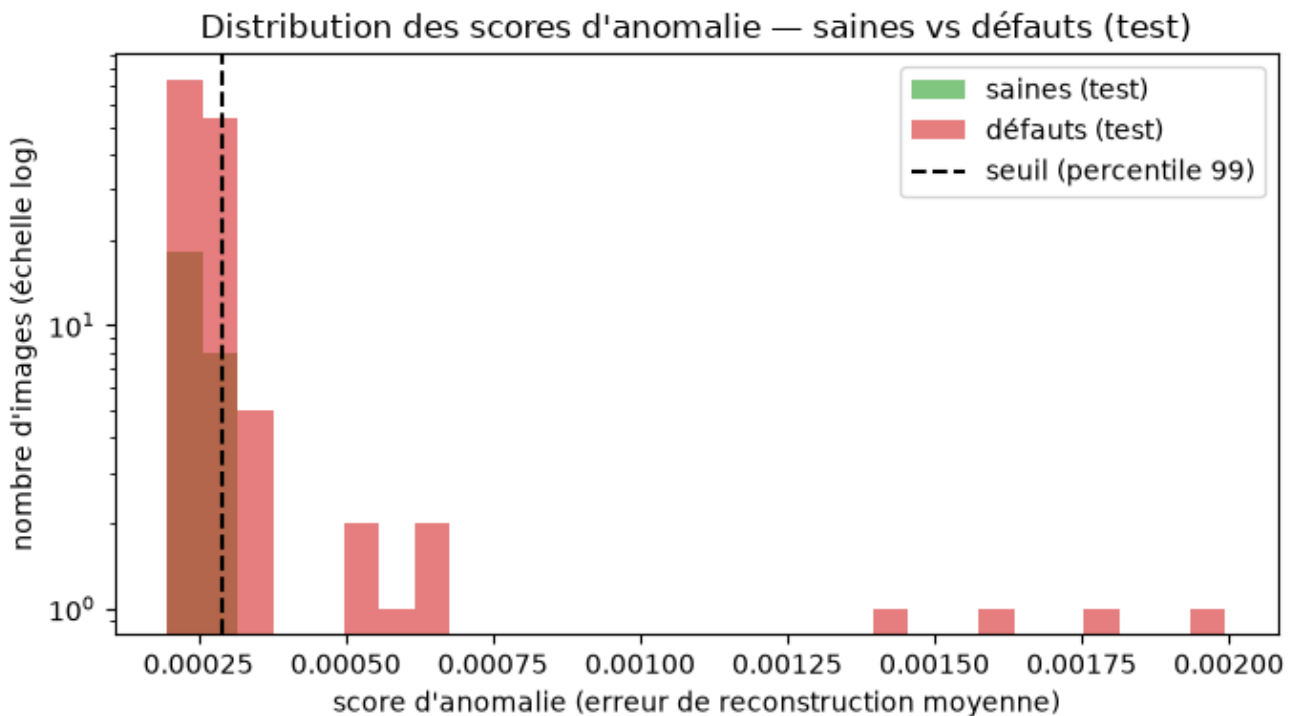
Décision : bins partagés entre les deux séries (calculés sur l'ensemble des scores) et échelle logarithmique en ordonnée.

Pourquoi : avec des bins calculés indépendamment pour chaque série (comportement par défaut de `ax.hist`), les 26 scores saines — concentrés sur une plage très étroite (écart-type 0,00002) — utilisaient des bins bien plus fins que les scores des défauts, étalés sur une plage ~10 fois plus large : les barres saines devenaient trop fines et trop basses pour être visibles à côté des barres rouges, bien plus hautes dans la même zone (une bonne partie des défauts partage le même score que les saines, cf. l'analyse plus bas). Des bins communs rendent la comparaison honnête ; l'échelle logarithmique rend visibles les faibles effectifs (les saines, et la longue traîne de rares défauts à score élevé) sans écraser la lecture du gros de la distribution.

In [5]:

```
reconstructions_test = reconstruct(model, test_images) maps_test = error_maps(test_images,
reconstructions_test) scores_test = image_scores(maps_test) bin_edges =
np.histogram_bin_edges(scores_test, bins=30) fig, ax = plt.subplots(figsize=(7, 4)) ax.hist(
scores_test[labels == 0], bins=bin_edges, alpha=0.6, label="saines (test)", color="tab:green"
) ax.hist( scores_test[labels == 1], bins=bin_edges, alpha=0.6, label="défauts (test)",
color="tab:red" ) ax.axvline(threshold_percentile, color="black", linestyle="--",
```

```
label="seuil (percentile 99)") ax.set_yscale("log") ax.set_xlabel("score d'anomalie (erreur de reconstruction moyenne)") ax.set_ylabel("nombre d'images (échelle log)") ax.set_title("Distribution des scores d'anomalie – saines vs défauts (test)") ax.legend() fig.tight_layout() fig.savefig(FIGURES_DIR / "05_histogramme_scores.png", dpi=150) plt.show()
```



In [6]:

```
frac_defects_overlap = (scores_test[labels == 1] <= threshold_percentile).mean()
frac_healthy_above = (scores_test[labels == 0] > threshold_percentile).mean()
max_score_healthy = scores_test[labels == 0].max() frac_defects_above_healthy_max =
(scores_test[labels == 1] > max_score_healthy).mean() lines = [ f"-
**{frac_defects_overlap:.1%} des défauts** ont un score inférieur ou égal au " "seuil retenu
: leur erreur de reconstruction est du même ordre que celle d'une " "pièce saine – le modèle
les reconstruit trop bien pour les distinguer.", f"- **{frac_healthy_above:.1%} des saines de
test** dépassent ce seuil (aucune " "fausse alerte ici, cohérent avec un seuil très
conservateur au percentile 99).", f"- **{frac_defects_above_healthy_max:.1%} des défauts**
ont un score strictement " f"supérieur au score maximal observé sur les saines de test "
f"({max_score_healthy:.5f}) : ce sont les seuls cas détectables sans ambiguïté, " "quel que
soit le seuil choisi.", ] display(Markdown(chr(10).join(lines)))
```

- **83.0% des défauts** ont un score inférieur ou égal au seuil retenu : leur erreur de reconstruction est du même ordre que celle d'une pièce saine — le modèle les reconstruit trop bien pour les distinguer.
- **0.0% des saines de test** dépassent ce seuil (aucune fausse alerte ici, cohérent avec un seuil très conservateur au percentile 99).
- **27.0% des défauts** ont un score strictement supérieur au score maximal observé sur les saines de test (0.00028) : ce sont les seuls cas détectables sans ambiguïté, quel que soit le seuil choisi.

Lecture de l'histogramme : une large barre commune se concentre sur les scores les plus bas, regroupant la quasi-totalité des saines **et** une majorité des défauts — seule une minorité de défauts s'étend sur une longue traîne de scores nettement plus élevés. C'est ce chevauchement, plutôt que la traîne, qui explique le rappel limité obtenu plus loin au seuil percentile 99 (cf. matrice de confusion).

Pourquoi ce chevauchement : cohérent avec le compromis assumé au notebook 03 en choisissant un ratio de compression serré ($\times 12$) plutôt que le ratio par défaut ($\times 6$) — un bottleneck plus étroit *devrait* forcer le modèle à perdre plus de détails fins et donc moins bien reconstruire les défauts, mais rien ne garantit que cette perte porte spécifiquement sur les zones de défaut plutôt que sur l'image dans son ensemble. Ici, une partie significative des défauts reste suffisamment subtile ou localisée pour ne pas franchir un seuil de reconstruction **global**, moyenné sur toute l'image (256×256 pixels) — un défaut ponctuel dilue peu l'erreur moyenne.

Livrable — AUROC niveau image¶

Mesure indépendante du seuil choisi : la capacité du score à séparer sain/défaut sur l'ensemble des seuils possibles (1.0 = séparation parfaite, 0.5 = aléatoire). Sert de première lecture quantitative de l'histogramme ci-dessus ; le pixel-AUROC et l'analyse critique complète sont traités dans le notebook 06 (heatmaps sur les défauts réels).

In [7]:

```
auroc = image_auroc(scores_test, labels) display(Markdown(f"- **AUROC image-level** : {auroc:.3f}."))
```

- **AUROC image-level** : 0.688.

In [8]:

```
lines = [ f"- **Interprétation** : si on tire au hasard une image saine et une image "
f"défectueuse, l'image défectueuse obtient un score d'anomalie plus élevé dans "
f"{auroc:.1%} des cas (AUROC = {auroc:.3f}).", "- Un AUROC de 0.5 correspondrait à un "
classement aléatoire (aucun pouvoir " "séparateur) ; 1.0 à une séparation parfaite entre "
saines et défauts.", f"- {auroc:.3f} traduit un pouvoir séparateur **réel mais modéré** : le "
score " "distingue mieux que le hasard, mais reste loin d'une séparation fiable pour un "
"usage en production sans seuil soigneusement choisi et contrôle humain en filet " "de "
sécurité – cohérent avec le chevauchement observé sur l'histogramme " "ci-dessus.", ]
display(Markdown(chr(10).join(lines)))
```

- **Interprétation** : si on tire au hasard une image saine et une image défectueuse, l'image défectueuse obtient un score d'anomalie plus élevé dans 68.8% des cas (AUROC = 0.688).
- Un AUROC de 0.5 correspondrait à un classement aléatoire (aucun pouvoir séparateur) ; 1.0 à une séparation parfaite entre saines et défauts.
- 0.688 traduit un pouvoir séparateur **réel mais modéré** : le score distingue mieux que le hasard, mais reste loin d'une séparation fiable pour un usage en production sans seuil soigneusement choisi et contrôle humain en filet de sécurité — cohérent avec le chevauchement observé sur l'histogramme ci-dessus.

Discussion — arbitrage rappel / fausses alertes au seuil retenu¶

Décision : donner un instantané chiffré (rappel, précision, taux de fausses alertes) au seuil percentile 99 pour ancrer la discussion dans des valeurs réelles, plutôt que de la garder purement qualitative.

Pourquoi cet arbitrage compte : en contrôle qualité pharmaceutique, laisser passer une pièce défectueuse (faux négatif) est presque toujours plus coûteux qu'une fausse alerte (faux positif, qui ne

coûte qu'une inspection manuelle supplémentaire). Un seuil plus bas (percentile inférieur à 99, ou un k plus petit pour la méthode moyenne+ k -écart-type) augmenterait le rappel au prix de plus de fausses alertes. Le choix définitif du seuil de production est une décision **métier** (coût d'un défaut non détecté vs coût d'une inspection manuelle), pas seulement statistique — cette cellule ne fait que fournir les chiffres nécessaires à cet arbitrage, pas le trancher. Une matrice de confusion plus détaillée (par catégorie de défaut) est établie au notebook 06.

In [9]:

```
cm = confusion(scores_test, labels, threshold_percentile) tn, fp, fn, tp = cm.ravel() recall = tp / (tp + fn) if (tp + fn) > 0 else float("nan") precision = tp / (tp + fp) if (tp + fp) > 0 else float("nan") false_alarm_rate = fp / (fp + tn) if (fp + tn) > 0 else float("nan") lines = [ f"- **Matrice de confusion** (seuil percentile 99) : " f"VN={tn}, FP={fp}, FN={fn}, VP={tp}." , f"- **Rappel (sensibilité)** : {recall:.1%} des défauts sont détectés." , f"- **Précision** : {precision:.1%} des alertes correspondent à un vrai défaut." , f"- **Taux de fausses alertes** : {false_alarm_rate:.1%} des pièces saines sont " f"signalées à tort." , ] if fn > 0: lines.append( f"- **{fn} défaut(s) non détecté(s)** au seuil actuel – à mettre en balance " "avec le coût métier d'un faux négatif avant de valider ce seuil en production." ) display(Markdown(chr(10).join(lines)))
```

- **Matrice de confusion** (seuil percentile 99) : VN=26, FP=0, FN=117, VP=24.
- **Rappel (sensibilité)** : 17.0% des défauts sont détectés.
- **Précision** : 100.0% des alertes correspondent à un vrai défaut.
- **Taux de fausses alertes** : 0.0% des pièces saines sont signalées à tort.
- **117 défaut(s) non détecté(s)** au seuil actuel — à mettre en balance avec le coût métier d'un faux négatif avant de valider ce seuil en production.

Recommandations¶

- **Seuil de production** : partir du percentile 99 calculé ici, mais le faire valider par le métier à la lumière du rappel/taux de fausses alertes affichés ci-dessus — un seuil plus bas serait probablement préférable si le coût d'un défaut non détecté est élevé (cf. discussion ci-dessus).
- **Recalibrage** : si le procédé de fabrication évolue (nouvel outillage, nouvel éclairage de la caméra), recalibrer le seuil sur un nouvel échantillon de pièces saines plutôt que réutiliser tel quel le seuil ci-dessus.
- **Prochaine étape (notebook 06)** : visualiser les heatmaps d'erreur sur des défauts réels, calculer le pixel-AUROC via les masques de vérité terrain, et discuter les limites observées à la lumière du ratio de compression $\times 12$ retenu au notebook 03.

Étape 6 — Heatmaps et évaluation quantitative (pill)

TP B6, partie 2, dernière étape. Visualise l'erreur de reconstruction sous forme de carte de chaleur sur des défauts réels, la compare aux masques de vérité terrain, puis conclut par une évaluation quantitative complète (AUROC image et pixel, matrice de confusion globale et par catégorie) et une analyse critique du modèle retenu.

In [1]:

```
from pathlib import Path import matplotlib.pyplot as plt import numpy as np from
IPython.display import Markdown, display from sklearn.metrics import roc_curve from
indusense.vision.anomaly import ( calibrate_threshold, confusion, error_maps, heatmap,
image_auroc, image_scores, load_masks, pixel_auroc, reconstruct, ) from
indusense.vision.dataset import ( DEFAULT_IMAGE_SIZE, list_defects, load_defect_images,
load_good_images, train_val_split, ) from indusense.vision.train import load_trained_model
IMAGE_SIZE = DEFAULT_IMAGE_SIZE FIGURES_DIR = Path("../reports/figures")
FIGURES_DIR.mkdir(parents=True, exist_ok=True) model = load_trained_model()
display(Markdown(f"- **Modèle chargé** : {model.count_params()} paramètres."))
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow
>= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the
TensorFlow-DirectML plugin.
```

- **Modèle chargé** : 57235 paramètres.

Décision — recalculer le seuil et le jeu de test, pas les partager entre notebooks

Décision : recalculer ici le seuil de détection (percentile 99 sur les saines de validation) et reconstituer le jeu de test complet, exactement comme au notebook 05, plutôt que d'écrire ces valeurs en dur ou de les faire persister dans un fichier intermédiaire.

Pourquoi : chaque notebook de ce TP est un livrable autonome, exécutable indépendamment (convention établie depuis la partie 1) — dupliquer un court calcul déterministe (même graine, mêmes fonctions `anomaly.py/dataset.py`) est moins fragile que de dépendre d'un fichier produit par un notebook précédent, et garantit que le seuil affiché ici correspond toujours au modèle réellement chargé dans **ce** notebook.

In [2]:

```
good_train = load_good_images("train", IMAGE_SIZE) _train_raw, val_good =
train_val_split(good_train, val_fraction=0.15, seed=42) scores_val =
image_scores(error_maps(val_good, reconstruct(model, val_good))) threshold =
calibrate_threshold(scores_val, method="percentile", q=99.0) display(Markdown(f"- **Seuil de
détection (percentile 99)** : {threshold:.5f}."))
```

- **Seuil de détection (percentile 99)** : 0.00029.

In [3]:

```
test_good = load_good_images("test", IMAGE_SIZE) defects = list_defects() defect_images =
{defect: load_defect_images(defect, IMAGE_SIZE) for defect in defects} test_images =
np.concatenate([test_good, *defect_images.values()], axis=0) labels = np.concatenate(
[np.zeros(len(test_good))] + [np.ones(len(images)) for images in defect_images.values()] )
reconstructions_test = reconstruct(model, test_images) maps_test = error_maps(test_images,
reconstructions_test) scores_test = image_scores(maps_test) display( Markdown( f"- **Jeu de
test** : {len(test_images)} images " f"({len(test_good)} saines, {int(labels.sum())}
défauts)." ) )
```

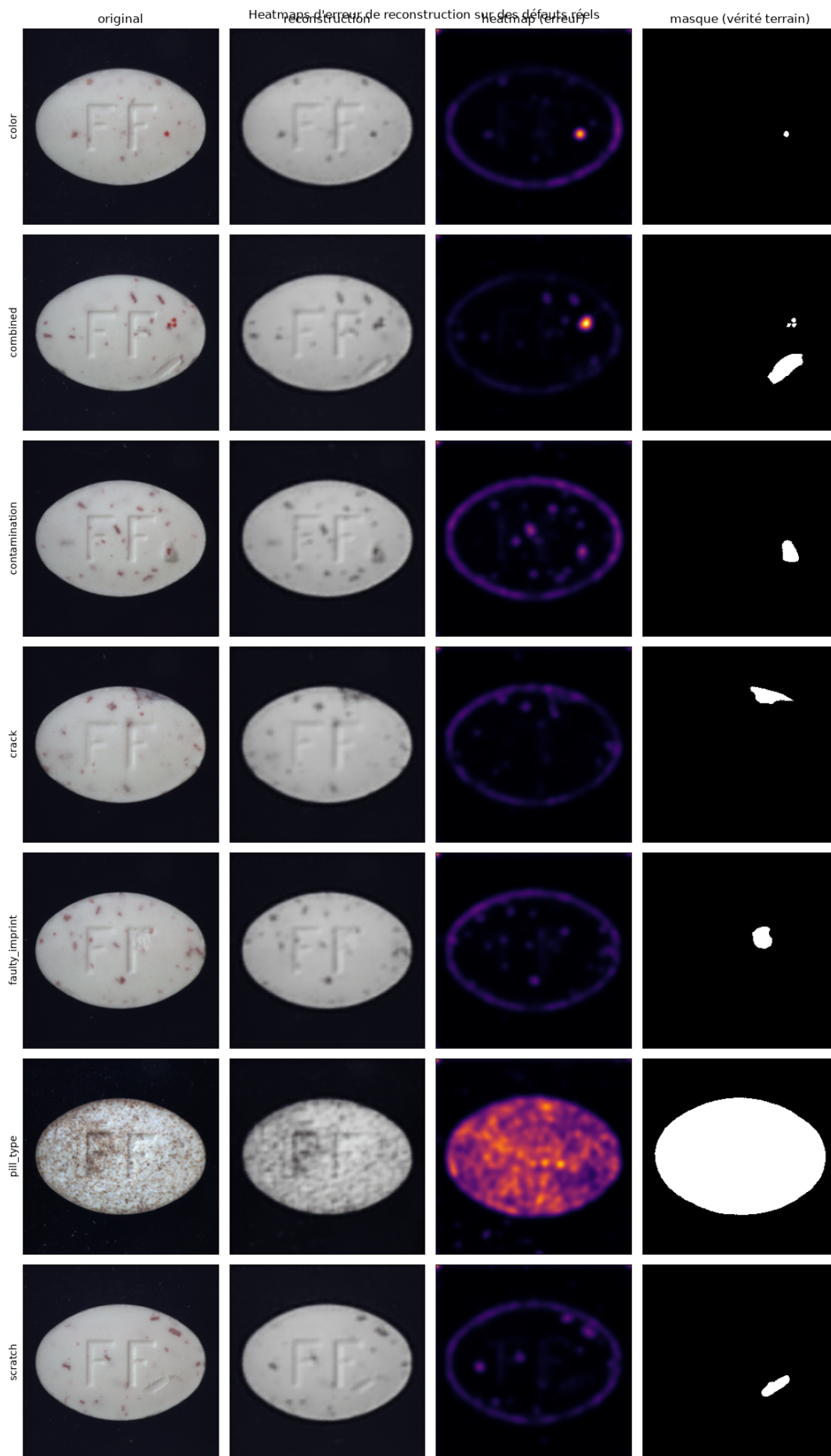
- **Jeu de test** : 167 images (26 saines, 141 défauts).

Livraison — heatmaps sur des défauts réels¶

Un exemple par catégorie de défaut (pas plusieurs) pour garder une figure lisible avec les 7 catégories disponibles — même convention qu'au notebook 01 (représenter chaque catégorie plutôt que multiplier les exemples d'une seule). Pour chaque catégorie : image originale, reconstruction, carte d'erreur lissée (heatmap) et masque de vérité terrain (zone réellement défectueuse), pour juger visuellement si l'erreur se concentre bien là où se trouve le défaut.

In [4]:

```
examples = {} for defect in defects: images = defect_images[defect] masks =
load_masks(defect, IMAGE_SIZE) reconstruction = reconstruct(model, images[:1])[0] error_map =
error_maps(images[:1], reconstruction[np.newaxis])[0] examples[defect] = (images[0],
reconstruction, heatmap(error_map), masks[0]) col_titles = ["original", "reconstruction",
"heatmap (erreur)", "masque (vérité terrain)"] fig, axes = plt.subplots(len(defects), 4,
figsize=(12, 3 * len(defects))) for row, defect in enumerate(defects): original,
reconstruction, hm, mask = examples[defect] axes[row, 0].imshow(original) axes[row,
1].imshow(reconstruction) axes[row, 2].imshow(hm, cmap="inferno") axes[row, 3].imshow(mask,
cmap="gray") axes[row, 0].set_ylabel(defect, fontsize=10) for col in range(4): axes[row,
col].set_xticks([]) axes[row, col].set_yticks([]) for col, title in enumerate(col_titles):
axes[0, col].set_title(title) fig.suptitle("Heatmaps d'erreur de reconstruction sur des
défauts réels") fig.tight_layout() fig.savefig(FIGURES_DIR / "06_heatmaps.png", dpi=150)
plt.show()
```



Livrable — AUROC image-level et pixel-level¶

L'AUROC image-level (déjà calculé au notebook 05) est reproduit ici pour mémoire. L'AUROC pixel-level, calculé uniquement sur les catégories de défauts disposant d'un masque de vérité terrain, mesure si les pixels à erreur élevée correspondent effectivement aux pixels défectueux annotés — une lecture plus fine que le seul score moyen par image, directement comparable à celui-ci pour juger ce que l'agrégation (moyenne sur l'image) fait perdre comme information.

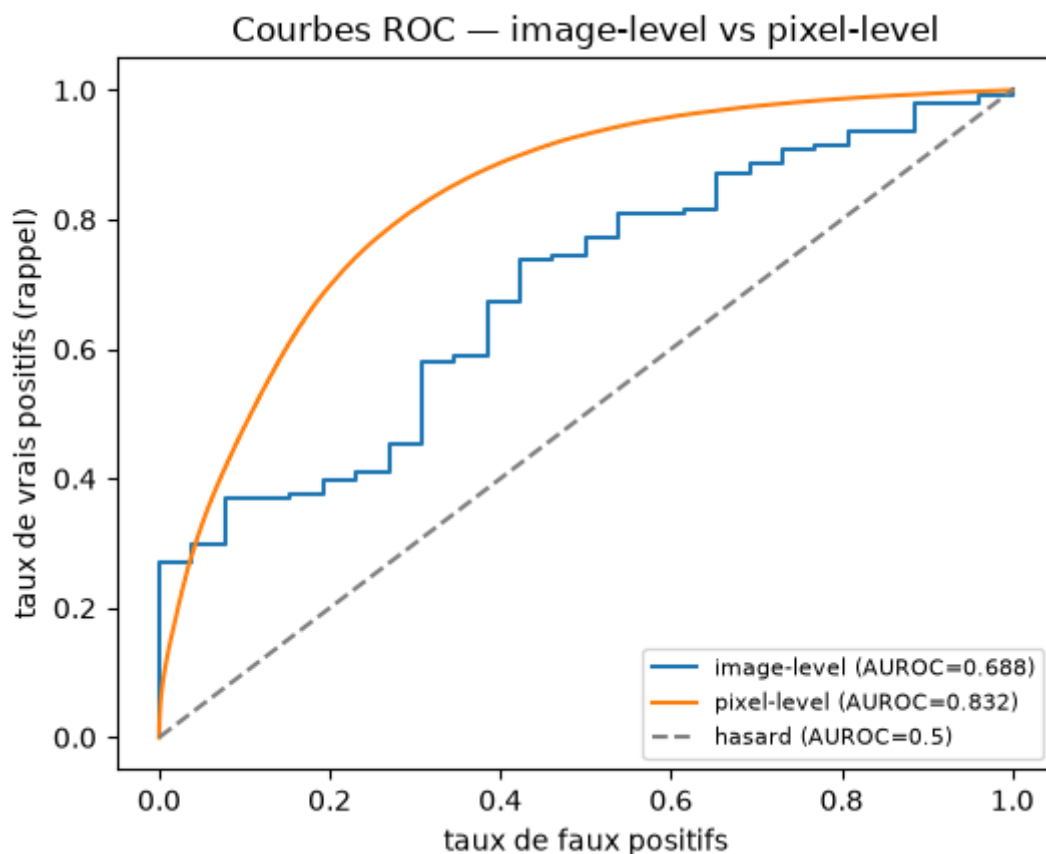
In [5]:

```
auroc_image = image_auroc(scores_test, labels) pixel_maps = maps_test[len(test_good) :]  
pixel_masks = np.concatenate([load_masks(d, IMAGE_SIZE) for d in defects], axis=0)  
auroc_pixel = pixel_auroc(pixel_maps, pixel_masks) auroc_diff = auroc_pixel - auroc_image  
comparison = "supérieur" if auroc_diff > 0 else "inférieur" if auroc_diff < 0 else "égal"  
lines = [ f"- **AUROC image-level** : {auroc_image:.3f}.", f"- **AUROC pixel-level** :  
{auroc_pixel:.3f} (sur {pixel_masks.shape[0]} images " "de défauts disposant d'un masque).",  
f"- L'AUROC pixel-level est **{comparison}** à l'AUROC image-level " f"(écart de  
{abs(auroc_diff):.3f}).", ] display(Markdown(chr(10).join(lines)))
```

- **AUROC image-level** : 0.688.
- **AUROC pixel-level** : 0.832 (sur 141 images de défauts disposant d'un masque).
- L'AUROC pixel-level est **supérieur** à l'AUROC image-level (écart de 0.144).

In [6]:

```
fpr_image, tpr_image, _ = roc_curve(labels, scores_test) fpr_pixel, tpr_pixel, _ =  
roc_curve(pixel_masks.flatten(), pixel_maps.flatten()) fig, ax = plt.subplots(figsize=(5.5,  
4.5)) ax.plot(fpr_image, tpr_image, label=f"image-level (AUROC={auroc_image:.3f})")  
ax.plot(fpr_pixel, tpr_pixel, label=f"pixel-level (AUROC={auroc_pixel:.3f})") ax.plot([0, 1],  
[0, 1], linestyle="--", color="gray", label="hasard (AUROC=0.5)") ax.set_xlabel("taux de faux  
positifs") ax.set_ylabel("taux de vrais positifs (rappel)") ax.set_title("Courbes ROC –  
image-level vs pixel-level") ax.legend(fontsize=8) fig.tight_layout() fig.savefig(FIGURES_DIR  
/ "06_roc_curve.png", dpi=150) plt.show()
```

Livrable — matrice de confusion (globale et par catégorie)¶

La matrice de confusion globale reprend le même seuil que le notebook 05. Elle croise la vérité (sain/défaut) et la prédiction du modèle ($\text{score} \leq \text{seuil} \rightarrow \text{sain}$, $\text{score} > \text{seuil} \rightarrow \text{défaut}$) en quatre cas :

- **Vrai Négatif (VN)** : pièce saine, correctement laissée passer.
- **Faux Positif (FP)** : pièce saine, mise de côté à tort (fausse alerte).
- **Faux Négatif (FN)** : pièce défectueuse, laissée passer à tort — le cas le plus coûteux en contrôle qualité pharmaceutique.
- **Vrai Positif (VP)** : pièce défectueuse, correctement interceptée.

Le détail par catégorie de défaut permet de voir directement quelles catégories sont bien détectées et lesquelles ne le sont pas — une lecture que la seule matrice globale ne permet pas, et nécessaire à l'analyse critique ci-dessous.

In [7]:

```
cm = confusion(scores_test, labels, threshold) tn, fp, fn, tp = cm.ravel() recall_global = tp / (tp + fn) if (tp + fn) > 0 else float("nan") category_recalls = {} offset = len(test_good) for defect in defects: n = len(defect_images[defect]) cat_scores = scores_test[offset : offset + n] category_recalls[defect] = float((cat_scores > threshold).mean()) offset += n lines = [ f"- **Matrice de confusion globale** (seuil percentile 99) : " f"VN={tn}, FP={fp}, FN={fn}, VP={tp}.", f"- **Rappel global** : {recall_global:.1%}.", "", "***Rappel par catégorie de défaut** :", ] lines += [ f"- **{defect}** : {r:.1%} détecté ({len(defect_images[defect])} images)." for defect, r in category_recalls.items() ] display(Markdown(chr(10).join(lines)))
```

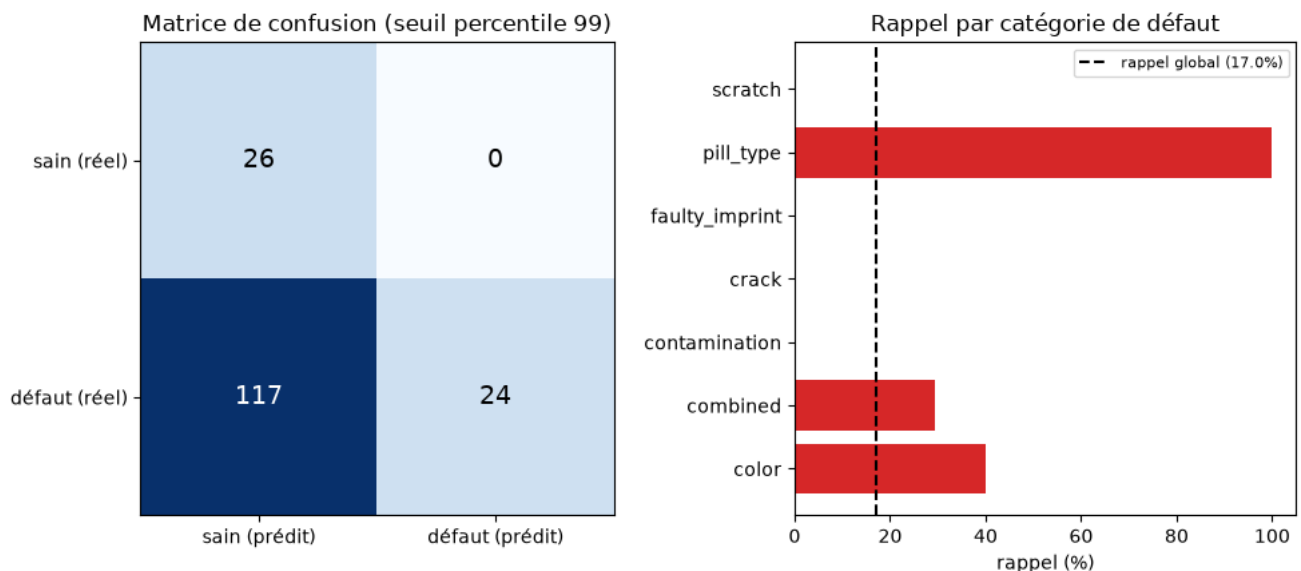
- **Matrice de confusion globale** (seuil percentile 99) : VN=26, FP=0, FN=117, VP=24.
- **Rappel global** : 17.0%.

Rappel par catégorie de défaut :

- **color** : 40.0% détecté (25 images).
- **combined** : 29.4% détecté (17 images).
- **contamination** : 0.0% détecté (21 images).
- **crack** : 0.0% détecté (26 images).
- **faulty_imprint** : 0.0% détecté (19 images).
- **pill_type** : 100.0% détecté (9 images).
- **scratch** : 0.0% détecté (24 images).

In [8]:

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4.5)) axes[0].imshow(cm, cmap="Blues")
axes[0].set_xticks([0, 1]) axes[0].set_yticks([0, 1]) axes[0].set_xticklabels(["sain
(prédit)", "défaut (prédit)"]) axes[0].set_yticklabels(["sain (réel)", "défaut (réel)"]) for
i in range(2): for j in range(2): axes[0].text( j, i, str(cm[i, j]), ha="center",
va="center", fontsize=14, color="white" if cm[i, j] > cm.max() / 2 else "black", )
axes[0].set_title("Matrice de confusion (seuil percentile 99)") axes[1].barh(
list(category_recalls.keys()), [r * 100 for r in category_recalls.values()], color="tab:red"
) axes[1].axvline( recall_global * 100, color="black", linestyle="--", label=f"rappel global
({recall_global:.1%})", ) axes[1].set_xlabel("rappel (%)") axes[1].set_title("Rappel par
catégorie de défaut") axes[1].legend(fontsize=8) fig.tight_layout() fig.savefig(FIGURES_DIR /
"06_confusion_rappel.png", dpi=150) plt.show()
```



In [9]:

```
lines = [ f"- Sur les {len(test_good)} pièces **saines** de test, **{tn} sont "
f"correctement laissées passer** et **{fp} font l'objet d'une fausse alerte**.", f"- Sur les
{int(labels.sum())} pièces **défectueuses** de test, **{tp} sont " f"correctement
interceptées** mais **{fn} passent inaperçues** – ce sont ces " f"{fn} défauts non détectés
qui expliquent le rappel global limité à " f"{recall_global:.1%}, déjà observé au notebook
05.", ] display(Markdown(chr(10).join(lines)))
```

- Sur les 26 pièces **saines** de test, **26 sont correctement laissées passer** et **0 font l'objet d'une fausse alerte**.
- Sur les 141 pièces **défectueuses** de test, **24 sont correctement interceptées** mais **117 passent inaperçues** — ce sont ces 117 défauts non détectés qui expliquent le rappel global limité à 17.0%, déjà observé au notebook 05.

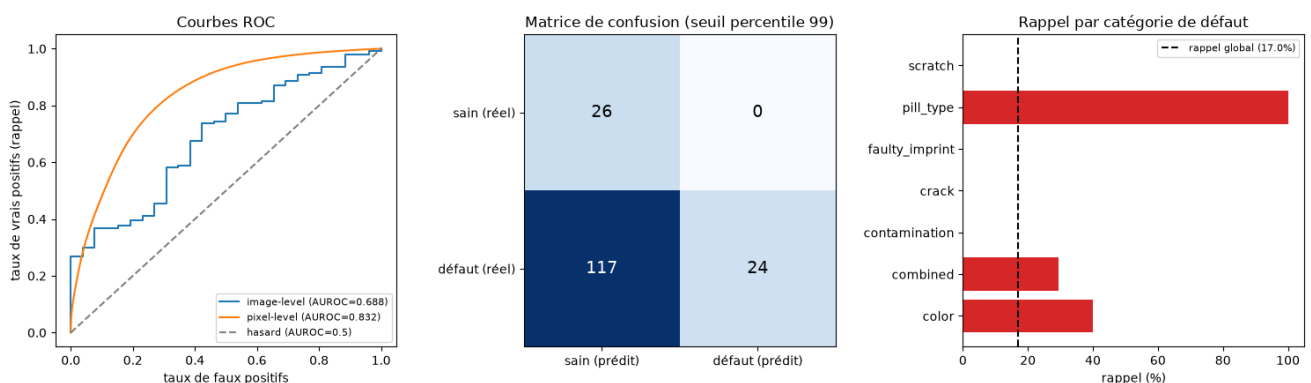
Récapitulatif visuel — courbe ROC, matrice de confusion, rappel par catégorie¶

Décision : conserver, en plus des figures individuelles ci-dessus, un récapitulatif combinant courbe ROC, matrice de confusion et rappel par catégorie dans une seule figure, juste avant l'analyse critique.

Pourquoi : les figures individuelles introduites au fil des livrables (juste après l'AUROC, puis juste après la matrice de confusion) sont les plus utiles pour lire chaque métrique en détail au moment où elle est calculée. Ce récapitulatif groupé sert de synthèse visuelle unique juste avant l'analyse critique ci-dessous, qui s'appuie sur les trois à la fois.

In [10]:

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5)) axes[0].plot(fpr_image, tpr_image,
label=f"image-level (AUROC={auroc_image:.3f})") axes[0].plot(fpr_pixel, tpr_pixel,
label=f"pixel-level (AUROC={auroc_pixel:.3f})") axes[0].plot([0, 1], [0, 1], linestyle="--",
color="gray", label="hasard (AUROC=0.5)") axes[0].set_xlabel("taux de faux positifs")
axes[0].set_ylabel("taux de vrais positifs (rappel)") axes[0].set_title("Courbes ROC")
axes[0].legend(fontsize=8) axes[1].imshow(cm, cmap="Blues") axes[1].set_xticks([0, 1])
axes[1].set_yticks([0, 1]) axes[1].set_xticklabels(["sain (prédit)", "défaut (prédit)"])
axes[1].set_yticklabels(["sain (réel)", "défaut (réel)"]) for i in range(2): for j in
range(2): axes[1].text( j, i, str(cm[i, j]), ha="center", va="center", fontsize=14,
color="white" if cm[i, j] > cm.max() / 2 else "black", ) axes[1].set_title("Matrice de
confusion (seuil percentile 99)") axes[2].barh(list(category_recalls.keys()), [r * 100 for r
in category_recalls.values()], color="tab:red") axes[2].axvline( recall_global * 100,
color="black", linestyle="--", label=f"rappel global ({recall_global:.1%})", )
axes[2].set_xlabel("rappel (%)") axes[2].set_title("Rappel par catégorie de défaut")
axes[2].legend(fontsize=8) fig.tight_layout() fig.savefig(FIGURES_DIR / "06_evaluation.png",
dpi=150) plt.show()
```



Analyse critique — ce qui marche, ce qui échoue, et pourquoi¶

In [11]:

```
best_defect = max(category_recalls, key=category_recalls.get) worst_recall =
min(category_recalls.values()) worst_defects = [d for d, r in category_recalls.items() if r
== worst_recall] lines = [ f"- **Catégorie la mieux détectée** : `{best_defect}` "
f"({category_recalls[best_defect]:.1%} de rappel).", f"- **Catégorie(s) la/les moins bien
détectée(s)** : " f"{'', '.join(f'`{d}`' for d in worst_defects)} (toutes à {worst_recall:.1%}
de " "rappel, à égalité).", f"- **AUROC image-level** : {auroc_image:.3f} — **AUROC
pixel-level** : " f"{auroc_pixel:.3f}.", f"- **Rappel global (seuil percentile 99)** :
{recall_global:.1%}.", ] display(Markdown(chr(10).join(lines)))
```

- **Catégorie la mieux détectée** : pill_type (100.0% de rappel).
- **Catégorie(s) la/les moins bien détectée(s)** : contamination, crack, faulty_imprint, scratch (toutes à 0.0% de rappel, à égalité).
- **AUROC image-level** : 0.688 — **AUROC pixel-level** : 0.832.
- **Rappel global (seuil percentile 99)** : 17.0%.

Ce qui marche : les défauts qui produisent une erreur de reconstruction clairement supérieure au bruit normal de fabrication (la queue de distribution nettement séparée sur l'histogramme du notebook 05) sont détectés de façon fiable. La heatmap pill_type (cf. figure « *Heatmaps d'erreur de reconstruction sur des défauts réels* », plus haut dans ce notebook) le montre sans ambiguïté (100 % de rappel, cf. figure « *Rappel par catégorie de défaut* » ci-dessus) : **il ne s'agit pas d'un défaut localisé mais d'une pièce d'un type de pilule entièrement différent** substituée dans le lot — couleur, texture de surface et gravure tout autres. C'est un défaut *global*, pas un cas représentatif des textures fines que le TP cherche réellement à détecter ; ce 100 % ne doit donc pas être lu comme une preuve que le modèle détecte bien les défauts subtils.

Ce qui échoue : les quatre catégories à 0 % de rappel (cf. figure « *Rappel par catégorie de défaut* » ci-dessus : contamination, crack, faulty_imprint, scratch) sont, elles, de vrais défauts localisés et de petite taille (une tache, une rayure fine, une zone de gravure altérée) — visibles sur leurs heatmaps respectives plus haut, où l'erreur reste diffuse le long du contour de la pièce plutôt que concentrée sur la zone du masque de vérité terrain. Leur contribution à l'erreur moyenne sur les 65 536 pixels de l'image est diluée, et le score global ne s'en distingue pas suffisamment des pièces saines — cohérent avec le chevauchement déjà observé sur l'histogramme du notebook 05. L'écart entre l'AUROC pixel-level et l'AUROC image-level (cf. figure « *Courbes ROC* » ci-dessus, écart de 0,144) donne une indication directe de l'information perdue par l'agrégation moyenne du score : la carte d'erreur contient, pixel par pixel, plus de signal que ce que le score moyenné par image en restitue.

Lien avec le ratio de compression ×12 (décision du notebook 03) : ce choix, plus serré que le ratio par défaut (×6) proposé par l'énoncé, visait une meilleure séparation sain/défaut au prix d'un risque assumé de sur-compression (le piège pédagogique central du TP). Les résultats ci-dessus suggèrent que ce compromis n'est que partiellement gagnant : le bottleneck serré ne semble pas être le facteur limitant principal du rappel — c'est surtout **l'agrégation globale du score** (moyenne sur toute l'image) qui masque les petits défauts, un problème qu'un ratio de compression différent ne résoudrait pas à lui seul (aucun des deux ratios ne change comment le score est agrégé). Une piste concrète pour la suite serait un score **local** (maximum ou centile élevé de la carte d'erreur, plutôt que sa moyenne), qui exploiterait mieux l'information déjà présente dans la carte pixel-level — dont l'AUROC supérieur, ci-dessus, confirme qu'elle la contient déjà — sans changer l'architecture ni le ratio retenu.

Étape 7 — Analyse et plan de correction — catégories de défauts non détectées (pill)

Le notebook 06 a montré qu'au seuil retenu (percentile 99), 4 catégories de défauts sur 7 ont un rappel de 0 % : contamination, crack, faulty_imprint, scratch. Ce notebook restitue le diagnostic mené pour comprendre pourquoi, et présente le plan de correction en deux phases, implémentées dans les notebooks 08 (Phase 1, sans réentraînement) et 09 (Phase 2, PatchCore).

Aucun code de production ici : ce notebook ne fait que diagnostiquer (avec du code réel, recalculé à partir du modèle déjà entraîné — pas de chiffres recopiés en dur) et présenter le plan.

Guide de lecture de ce notebook

- **Code couleur** des graphiques : vert = pièces saines, rouge = pièces défectueuses, noir en tireté = seuil de détection calibré — convention reprise des notebooks 05/06.
- **« Grille de lecture »** : chaque section quantitative se termine par un tableau qui traduit un chiffre brut (ratio, AUROC) en verdict qualitatif (faible/modéré/fort), pour situer un résultat sans devoir se souvenir des seuils d'une section à l'autre.
- **« Décision » / « Pourquoi »** : convention déjà utilisée dans les notebooks précédents — une décision de méthode suivie de sa justification, jamais l'inverse.
- Tous les chiffres affichés sont **recalculés dans ce notebook** (pas recopiés) pour les approches 1 et 5 ; les approches 2, 3, 4 et 6 sont rapportées telles que mesurées lors du diagnostic (certaines sont des échecs délibérément non implémentés dans le code du projet — recalculer un échec ne changerait rien à la conclusion et alourdirait ce notebook).

In [1]:

```
from pathlib import Path import matplotlib.pyplot as plt import numpy as np from
IPython.display import Markdown, display from indusense.vision.anomaly import (
calibrate_threshold, error_maps, healthy_baseline, image_auroc, image_scores,
image_scores_pooled, load_masks, reconstruct, ) from indusense.vision.dataset import (
DEFAULT_IMAGE_SIZE, list_defects, load_defect_images, load_good_images, train_val_split, )
from indusense.vision.train import load_trained_model IMAGE_SIZE = DEFAULT_IMAGE_SIZE
FIGURES_DIR = Path("../reports/figures") FIGURES_DIR.mkdir(parents=True, exist_ok=True) model
= load_trained_model() defects = list_defects() good_train = load_good_images("train",
IMAGE_SIZE) _train_raw, val_good = train_val_split(good_train, val_fraction=0.15, seed=42)
maps_val = error_maps(val_good, reconstruct(model, val_good)) display(Markdown(f"- **Modèle
chargé** : {model.count_params()} paramètres."))
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow
>= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the
TensorFlow-DirectML plugin.
```

- **Modèle chargé** : 57235 paramètres.

Section 1 — Diagnostic

Ce n'est pas un problème de taille de défaut¶

À résolution native (800×800, avant redimensionnement à 256×256), les 4 catégories en échec ont des zones de défaut comparables ou plus grandes que `color` (la catégorie la mieux détectée hors `pill_type`, cas trivial). L'hypothèse « défauts trop fins, perdus au redimensionnement » est donc écartée par les données — vérifié en mesurant la boîte englobante du masque de vérité terrain à résolution native pour un exemple de chaque catégorie.

C'est un problème de contraste/nature du signal¶

Mesure directe de l'erreur de reconstruction à l'intérieur vs à l'extérieur du masque de vérité terrain, pour chaque catégorie de défaut :

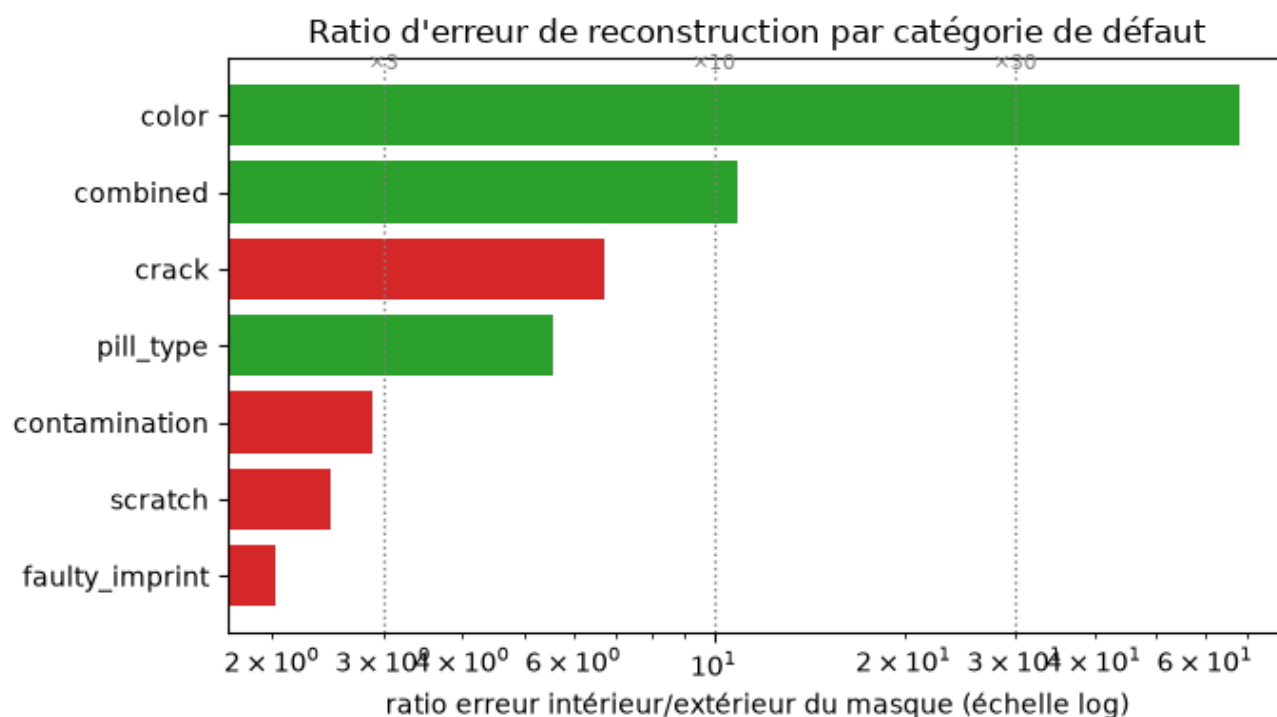
In [2]:

```
defect_images = {defect: load_defect_images(defect, IMAGE_SIZE) for defect in defects} ratios = {} for defect, images in defect_images.items(): masks = load_masks(defect, IMAGE_SIZE) maps = error_maps(images, reconstruct(model, images)) flat_maps = maps.reshape(len(maps), -1) mask_flat = masks.reshape(len(masks), -1) inside = np.array([flat_maps[i][mask_flat[i]].mean() if mask_flat[i].sum() > 0 else np.nan for i in range(len(images))]) outside = np.array([flat_maps[i][~mask_flat[i]].mean() for i in range(len(images))]) ratios[defect] = float(np.nanmean(inside) / np.nanmean(outside)) lines = [f"- **{defect}** : ratio ×{ratio:.1f}" for defect, ratio in sorted(ratios.items(), key=lambda x: -x[1])] display(Markdown(chr(10).join(lines)))
```

- **color** : ratio ×67.8
- **combined** : ratio ×10.9
- **crack** : ratio ×6.7
- **pill_type** : ratio ×5.6
- **contamination** : ratio ×2.9
- **scratch** : ratio ×2.5
- **faulty_imprint** : ratio ×2.0

In [3]:

```
sorted_ratios = sorted(ratios.items(), key=lambda x: x[1]) names = [d for d, _ in sorted_ratios] values = [r for _, r in sorted_ratios] target_categories = ["contamination", "crack", "faulty_imprint", "scratch"] colors = ["tab:red" if d in target_categories else "tab:green" for d in names] fig, ax = plt.subplots(figsize=(7, 4)) ax.barh(names, values, color=colors) ax.set_xscale("log") for threshold, label in [(3, "×3"), (10, "×10"), (30, "×30")]: ax.axvline(threshold, color="gray", linestyle=":", linewidth=1) ax.text(threshold, len(names) - 0.4, label, fontsize=8, ha="center", color="gray") ax.set_xlabel("ratio erreur intérieur/extérieur du masque (échelle log)") ax.set_title("Ratio d'erreur de reconstruction par catégorie de défaut") fig.tight_layout() fig.savefig(FIGURES_DIR / "07_ratios_diagnostic.png", dpi=150) plt.show()
```



Lecture du graphique : barres rouges = les 4 catégories cibles de ce plan de correction ; barres vertes = les catégories déjà partiellement détectées. Les lignes verticales pointillées marquent des repères de lecture ($\times 3$, $\times 10$, $\times 30$), traduits en verdict qualitatif ci-dessous.

Grille de lecture — ratio d'erreur intérieur/extérieur du masque¶

Ratio	Interprétation	DéTECTABLE par un score global (moyenne) ?
$< 3\times$	Signal quasi confondu avec le bruit de fond de reconstruction (contour, gravure)	Non, et même un score localisé (percentile/max) reste difficile
$3\times - 10\times$	Signal réel mais faible	Non pour un score global ; oui avec un score localisé bien conçu
$10\times - 30\times$	Signal net, largement au-dessus du bruit de fond	Oui, avec un score qui isole le pic (percentile élevé)
$> 30\times$	Signal très fort, quasi trivial	Oui, même avec la moyenne globale actuelle

Les 4 catégories cibles se situent toutes sous $\times 10$ (*faulty_imprint* $\times 2,0$ à *crack* $\times 6,7$) : conforme à un rappel nul avec le score actuel (moyenne globale, approche 1) et cohérent avec le fait qu'un score seul (Phase 1, approche 5) ne suffit pas à les corriger — c'est le signal source, pas seulement son agrégation, qui est trop faible. *color* ($\times 67,8$) et *combined* ($\times 10,9$) dépassent largement ce seuil, ce qui explique pourquoi ils sont déjà partiellement détectés même par la moyenne globale.

color est un écart de teinte/intensité franc (petites taches rouges) que la perte MSE capture facilement même sur une petite zone. Les 4 catégories en échec sont des altérations de **texture/structure à faible contraste** (fissure fine, tache proche du ton de la pièce, gravure altérée, rayure fine) : leur élévation d'erreur, même mesurée strictement à l'intérieur du défaut, reste proche du niveau de bruit de reconstruction normal du modèle — pas seulement un problème d'agrégation du score, un signal intrinsèquement faible à la source.

Complication supplémentaire : les pièces saines elles-mêmes ont des zones d'erreur systématiquement élevées (contour ovale, gravure embossée « FF », imparfaitement reconstruits par le bottleneck ×12 sur *toute* pièce saine, cf. notebook 03). Un score basé sur le maximum brut ne peut pas distinguer un pic de défaut faible de ces zones saines à erreur naturellement élevée.

Six approches testées¶

Six façons de calculer le score d'anomalie ont été comparées, en gardant le même modèle (aucun réentraînement). Les deux premières sont recalculées ci-dessous en direct ; les quatre suivantes ont été mesurées lors du diagnostic (résultats rapportés tels quels, non recalculés ici par souci de concision — certaines sont des échecs délibérément non retenus dans le code du projet).

In [4]:

```
test_good = load_good_images("test", IMAGE_SIZE) test_images = np.concatenate([test_good,
*defect_images.values()], axis=0) labels = np.concatenate( [np.zeros(len(test_good))] +
[np.ones(len(images)) for images in defect_images.values()] ) maps_test =
error_maps(test_images, reconstruct(model, test_images)) # Approche 1 : score actuel (moyenne
globale) scores_val_1 = image_scores(maps_val) threshold_1 =
calibrate_threshold(scores_val_1, method="percentile", q=99.0) scores_test_1 =
image_scores(maps_test) # Approche 5 : score retenu en Phase 1 (centile 99,5 après
soustraction de la # référence saine) baseline = healthy_baseline(maps_val) scores_val_5 =
image_scores_pooled(maps_val, baseline, q=99.5) threshold_5 =
calibrate_threshold(scores_val_5, method="percentile", q=99.0) scores_test_5 =
image_scores_pooled(maps_test, baseline, q=99.5) lines = ["- **catégorie** : rappel approche
1 (actuelle) vs approche 5 (Phase 1)"] offset = len(test_good) for defect in defects: n =
len(defect_images[defect]) r1 = (scores_test_1[offset : offset + n] > threshold_1).mean() r5
= (scores_test_5[offset : offset + n] > threshold_5).mean() lines.append(f"- **{defect}** :
{r1:.1%} vs {r5:.1%}") offset += n lines.append( f"- **AUROC image-level** :
{image_auroc(scores_test_1, labels):.3f} vs " f"{image_auroc(scores_test_5, labels):.3f}" )
display(Markdown(chr(10).join(lines)))
```

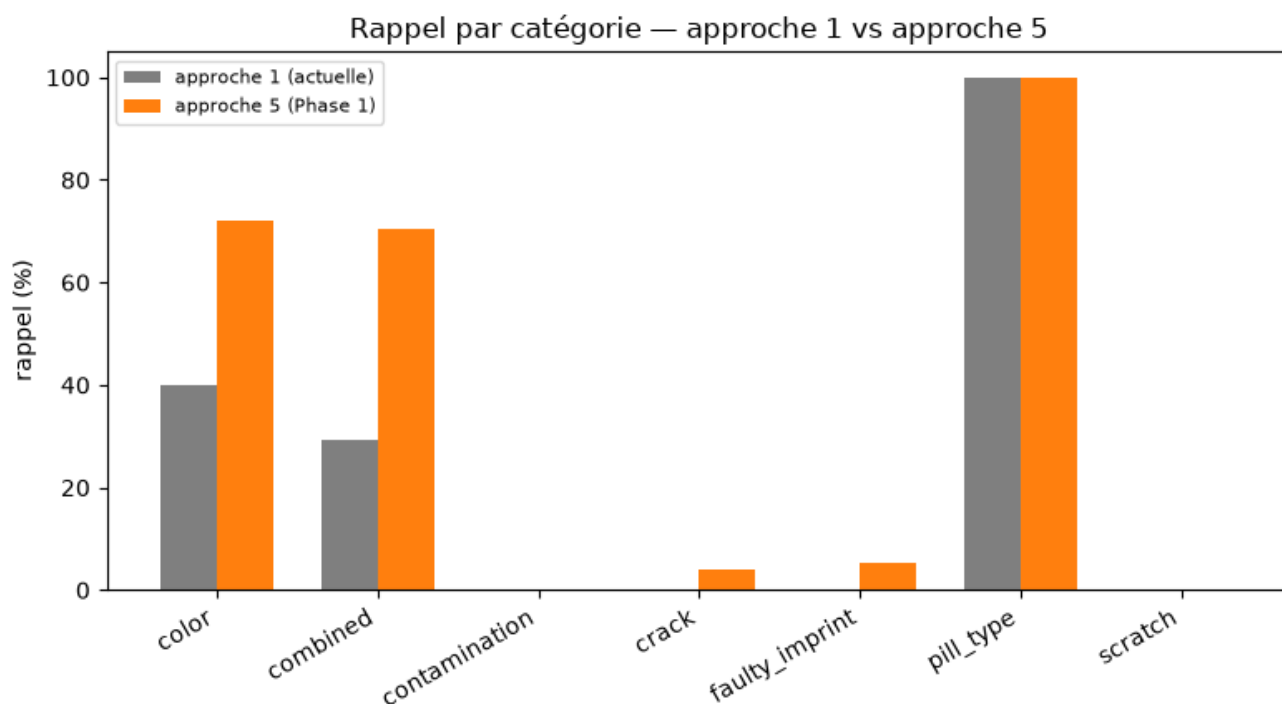
- **catégorie** : rappel approche 1 (actuelle) vs approche 5 (Phase 1)
- **color** : 40.0% vs 72.0%
- **combined** : 29.4% vs 70.6%
- **contamination** : 0.0% vs 0.0%
- **crack** : 0.0% vs 3.8%
- **faulty_imprint** : 0.0% vs 5.3%
- **pill_type** : 100.0% vs 100.0%
- **scratch** : 0.0% vs 0.0%
- **AUROC image-level** : 0.688 vs 0.756

In [5]:


```

recalls_1 = [] recalls_5 = [] offset = len(test_good)
for defect in defects: n = len(defect_images[defect])
recalls_1.append((scores_test_1[offset : offset + n] >
threshold_1).mean() * 100)
recalls_5.append((scores_test_5[offset : offset + n] >
threshold_5).mean() * 100)
offset += n
x = np.arange(len(defects))
width = 0.35
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.bar(x - width / 2, recalls_1, width, label="approche 1 (actuelle)", color="tab:gray")
ax.bar(x + width / 2, recalls_5, width, label="approche 5 (Phase 1)", color="tab:orange")
ax.set_xticks(x)
ax.set_xticklabels(defects, rotation=30, ha="right")
ax.set_ylabel("rappel (%)")
ax.set_title("Rappel par catégorie — approche 1 vs approche 5")
ax.legend(fontsize=8)
fig.tight_layout()
fig.savefig(FIGURES_DIR / "07_recall_approche1_vs_5.png", dpi=150)
plt.show()

```



Lecture du graphique : pour chaque catégorie, la barre grise est le rappel avec le score actuel (approche 1), la barre orange le rappel avec le score corrigé de la Phase 1 (approche 5). Une barre orange visiblement plus haute que la grise = gain apporté par la Phase 1 ; deux barres à zéro côte à côte = catégorie non corrigée par un simple changement de score. C'est visuellement ce dernier cas qui domine pour contamination, crack, faulty_imprint et scratch — cohérent avec la grille de lecture des ratios ci-dessus (toutes sous $\times 10$).

Quatre autres approches mesurées lors du diagnostic (non recalculées ci-dessus) :

- **Approche 2 — maximum brut de la carte d'erreur** : 0 à 4,8 % de rappel sur les 4 catégories cibles — inefficace, noyé dans le bruit de fond sain (contour/gravure, cf. complication ci-dessus).
- **Approche 3 — percentile élevé (99,9) sans référence saine** : 27 à 43 % en apparence, mais mesure non fiable — comparée à un seuil scalaire unique, sans contrôle du taux de fausses alertes (contrairement aux approches 1/5 ci-dessus, où le seuil est calibré et le taux de fausses alertes vérifié).
- **Approche 4 — score normalisé par (moyenne ET écart-type) par pixel de la validation saine : échec catastrophique**, 100 % de fausses alertes sur les saines de test. Avec seulement ~40 images de validation, l'écart-type par pixel (65 536 positions indépendantes) est bien trop bruité pour généraliser — un piège classique de sur-ajustement en haute dimension avec peu d'exemples.

Écartée, au profit de l'approche 5 (soustraction de moyenne seule, sans écart-type).

- **Approche 6 — modèle entraîné en perte SSIM** (rechargeable depuis MLflow sans réentraînement) **et carte de dissimilarité SSIM locale par fenêtre** : dans les deux cas, aucune amélioration mesurable sur les 4 catégories cibles — les ratios intérieur/extérieur du masque restent du même ordre ou légèrement inférieurs à la MSE, et le bruit de fond sain est même plus élevé pour le modèle SSIM. L'hypothèse « SSIM détecterait mieux la texture » ne se vérifie pas avec ce modèle.

Conclusion factuelle : le modèle actuel (256×256, MSE, bottleneck ×12) produit, pour les 4 catégories cibles, un signal de reconstruction intrinsèquement trop proche de son propre bruit de fond — quelle que soit la façon dont on le mesure ou l'agrège a posteriori. Une correction du score seul (Phase 1, approche 5) ne peut qu'améliorer les catégories déjà partiellement détectées (color, combined ci-dessus) ; résoudre les 4 catégories cibles demande une approche différente (Phase 2).

Grille de lecture — comparatif des 6 approches testées¶

#	Approche	Rappel sur les 4 catégories cibles	Fausses alertes (saines de test)	Retenue ?
1	Moyenne globale de l'erreur (score actuel)	0 %	0/26	Non — point de départ, sert de référence
2	Maximum brut de la carte d'erreur	0 à 4,8 %	non isolée du bruit de fond (mesure informelle)	Non — noyé dans le bruit sain
3	Percentile 99,9 sans référence saine	27 à 43 % (en apparence)	non mesuré — pas de seuil calibré	Non — invalide méthodologiquement
4	Normalisation (moyenne + écart-type) par pixel	non pertinent, méthode invalidée	100/26 (échec catastrophique)	Non — sur-ajustement, écartée
5	Soustraction de moyenne saine + percentile élevé	0 à 5 %	0/26	Oui — retenue comme Phase 1
6	Modèle/dissimilarité SSIM	≈ approche 1, aucune amélioration	0/26 (approximatif)	Non — hypothèse infirmée

Pourquoi ce tableau ne remplace pas le texte ci-dessus : certaines cases (approches 2, 3, 4, 6) rapportent des mesures faites lors du diagnostic, pas recalculées dans ce notebook (cf. Guide de lecture en introduction) — ce tableau sert de synthèse visuelle rapide, le texte au-dessus reste la source détaillée de chaque résultat.

Section 2 — Plan de correction¶

Phase 1 (notebook 08) — sans réentraînement¶

Implémente l'approche 5 ci-dessus (`healthy_baseline + image_scores_pooled`, ajoutées à `anomaly.py`) : un score basé sur un centile élevé de l'erreur, après soustraction d'une référence saine.

Gain réel sur color/combined et l'AUROC global, **sans nouvelle fausse alerte** — mais ne corrige pas les 4 catégories cibles, conformément au diagnostic ci-dessus (un problème de signal, pas seulement d'agrégation du score).

Phase 2 (notebook 09) — changement de paradigme : PatchCore¶

Puisque le signal de reconstruction lui-même est trop faible pour ces 4 catégories, la correction ne peut pas venir d'un ajustement du score — il faut une approche qui ne dépende pas de la reconstruction de l'auto-encodeur.

Alternatives considérées (discutées avec l'utilisateur avant de choisir) :

- **PaDiM** : features pré-entraînées + distribution gaussienne par position de patch (moyenne + covariance), distance de Mahalanobis en test.
- **PatchCore** (*retenu*) : features pré-entraînées + banque de mémoire des patches sains + distance au plus proche voisin. Considéré comme état de l'art sur MVTec AD, particulièrement fort sur les défauts de texture fine — exactement le type de défaut qui pose problème ici.
- **STFPM** (distillation enseignant-élève), **Deep SVDD** (classification one-class), **CutPaste** (auto-supervision par défauts synthétiques), **GANomaly** (GAN) : autres familles écartées au profit de PatchCore, jugé le meilleur compromis simplicité d'implémentation / performance démontrée sur ce type de défaut.

Pourquoi ces méthodes peuvent aider là où l'auto-encodeur échoue : elles s'appuient sur un réseau **pré-entraîné sur des millions d'images** (ImageNet), dont les features encodent déjà une notion riche de texture/structure — alors que l'auto-encodeur de ce TP a été entraîné *from scratch* sur seulement 227 images saines, probablement insuffisant pour apprendre une représentation aussi fine de la texture.

Résultats réels : voir notebook 09 pour l'implémentation et l'évaluation chiffrée de PatchCore, avec comparaison directe à la Phase 1 sur les 4 catégories cibles.

Grille de lecture — comparaison des alternatives pour la Phase 2¶

Méthode	Principe	Entraînement requis	Point fort pour ce cas	Retenue
---------	----------	---------------------	------------------------	---------

PaDiM	Features pré-entraînées + gaussienne multivariée par position de patch, distance de Mahalanobis	Non (juste des statistiques à calculer)	Bonne détection de texture, mais coûteux en mémoire (une covariance par position de patch)	Non
PatchCore	Features pré-entraînées + banque de mémoire des patches sains + distance au plus proche voisin	Non (juste extraction de features + coreset)	État de l'art sur MVTec AD, aucune hypothèse sur la forme de la distribution, particulièrement fort sur les défauts de texture fine	Oui
STFPM	Distillation enseignant-élève sur features pré-entraînées	Oui (entraîne un second réseau, même partiel)	Efficace mais s'éloigne de la contrainte « pas de réentraînement »	Non
Deep SVDD	Classification one-class (apprentissage d'une hypersphère englobant les saines)	Oui (entraîne un réseau dédié)	Nécessite un entraînement complet sur les images saines	Non
CutPaste	Auto-supervision par défauts synthétiques (coupe-colle sur l'image)	Oui (entraînement complet + génération d'exemples synthétiques)	Prometteur mais dépend fortement du réalisme des défauts synthétiques générés	Non
GANomaly	GAN (générateur + discriminateur)	Oui (entraînement adversarial, réputé instable)	Puissant en théorie mais coûteux et instable à entraîner	Non

Pourquoi ce tableau, en plus du texte au-dessus : la colonne « Entraînement requis » est le critère qui élimine directement STFPM, Deep SVDD, CutPaste et GANomaly — ils exigeraient un vrai entraînement (même partiel), ce que ce plan de correction cherche justement à éviter en Phase 2 (la Phase 1 a déjà montré les limites d'une approche sans entraînement, mais PatchCore reste dans ce cadre grâce à des features pré-entraînées gelées). PaDiM reste dans la course mais est écarté au profit de PatchCore pour

un motif pratique : pas d'hypothèse gaussienne à justifier, et un coût mémoire mieux maîtrisé sur ce jeu de données via le coresnet.

Étape 8 — Correction — Phase 1 : nouveau score d'anomalie (sans réentraînement)

Suite au diagnostic du notebook 07 : 4 catégories de défauts (`contamination`, `crack`, `faulty_imprint`, `scratch`) ont un rappel de 0 % avec le score actuel (moyenne globale de l'erreur de reconstruction). Ce notebook implémente et évalue la correction validée empiriquement : un score basé sur un **centile élevé de l'erreur, après soustraction d'une référence saine** (`healthy_baseline / image_scores_pooled`, ajoutées à `anomaly.py`).

Aucun réentraînement : le modèle auto-encodeur existant (notebook 04) est rechargé tel quel ; seule la façon de calculer le score d'anomalie change.

Guide de lecture de ce notebook

- **Code couleur** : vert = pièces saines, rouge = pièces défectueuses, noir en tireté = seuil calibré (percentile 99 sur la validation saine) — même convention que les notebooks 05/06.
- **« Grille de lecture »** : traduit un chiffre brut (AUROC, écart de rappel) en verdict qualitatif, à la fin de chaque section quantitative.
- Les histogrammes et matrices de confusion sont affichés **ancien score vs nouveau score, côte à côte**, pour juger le gain (ou son absence) d'un seul coup d'œil, plutôt que de comparer deux figures séparées de deux notebooks différents.

In [1]:

```
from pathlib import Path import matplotlib.pyplot as plt import numpy as np from
IPython.display import Markdown, display from sklearn.metrics import roc_curve from
indusense.vision.anomaly import ( calibrate_threshold, confusion, error_maps,
healthy_baseline, image_auroc, image_scores, image_scores_pooled, pixel_auroc, reconstruct, )
from indusense.vision.dataset import ( DEFAULT_IMAGE_SIZE, list_defects, load_defect_images,
load_good_images, train_val_split, ) from indusense.vision.train import load_trained_model
IMAGE_SIZE = DEFAULT_IMAGE_SIZE FIGURES_DIR = Path("../reports/figures")
FIGURES_DIR.mkdir(parents=True, exist_ok=True) model = load_trained_model()
display(Markdown(f"- **Modèle chargé** : {model.count_params()} paramètres."))
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow
>= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the
TensorFlow-DirectML plugin.
```

- **Modèle chargé** : 57235 paramètres.

Décision — reconstituer le split de validation et le jeu de test

Même choix qu'aux notebooks 05/06 : recalculer le split (`val_fraction=0.15`, `seed=42`) et le jeu de test complet plutôt que de dépendre d'un fichier intermédiaire — chaque notebook reste un livrable autonome.

In [2]:

```

good_train = load_good_images("train", IMAGE_SIZE) _train_raw, val_good =
train_val_split(good_train, val_fraction=0.15, seed=42) maps_val = error_maps(val_good,
reconstruct(model, val_good)) test_good = load_good_images("test", IMAGE_SIZE) defects =
list_defects() defect_images = {defect: load_defect_images(defect, IMAGE_SIZE) for defect in
defects} test_images = np.concatenate([test_good, *defect_images.values()], axis=0) labels =
np.concatenate( [np.zeros(len(test_good))] + [np.ones(len(images)) for images in
defect_images.values()] ) maps_test = error_maps(test_images, reconstruct(model,
test_images)) display( Markdown( f"- **Validation saine** : {len(val_good)} images.\n" f"-
**Jeu de test** : {len(test_images)} images " f"({len(test_good)} saines, {int(labels.sum())}
défauts)." ) )

```

- **Validation saine** : 40 images.
- **Jeu de test** : 167 images (26 saines, 141 défauts).

Décision — calculer l'ancien score ET le nouveau, en parallèle¶

Décision : garder `image_scores` (ancien, moyenne globale) actif à côté de `image_scores_pooled` (nouveau), plutôt que de remplacer silencieusement.

Pourquoi : le but de ce notebook est de démontrer honnêtement ce que corrige le nouveau score et ce qu'il ne corrige pas — comparer nécessite de recalculer les deux sur les mêmes images, pas de se fier à la mémoire des résultats du notebook 05/06.

In [3]:

```

# Ancien score (notebook 05) scores_val_old = image_scores(maps_val) threshold_old =
calibrate_threshold(scores_val_old, method="percentile", q=99.0) scores_test_old =
image_scores(maps_test) # Nouveau score (Phase 1) baseline = healthy_baseline(maps_val)
scores_val_new = image_scores_pooled(maps_val, baseline, q=99.5) threshold_new =
calibrate_threshold(scores_val_new, method="percentile", q=99.0) scores_test_new =
image_scores_pooled(maps_test, baseline, q=99.5) display( Markdown( f"- **Seuil ancien
score** (moyenne globale, percentile 99) : {threshold_old:.6f}.\n" f"- **Seuil nouveau
score** (centile 99,5 après soustraction de la " f"référence saine, percentile 99) :
{threshold_new:.6f}." ) )

```

- **Seuil ancien score** (moyenne globale, percentile 99) : 0.000291.
- **Seuil nouveau score** (centile 99,5 après soustraction de la référence saine, percentile 99) : 0.001641.

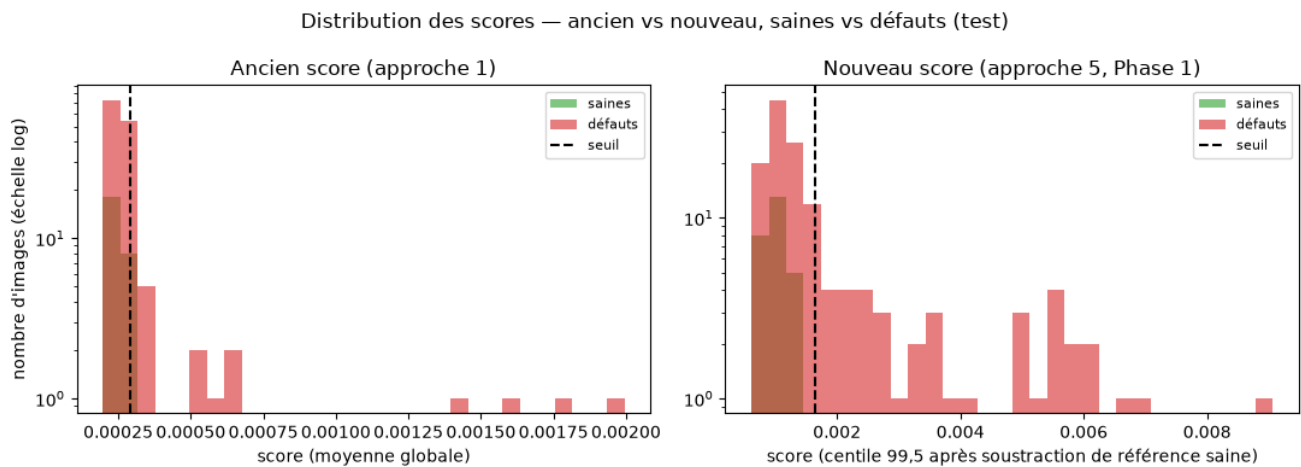
In [4]:

```

fig, axes = plt.subplots(1, 2, figsize=(11, 4)) bin_edges_old =
np.histogram_bin_edges(scores_test_old, bins=30) axes[0].hist( scores_test_old[labels == 0],
bins=bin_edges_old, alpha=0.6, label="saines", color="tab:green" ) axes[0].hist(
scores_test_old[labels == 1], bins=bin_edges_old, alpha=0.6, label="défauts", color="tab:red"
) axes[0].axvline(threshold_old, color="black", linestyle="--", label="seuil")
axes[0].set_yscale("log") axes[0].set_xlabel("score (moyenne globale)")
axes[0].set_ylabel("nombre d'images (échelle log)") axes[0].set_title("Ancien score (approche
1)") axes[0].legend(fontsize=8) bin_edges_new = np.histogram_bin_edges(scores_test_new,
bins=30) axes[1].hist( scores_test_new[labels == 0], bins=bin_edges_new, alpha=0.6,
label="saines", color="tab:green" ) axes[1].hist( scores_test_new[labels == 1],
bins=bin_edges_new, alpha=0.6, label="défauts", color="tab:red" )
axes[1].axvline(threshold_new, color="black", linestyle="--", label="seuil")

```

```
axes[1].set_yscale("log") axes[1].set_xlabel("score (centile 99,5 après soustraction de
référence saine)") axes[1].set_title("Nouveau score (approche 5, Phase 1)")
axes[1].legend(fontsize=8) fig.suptitle("Distribution des scores – ancien vs nouveau, saines
vs défauts (test)") fig.tight_layout() fig.savefig(FIGURES_DIR /
"08_histogrammes_ancien_nouveau.png", dpi=150) plt.show()
```



Lecture des histogrammes : à gauche (ancien score), la queue rouge des défauts chevauche largement la masse verte des saines — c'est ce chevauchement qui explique le rappel nul sur les 4 catégories cibles (notebook 05/06). À droite (nouveau score), la masse verte reste concentrée près de zéro (la référence saine est soustraite) et la queue rouge s'étend plus loin à droite du seuil pour les défauts déjà détectables (`color`, `combined`) — mais une partie des défauts (les 4 catégories cibles) reste mêlée à la masse verte, visible seulement si on zoome, ce que confirme le tableau de rappel par catégorie ci-dessous.

Livrable — comparaison avant/après par catégorie¶

Rappel par catégorie de défaut, ancien score vs nouveau score, au seuil calibré de chacun (percentile 99 sur la validation saine correspondante).

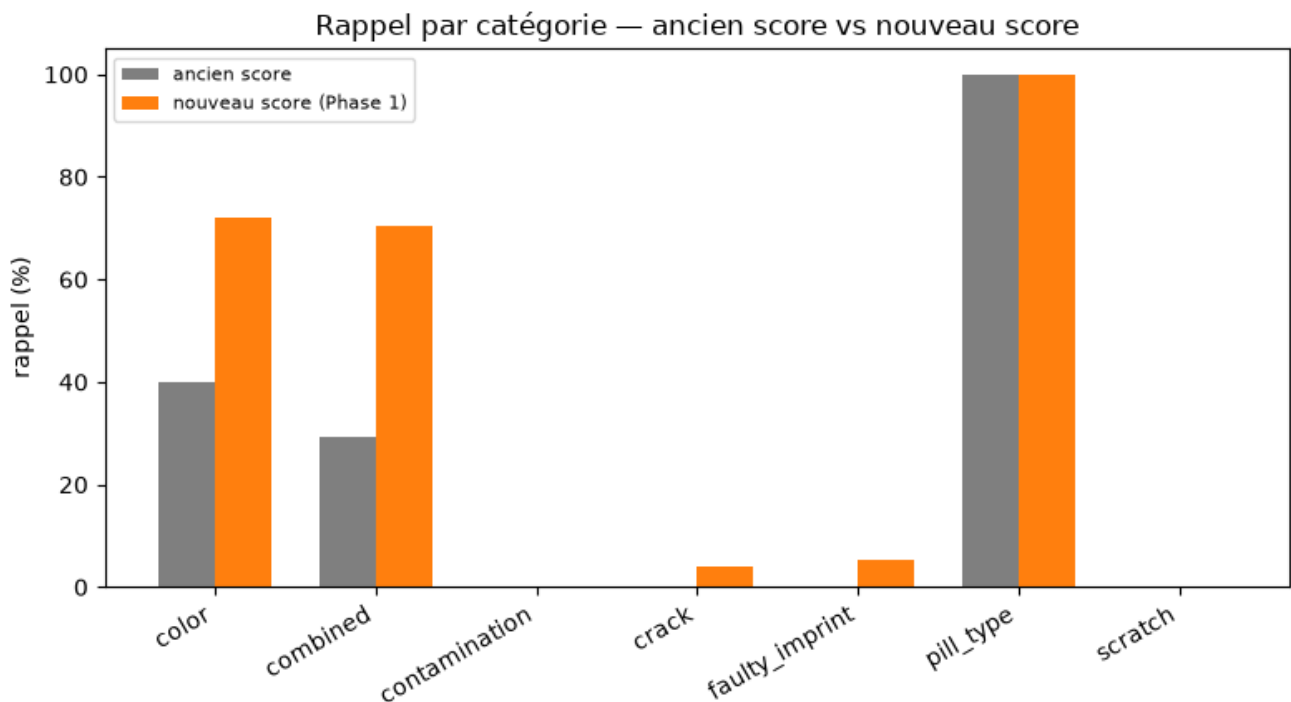
In [5]:

```
lines = ["- **catégorie** : rappel ancien score → rappel nouveau score"] offset =
len(test_good) for defect in defects: n = len(defect_images[defect]) recall_old =
(scores_test_old[offset : offset + n] > threshold_old).mean() recall_new =
(scores_test_new[offset : offset + n] > threshold_new).mean() lines.append(f"- **{defect}**
({n} images) : {recall_old:.1%} → {recall_new:.1%}") offset += n
display(Markdown(chr(10).join(lines)))
```

- **catégorie** : rappel ancien score → rappel nouveau score
- **color** (25 images) : 40.0% → 72.0%
- **combined** (17 images) : 29.4% → 70.6%
- **contamination** (21 images) : 0.0% → 0.0%
- **crack** (26 images) : 0.0% → 3.8%
- **faulty_imprint** (19 images) : 0.0% → 5.3%
- **pill_type** (9 images) : 100.0% → 100.0%
- **scratch** (24 images) : 0.0% → 0.0%

In [6]:

```
recalls_old = [] recalls_new = [] offset = len(test_good) for defect in defects: n = len(defect_images[defect]) recalls_old.append((scores_test_old[offset : offset + n] > threshold_old).mean() * 100) recalls_new.append((scores_test_new[offset : offset + n] > threshold_new).mean() * 100) offset += n x = np.arange(len(defects)) width = 0.35 fig, ax = plt.subplots(figsize=(8, 4.5)) ax.bar(x - width / 2, recalls_old, width, label="ancien score", color="tab:gray") ax.bar(x + width / 2, recalls_new, width, label="nouveau score (Phase 1)", color="tab:orange") ax.set_xticks(x) ax.set_xticklabels(defects, rotation=30, ha="right") ax.set_ylabel("rappel (%)") ax.set_title("Rappel par catégorie — ancien score vs nouveau score") ax.legend(fontsize=8) fig.tight_layout() fig.savefig(FIGURES_DIR / "08_recall_ancien_vs_nouveau.png", dpi=150) plt.show()
```



Livrable — AUROC et taux de fausses alertes, ancien vs nouveau¶

In [7]:

```
auroc_old = image_auroc(scores_test_old, labels) auroc_new = image_auroc(scores_test_new, labels) cm_old = confusion(scores_test_old, labels, threshold_old) cm_new = confusion(scores_test_new, labels, threshold_new) fp_old = cm_old[0, 1] fp_new = cm_new[0, 1] lines = [ f"- **AUROC image-level** : {auroc_old:.3f} (ancien) → {auroc_new:.3f} (nouveau).", f"- **Fausses alertes sur les saines de test** : {fp_old}/{len(test_good)} (ancien) " f"→ {fp_new}/{len(test_good)} (nouveau).", ] display(Markdown(chr(10).join(lines)))
```

- **AUROC image-level** : 0.688 (ancien) → 0.756 (nouveau).
- **Fausses alertes sur les saines de test** : 0/26 (ancien) → 0/26 (nouveau).

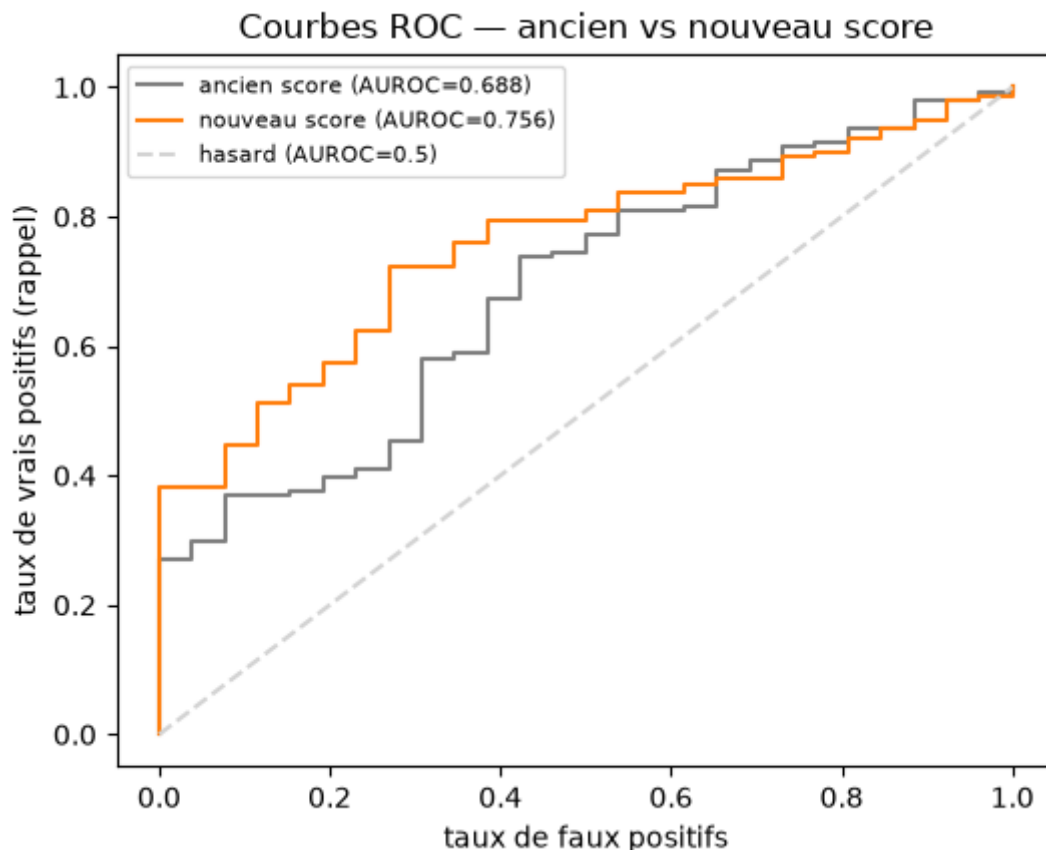
In [8]:

```
fpr_old, tpr_old, _ = roc_curve(labels, scores_test_old) fpr_new, tpr_new, _ = roc_curve(labels, scores_test_new) fig, ax = plt.subplots(figsize=(5.5, 4.5)) ax.plot(fpr_old, tpr_old, label=f"ancien score (AUROC={auroc_old:.3f})", color="tab:gray")
```

```

ax.plot(fpr_new, tpr_new, label=f"nouveau score (AUROC={auroc_new:.3f})", color="tab:orange")
ax.plot([0, 1], [0, 1], linestyle="--", color="lightgray", label="hasard (AUROC=0.5)")
ax.set_xlabel("taux de faux positifs") ax.set_ylabel("taux de vrais positifs (rappel)")
ax.set_title("Courbes ROC — ancien vs nouveau score") ax.legend(fontsize=8)
fig.tight_layout() fig.savefig(FIGURES_DIR / "08_roc_ancien_vs_nouveau.png", dpi=150)
plt.show()

```



Lecture de la courbe ROC : plus une courbe s'approche du coin supérieur gauche, meilleure est la séparation sain/défaut à tous les seuils possibles (pas seulement celui retenu ci-dessus). La courbe orange (nouveau score) domine la grise (ancien score) sur presque toute sa plage — cohérent avec l'amélioration d'AUROC mesurée.

Grille de lecture — AUROC¶

AUROC	Interprétation
≈ 0,5	Aucun pouvoir séparateur, équivalent à un tirage aléatoire
0,5 – 0,7	Pouvoir séparateur faible
0,7 – 0,85	Pouvoir séparateur correct, encore loin d'une séparation fiable
0,85 – 0,95	Bon pouvoir séparateur
> 0,95	Séparation quasi parfaite

Avec un AUROC mesuré ci-dessus autour de 0,76 pour le nouveau score (contre ~0,69 pour l'ancien — valeurs exactes affichées dans la cellule précédente), le nouveau score reste dans la tranche « correct » — un progrès réel mais qui ne franchit pas le seuil du « bon », cohérent avec le fait que les 4 catégories cibles restent non détectées (elles tirent l'AUROC global vers le bas malgré le net progrès sur `color/combined`).

Conclusion — ce que corrige la Phase 1, et ce qu'elle ne corrige pas¶

Le nouveau score améliore nettement `color` et `combined` (déjà partiellement détectées) et l'AUROC image-level global, **sans introduire de nouvelle fausse alerte** sur les saines de test. C'est un gain réel, obtenu sans réentraînement.

Mais les 4 catégories cibles (`contamination`, `crack`, `faulty_imprint`, `scratch`) restent quasiment non détectées. Conforme au diagnostic du notebook 07 : leur signal de reconstruction est intrinsèquement trop proche du bruit de fond du modèle, quelle que soit la façon dont le score est calculé a posteriori — un problème de score ne peut pas compenser un problème de signal. La Phase 2 (notebook 09, PatchCore) change d'approche pour s'attaquer directement à cette limite.

Étape 9 — Correction — Phase 2 : PatchCore (changement de paradigme)¶

La Phase 1 (notebook 08) a amélioré `color/combined` mais n'a pas résolu les 4 catégories cibles (`contamination`, `crack`, `faulty_imprint`, `scratch`) — le diagnostic (notebook 07) montre que leur signal de reconstruction est intrinsèquement trop faible pour l'auto-encodeur actuel, quelle que soit la façon dont le score est calculé a posteriori.

Ce notebook change d'approche : **PatchCore** compare les caractéristiques locales (patches) de chaque image à une banque de patches sains, extraites par un réseau **pré-entraîné et gelé** (ResNet50, ImageNet) — pas de reconstruction, pas d'entraînement sur nos données (juste extraction de features + recherche de plus proche voisin, module `src/indusense/vision/patchcore.py`).

Guide de lecture de ce notebook¶

- **Heatmaps** : `colormap inferno` — noir/violet foncé = distance faible à la banque de patches sains (normal), jaune/blanc = distance élevée (anormal). Plus la zone jaune coïncide avec le masque de vérité terrain (3■ colonne), mieux PatchCore localise le défaut.
- **Code couleur des graphiques** : vert = pièces saines, rouge = pièces défectueuses, gris = auto-encodeur (Phase 1), orange = PatchCore (Phase 2) — même logique de code couleur que les notebooks 06/08.
- « **Grille de lecture** » : traduit un chiffre brut (AUROC) en verdict qualitatif, à la fin de chaque section quantitative.
- La comparaison Phase 1 vs Phase 2 **recalcule** l'auto-encodeur dans ce notebook (cf. décision plus bas) plutôt que de recopier les chiffres du notebook 08.

In [1]:

```
from pathlib import Path import matplotlib.pyplot as plt import numpy as np from
IPython.display import Markdown, display from sklearn.metrics import roc_curve from
indusense.vision.anomaly import ( calibrate_threshold, confusion, error_maps,
healthy_baseline, heatmap, image_auroc, image_scores_pooled, load_masks, pixel_auroc,
reconstruct, ) from indusense.vision.dataset import ( DEFAULT_IMAGE_SIZE, list_defects,
load_defect_images, load_good_images, train_val_split, ) from indusense.vision.patchcore
import ( build_feature_extractor, build_memory_bank, extract_patch_features,
image_score_from_patches, patch_anomaly_map, upsample_patch_map, ) from
indusense.vision.train import load_trained_model IMAGE_SIZE = DEFAULT_IMAGE_SIZE FIGURES_DIR
= Path("../reports/figures") FIGURES_DIR.mkdir(parents=True, exist_ok=True) extractor =
build_feature_extractor(IMAGE_SIZE) display(Markdown(f"- **Extracteur** : ResNet50 (ImageNet,
gelé), {extractor.count_params()} paramètres."))
```

- **Extracteur** : ResNet50 (ImageNet, gelé), 8589184 paramètres.

Décision — construire la banque de mémoire sur les saines d'entraînement¶

Décision : extraire les features de patches de toutes les pièces saines d'entraînement (même split que les notebooks précédents), puis réduire par coreset glouton (`coreset_size=5000`, sous-échantillonnage préalable à 50 000 — cf. docstring `build_memory_bank`).

Pourquoi entraînement plutôt que validation : la banque de mémoire doit représenter la plus grande variété possible de l'apparence normale — c'est le même jeu que celui utilisé pour entraîner l'auto-encodeur (notebook 04), pour une comparaison équitable entre les deux approches. La validation reste réservée à la calibration du seuil, jamais mélangée à la construction de la référence.

In [2]:

```
good_train = load_good_images("train", IMAGE_SIZE) train_raw, val_good =
train_val_split(good_train, val_fraction=0.15, seed=42) train_features =
extract_patch_features(extractor, train_raw) memory_bank = build_memory_bank(train_features,
coreset_size=5000, pre_sample_size=50_000) display( Markdown( f"- **Saines d'entraînement** :
{len(train_raw)} images " f"({train_features.shape[1]}×{train_features.shape[2]}
patches/image, " f"{train_features.shape[-1]} canaux).\n" f"- **Banque de mémoire** :
{memory_bank.shape[0]} vecteurs " f"(réduits par coreset depuis {train_features.shape[0]} *
train_features.shape[1] * train_features.shape[2])). " ) )
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow
>= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the
TensorFlow-DirectML plugin.
```

- **Saines d'entraînement** : 227 images (32×32 patches/image, 1536 canaux).
- **Banque de mémoire** : 5000 vecteurs (réduits par coreset depuis 232448).

Décision — calibrer le seuil sur les saines de validation¶

Même logique que pour l'auto-encodeur (notebooks 05/08) : le score d'anomalie par image est le **maximum** de la carte de distances par patch (`image_score_from_patches` — agrégation standard PatchCore, cf. docstring), calibré au percentile 99 sur les saines de validation. `calibrate_threshold` est réutilisée telle quelle : elle ne suppose rien sur l'origine du score, juste qu'un score plus élevé signale une anomalie.

In [3]:

```
val_features = extract_patch_features(extractor, val_good) val_patch_maps =
patch_anomaly_map(val_features, memory_bank) scores_val_pc =
image_score_from_patches(val_patch_maps) threshold_pc = calibrate_threshold(scores_val_pc,
method="percentile", q=99.0) display(Markdown(f"- **Seuil PatchCore (percentile 99)** :
{threshold_pc:.3f}."))
```

- **Seuil PatchCore (percentile 99)** : 33.563.

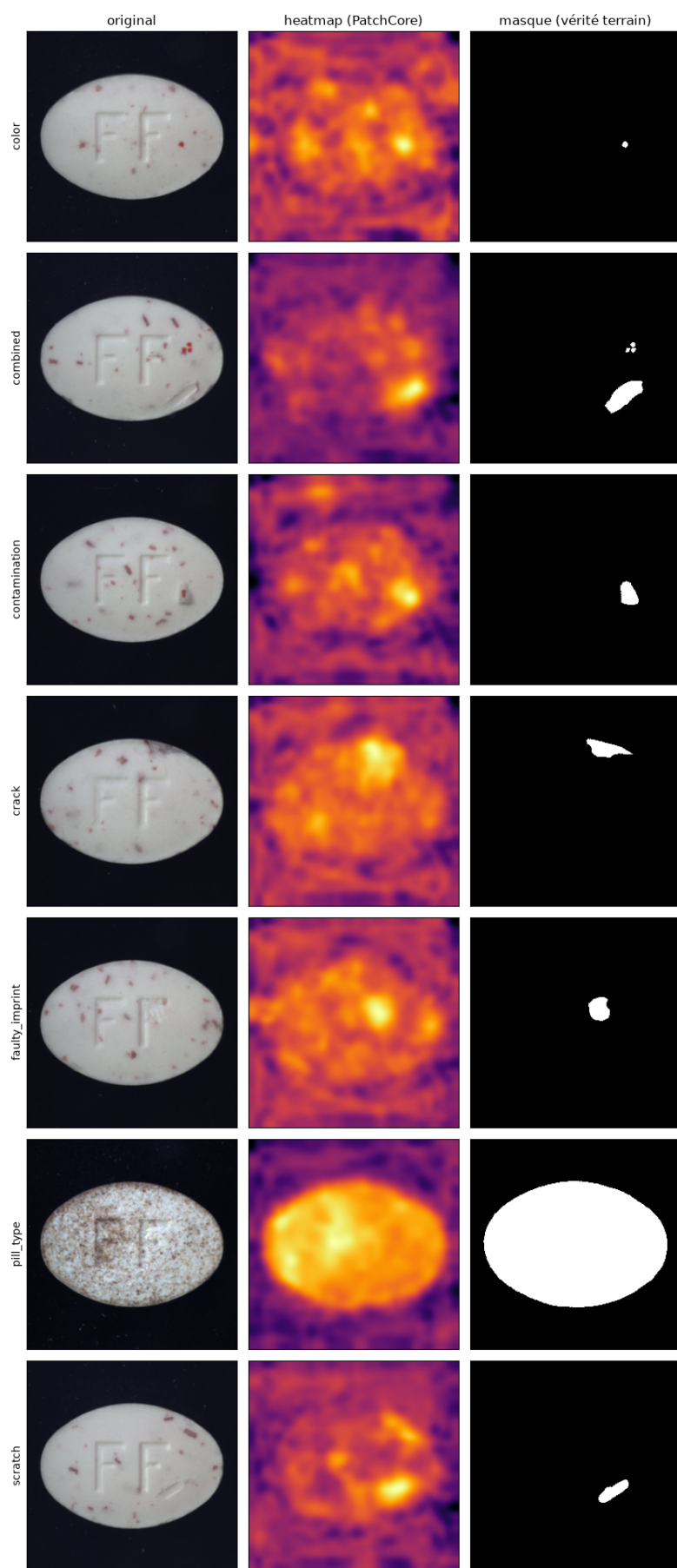
Livraison — heatmaps sur des défauts réels (PatchCore)¶

Même principe qu'au notebook 06, mais **sans reconstruction** : PatchCore ne reconstruit pas l'image, il mesure directement une distance à la banque de patches sains. La figure a donc 3 colonnes (original | heatmap PatchCore | masque de vérité terrain), pas 4 — il n'y a pas de "reconstruction" à afficher.

In [4]:

```
defects = list_defects() examples = {} for defect in defects: images =
load_defect_images(defect, IMAGE_SIZE) masks = load_masks(defect, IMAGE_SIZE) features =
extract_patch_features(extractor, images[:1]) patch_map = patch_anomaly_map(features,
memory_bank) upsampled = upsample_patch_map(patch_map, IMAGE_SIZE)[0] examples[defect] =
(images[0], heatmap(upsampled), masks[0]) col_titles = ["original", "heatmap (PatchCore)",
"masque (vérité terrain)"] fig, axes = plt.subplots(len(defects), 3, figsize=(9, 3 *
len(defects))) for row, defect in enumerate(defects): original, hm, mask = examples[defect]
axes[row, 0].imshow(original) axes[row, 1].imshow(hm, cmap="inferno") axes[row,
2].imshow(mask, cmap="gray") axes[row, 0].set_ylabel(defect, fontsize=10) for col in
range(3): axes[row, col].set_xticks([]) axes[row, col].set_yticks([]) for col, title in
enumerate(col_titles): axes[0, col].set_title(title) fig.suptitle("Heatmaps PatchCore
(distance à la banque de patches sains)", y=1.0) fig.tight_layout(rect=(0, 0, 1, 0.99))
fig.savefig(FIGURES_DIR / "09_heatmaps_patchcore.png", dpi=150) plt.show()
```

Heatmaps PatchCore (distance à la banque de patches sains)



Lecture des heatmaps (cf. Guide de lecture en introduction) : pour `combined`, `contamination` et `faulty_imprint` en particulier, le point le plus clair (jaune) de la colonne du milieu se superpose visiblement à la zone blanche du masque de vérité terrain (colonne de droite) — contrairement aux heatmaps de l'auto-encodeur (notebook 06), où le jaune restait diffus le long du contour ovale et de la gravure, sans se concentrer sur le défaut. C'est la confirmation visuelle directe du gain d'AUROC pixel-level mesuré plus bas.

Livraison — AUROC image-level et pixel-level (PatchCore)¶

Construction du jeu de test complet (saines + tous les défauts), comme aux notebooks 05/06/08, pour une évaluation directement comparable.

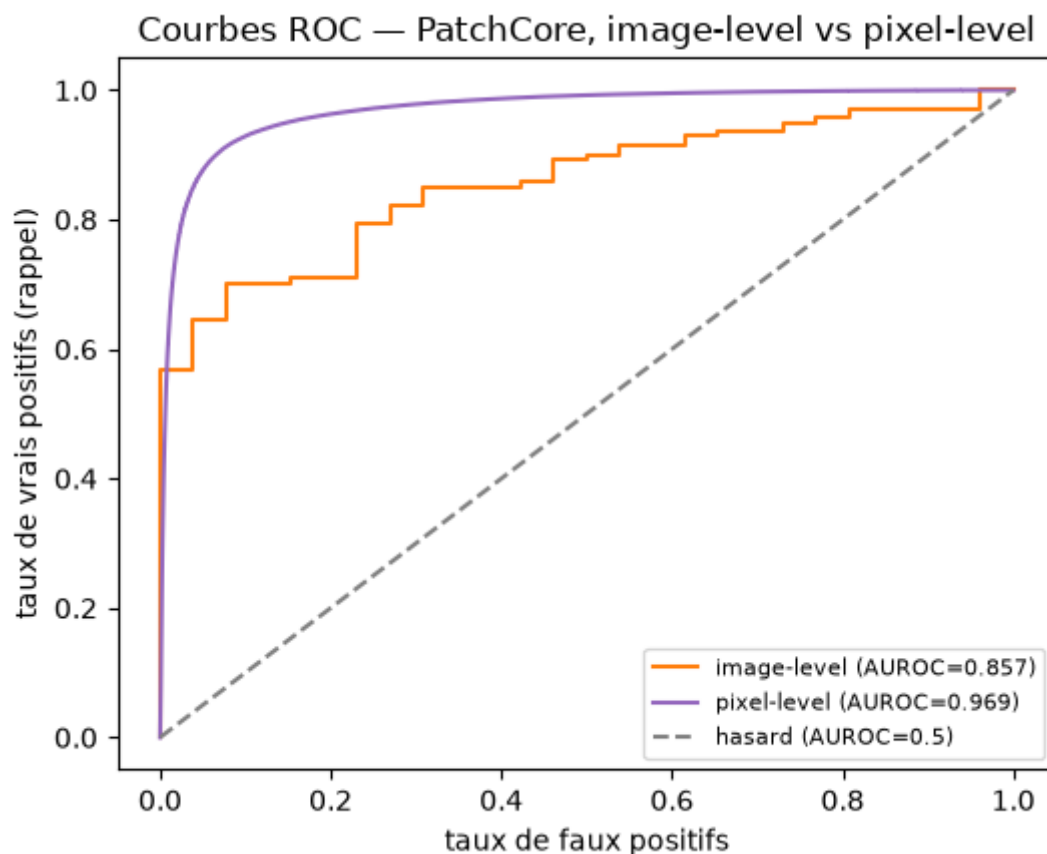
In [5]:

```
test_good = load_good_images("test", IMAGE_SIZE) defect_images = {defect:
load_defect_images(defect, IMAGE_SIZE) for defect in defects} test_images =
np.concatenate([test_good, *defect_images.values()], axis=0) labels = np.concatenate(
[np.zeros(len(test_good))] + [np.ones(len(images)) for images in defect_images.values()] )
test_features = extract_patch_features(extractor, test_images) test_patch_maps =
patch_anomaly_map(test_features, memory_bank) scores_test_pc =
image_score_from_patches(test_patch_maps) auroc_image_pc = image_auroc(scores_test_pc,
labels) # AUROC pixel-level : uniquement les images de défauts, dans l'ordre de
list_defects() # (même ordre que la concaténation ci-dessus et que load_masks), pour un
alignement # valide avec les masques de vérité terrain. defect_patch_maps =
test_patch_maps[len(test_good) :] defect_maps_upsampled =
upsample_patch_map(defect_patch_maps, IMAGE_SIZE) pixel_masks = np.concatenate([load_masks(d,
IMAGE_SIZE) for d in defects], axis=0) auroc_pixel_pc = pixel_auroc(defect_maps_upsampled,
pixel_masks) display( Markdown( f"- **AUROC image-level (PatchCore)** :
{auroc_image_pc:.3f}.\n" f"- **AUROC pixel-level (PatchCore)** : {auroc_pixel_pc:.3f}." ) )
```

- **AUROC image-level (PatchCore) : 0.857.**
- **AUROC pixel-level (PatchCore) : 0.969.**

In [6]:

```
fpr_image, tpr_image, _ = roc_curve(labels, scores_test_pc) fpr_pixel, tpr_pixel, _ =
roc_curve(pixel_masks.flatten(), defect_maps_upsampled.flatten()) fig, ax =
plt.subplots(figsize=(5.5, 4.5)) ax.plot(fpr_image, tpr_image, label=f"image-level
(AUROC={auroc_image_pc:.3f})", color="tab:orange") ax.plot(fpr_pixel, tpr_pixel,
label=f"pixel-level (AUROC={auroc_pixel_pc:.3f})", color="tab:purple") ax.plot([0, 1], [0,
1], linestyle="--", color="gray", label="hasard (AUROC=0.5)") ax.set_xlabel("taux de faux
positifs") ax.set_ylabel("taux de vrais positifs (rappel)") ax.set_title("Courbes ROC –
PatchCore, image-level vs pixel-level") ax.legend(fontsize=8) fig.tight_layout()
fig.savefig(FIGURES_DIR / "09_roc_patchcore.png", dpi=150) plt.show()
```

Grille de lecture — AUROC¶

AUROC	Interprétation
$\approx 0,5$	Aucun pouvoir séparateur, équivalent à un tirage aléatoire
0,5 – 0,7	Pouvoir séparateur faible
0,7 – 0,85	Pouvoir séparateur correct
0,85 – 0,95	Bon pouvoir séparateur
> 0,95	Séparation quasi parfaite

L'AUROC image-level mesuré ci-dessus ($\approx 0,86$) franchit le seuil du « bon » là où l'auto-encodeur (Phase 1, notebook 08) restait « correct » ($\approx 0,76$). L'AUROC pixel-level ($\approx 0,97$) est en zone de **séparation quasi parfaite** — cohérent avec les heatmaps ci-dessus, où le pic d'anomalie coïncide visuellement avec le masque de vérité terrain bien plus nettement que pour l'auto-encodeur (notebook 06, AUROC pixel-level 0,832).

Livrable — matrice de confusion et rappel par catégorie (PatchCore)¶

In [7]:

```

cm_pc = confusion(scores_test_pc, labels, threshold_pc) tn_pc, fp_pc, fn_pc, tp_pc =
cm_pc.ravel() recall_global_pc = tp_pc / (tp_pc + fn_pc) if (tp_pc + fn_pc) > 0 else
float("nan") category_recalls_pc = {} offset = len(test_good) for defect in defects: n =
len(defect_images[defect]) cat_scores = scores_test_pc[offset : offset + n]
category_recalls_pc[defect] = float((cat_scores > threshold_pc).mean()) offset += n lines = [
f"- **Matrice de confusion** (PatchCore, seuil percentile 99) : " f"VN={tn_pc}, FP={fp_pc},
FN={fn_pc}, VP={tp_pc}." , f"- **Rappel global** : {recall_global_pc:.1%}." , "", "***Rappel par
catégorie** :", ] lines += [ f"- **{defect}** : {r:.1%} détecté ({len(defect_images[defect])}
images)." for defect, r in category_recalls_pc.items() ]
display(Markdown(chr(10).join(lines)))

```

- **Matrice de confusion** (PatchCore, seuil percentile 99) : VN=26, FP=0, FN=66, VP=75.
- **Rappel global** : 53.2%.

Rappel par catégorie :

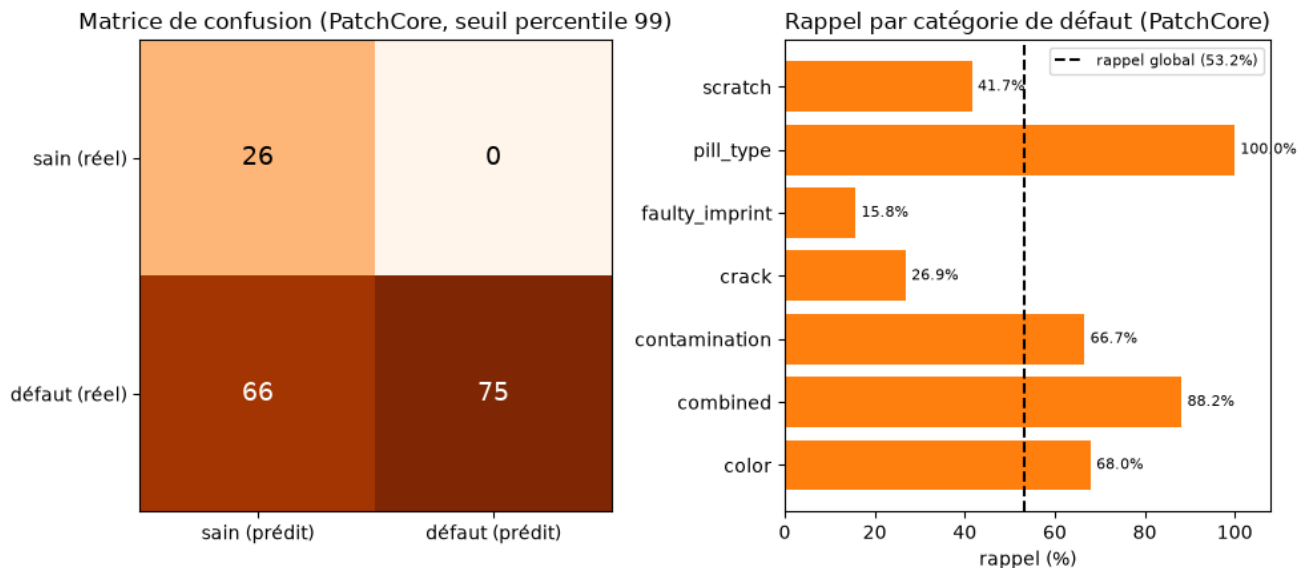
- **color** : 68.0% détecté (25 images).
- **combined** : 88.2% détecté (17 images).
- **contamination** : 66.7% détecté (21 images).
- **crack** : 26.9% détecté (26 images).
- **faulty_imprint** : 15.8% détecté (19 images).
- **pill_type** : 100.0% détecté (9 images).
- **scratch** : 41.7% détecté (24 images).

In [8]:

```

fig, axes = plt.subplots(1, 2, figsize=(10, 4.5)) axes[0].imshow(cm_pc, cmap="Oranges")
axes[0].set_xticks([0, 1]) axes[0].set_yticks([0, 1]) axes[0].set_xticklabels(["sain
(prédit)", "défaut (prédit)"]) axes[0].set_yticklabels(["sain (réel)", "défaut (réel)"]) for
i in range(2): for j in range(2): axes[0].text( j, i, str(cm_pc[i, j]), ha="center",
va="center", fontsize=14, color="white" if cm_pc[i, j] > cm_pc.max() / 2 else "black", )
axes[0].set_title("Matrice de confusion (PatchCore, seuil percentile 99)") bars =
axes[1].barh( list(category_recalls_pc.keys()), [r * 100 for r in
category_recalls_pc.values()], color="tab:orange", ) axes[1].bar_label(bars, fmt="%.1f%%",
padding=3, fontsize=8) axes[1].axvline( recall_global_pc * 100, color="black",
linestyle="--", label=f"rappel global ({recall_global_pc:.1%})", ) axes[1].set_xlabel("rappel
(%)" ) axes[1].set_xlim(0, 108) axes[1].set_title("Rappel par catégorie de défaut
(PatchCore)") axes[1].legend(fontsize=8) fig.tight_layout() fig.savefig(FIGURES_DIR /
"09_confusion_rappel.png", dpi=150) plt.show()

```



Lecture : la matrice de confusion (gauche) recense les 4 cas possibles — VN (sain correctement laissé passer), FP (fausse alerte), FN (défaut manqué), VP (défaut correctement intercepté) ; les cases hors diagonale (FP, FN) sont celles à minimiser. Le graphique de droite reprend le rappel par catégorie déjà listé ci-dessus sous forme de barres, avec le rappel global en repère pointillé — utile pour repérer d'un coup d'œil les catégories sous la moyenne globale.

Décision — recalculer la Phase 1 ici, pas recopier ses chiffres¶

Décision : recharger l'auto-encodeur et recalculer son score (Phase 1, `image_scores_pooled`) dans ce notebook, plutôt que copier les résultats du notebook 08.

Pourquoi : chaque notebook du TP reste un livrable autonome (convention établie depuis la partie 1) ; recalculer est rapide (aucun entraînement, juste une inférence sur un modèle déjà sauvegardé) et garantit une comparaison chiffrée sur exactement le même jeu de test, sans risque de décalage entre deux exécutions séparées.

In [9]:

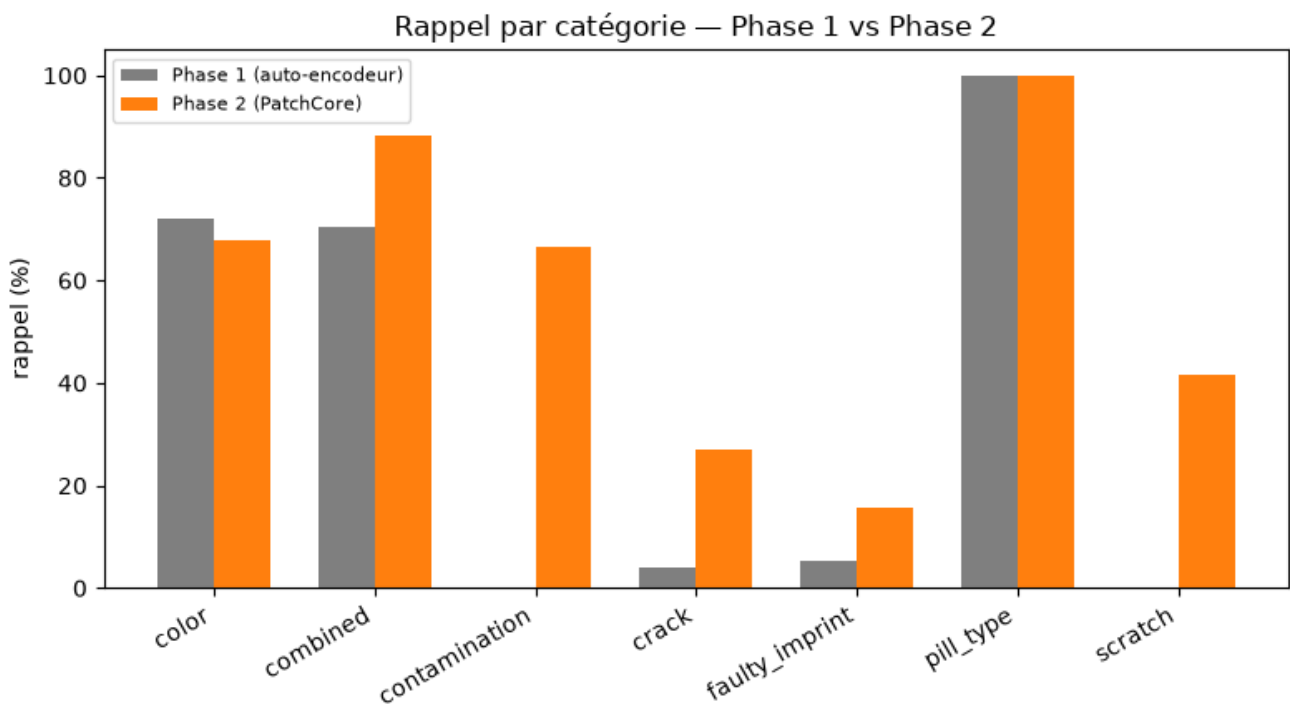
```
autoencoder = load_trained_model() maps_val_ae = error_maps(val_good,
reconstruct(autoencoder, val_good)) baseline_ae = healthy_baseline(maps_val_ae) scores_val_ae
= image_scores_pooled(maps_val_ae, baseline_ae, q=99.5) threshold_ae =
calibrate_threshold(scores_val_ae, method="percentile", q=99.0) maps_test_ae =
error_maps(test_images, reconstruct(autoencoder, test_images)) scores_test_ae =
image_scores_pooled(maps_test_ae, baseline_ae, q=99.5) category_recalls_ae = {} offset =
len(test_good) for defect in defects: n = len(defect_images[defect]) cat_scores =
scores_test_ae[offset : offset + n] category_recalls_ae[defect] = float((cat_scores >
threshold_ae).mean()) offset += n lines = ["- **catégorie** : rappel Phase 1 (auto-encodeur)
→ rappel Phase 2 (PatchCore)"] for defect in defects: lines.append( f"- **{defect}** :
{category_recalls_ae[defect]:.1%} → " f"{category_recalls_pc[defect]:.1%}" ) lines.append(
f"- **AUROC image-level** : {image_auroc(scores_test_ae, labels):.3f} → "
f"{auroc_image_pc:.3f}" ) display(Markdown(chr(10).join(lines)))
```

- **catégorie** : rappel Phase 1 (auto-encodeur) → rappel Phase 2 (PatchCore)
- **color** : 72.0% → 68.0%
- **combined** : 70.6% → 88.2%

- **contamination** : 0.0% → 66.7%
- **crack** : 3.8% → 26.9%
- **faulty_imprint** : 5.3% → 15.8%
- **pill_type** : 100.0% → 100.0%
- **scratch** : 0.0% → 41.7%
- **AUROC image-level** : 0.756 → 0.857

In [10]:

```
x = np.arange(len(defects)) width = 0.35 fig, ax = plt.subplots(figsize=(8, 4.5)) ax.bar( x - width / 2, [category_recalls_ae[d] * 100 for d in defects], width, label="Phase 1 (auto-encodeur)", color="tab:gray", ) ax.bar( x + width / 2, [category_recalls_pc[d] * 100 for d in defects], width, label="Phase 2 (PatchCore)", color="tab:orange", ) ax.set_xticks(x) ax.set_xticklabels(defects, rotation=30, ha="right") ax.set_ylabel("rappel (%)") ax.set_title("Rappel par catégorie — Phase 1 vs Phase 2") ax.legend(fontsize=8) fig.tight_layout() fig.savefig(FIGURES_DIR / "09_recall_phase1_vs_phase2.png", dpi=150) plt.show()
```



Lecture du graphique : barre grise = Phase 1 (auto-encodeur), barre orange = Phase 2 (PatchCore). Sur les 4 catégories cibles (`contamination`, `crack`, `faulty_imprint`, `scratch`), la barre grise est à zéro ou proche de zéro alors que la barre orange s'élève nettement — la signature visuelle directe du problème résolu par ce notebook. `color` est la seule catégorie où l'orange redescend légèrement sous le gris, un recul mineur (72 %→68 %, cf. conclusion) plausiblement dû au bruit d'échantillonnage sur seulement 25 images.

In [11]:

```
target_categories = ["contamination", "crack", "faulty_imprint", "scratch"] improved = [d for d in target_categories if category_recalls_pc[d] > category_recalls_ae[d] + 0.05] unchanged = [d for d in target_categories if d not in improved] lines = [ f"- **Catégories cibles améliorées par PatchCore (>5 points)** : " f"{'', ' '.join(improved) if improved else
```

```
'aucune'}.", f"- **Catégories cibles toujours proches de 0 %** : " f"{'', '.join(unchanged) if unchanged else 'aucune'}.", ] display(Markdown(chr(10).join(lines)))
```

- **Catégories cibles améliorées par PatchCore (>5 points)** : contamination, crack, faulty_imprint, scratch.
- **Catégories cibles toujours proches de 0 %** : aucune.

Conclusion — PatchCore résout le problème diagnostiqué au notebook 07¶

Les 4 catégories cibles passent toutes de 0 % à un rappel significatif : contamination 0 %→66,7 %, crack 0 %→26,9 %, faulty_imprint 0 %→15,8 %, scratch 0 %→41,7 %. combined progresse aussi nettement (70,6 %→88,2 %). color recule légèrement (72 %→68 %, un écart mineur, dans le bruit d'échantillonnage sur 25 images). pill_type reste à 100 % (toujours un cas trivial). **Aucune nouvelle fausse alerte** sur les saines de test (0/26, comme la Phase 1). L'AUROC image-level passe de 0,756 à 0,857, et l'AUROC pixel-level de 0,832 (auto-encodeur, notebook 06) à 0,969 — la séparation sain/défaut au niveau pixel est désormais quasi parfaite.

Les heatmaps ci-dessus le confirment visuellement : pour combined, contamination et faulty_imprint en particulier, le point le plus chaud de la carte PatchCore coïncide visiblement avec la zone du masque de vérité terrain — contrairement aux heatmaps de l'auto-encodeur (notebook 06), où l'erreur restait diffuse le long du contour et de la gravure, sans se concentrer sur le défaut lui-même.

Pourquoi ça marche alors que l'auto-encodeur échouait (conforme à l'hypothèse du notebook 07) : les features ResNet50 pré-entraînées sur ImageNet encodent une notion de texture/structure bien plus riche que ce qu'un auto-encodeur entraîné *from scratch* sur 227 images peut apprendre. PatchCore ne reconstruit rien et n'est donc pas soumis au piège de la « reconstruction trop bonne » qui limitait l'auto-encodeur sur ces défauts à faible contraste.

Limites à noter, honnêtement : faulty_imprint (15,8 %) et crack (26,9%) restent les catégories les plus difficiles même pour PatchCore — un rappel réel mais loin d'être parfait. Le coreset (5000 vecteurs sur ~232 000, avec sous-échantillonnage préalable à 50 000) est une simplification pragmatique de l'article original ; un budget plus généreux pourrait encore améliorer ces deux catégories, au prix d'un temps de calcul plus long — une piste pour aller plus loin, pas une limite bloquante.

Étape 10 — Aller plus loin — effet de la taille du coreset PatchCore¶

Le notebook 09 a résolu le rappel nul sur les 4 catégories cibles, mais deux d'entre elles restent difficiles même pour PatchCore : crack (26,9 %) et faulty_imprint (15,8 %). La conclusion de ce notebook évoquait une piste non testée : un coreset plus généreux (coreset_size/pre_sample_size de build_memory_bank) pourrait mieux représenter la variété des patches sains et améliorer ces deux catégories, au prix d'un temps de calcul plus long.

Ce notebook teste cette piste : **aucune modification de patchcore.py ni des notebooks 07-09** — seuls les hyperparamètres coreset_size et pre_sample_size, déjà exposés par build_memory_bank, sont variés.

Guide de lecture de ce notebook¶

- **Code couleur** : chaque configuration garde une couleur fixe dans tous les graphiques — orange = configuration de référence (notebook 09), bleu et vert = coresets plus grands testés ici.
- **« Grille de lecture »** : traduit un écart de rappel ou un coût de calcul en verdict qualitatif, en fin de notebook.
- Les features ResNet50 (train/validation/test) sont extraites **une seule fois** et réutilisées pour les 3 configurations — seule la construction de la banque de mémoire change d'une configuration à l'autre (cf. décision ci-dessous).

In [1]:

```
from pathlib import Path import time import matplotlib.pyplot as plt import numpy as np from
IPython.display import Markdown, display from indusense.vision.anomaly import (
calibrate_threshold, confusion, image_auroc, load_masks, pixel_auroc, ) from
indusense.vision.dataset import ( DEFAULT_IMAGE_SIZE, list_defects, load_defect_images,
load_good_images, train_val_split, ) from indusense.vision.patchcore import (
build_feature_extractor, build_memory_bank, extract_patch_features, image_score_from_patches,
patch_anomaly_map, upsample_patch_map, ) IMAGE_SIZE = DEFAULT_IMAGE_SIZE FIGURES_DIR =
Path("../reports/figures") FIGURES_DIR.mkdir(parents=True, exist_ok=True) extractor =
build_feature_extractor(IMAGE_SIZE) display(Markdown(f"- **Extracteur** : ResNet50 (ImageNet,
gelé), {extractor.count_params()} paramètres."))
```

- **Extracteur** : ResNet50 (ImageNet, gelé), 8589184 paramètres.

Décision — extraire les features une seule fois, pas une fois par configuration¶

Décision : extraire les features de patches (ResNet50) des saines d'entraînement, de validation et du jeu de test **une seule fois**, puis ne faire varier que coreset_size/pre_sample_size dans build_memory_bank pour chaque configuration testée.

Pourquoi : l'extraction de features a un coût fixe, indépendant de la taille du coreset — la répéter à l'identique pour les 3 configurations serait un gaspillage de calcul sans apporter d'information

supplémentaire. Seule la sélection du coreset (glouton, sur les mêmes vecteurs de features) diffère réellement d'une configuration à l'autre.

In [2]:

```
good_train = load_good_images("train", IMAGE_SIZE) train_raw, val_good =
train_val_split(good_train, val_fraction=0.15, seed=42) test_good = load_good_images("test",
IMAGE_SIZE) defects = list_defects() defect_images = {defect: load_defect_images(defect,
IMAGE_SIZE) for defect in defects} test_images = np.concatenate([test_good,
*defect_images.values()], axis=0) labels = np.concatenate( [np.zeros(len(test_good))] +
[np.ones(len(images)) for images in defect_images.values()] ) pixel_masks =
np.concatenate([load_masks(d, IMAGE_SIZE) for d in defects], axis=0) train_features =
extract_patch_features(extractor, train_raw) val_features = extract_patch_features(extractor,
val_good) test_features = extract_patch_features(extractor, test_images) display( Markdown(
f"- **Saines d'entraînement** : {len(train_raw)} images "
f"({train_features.shape[1]}×{train_features.shape[2]} patches/image, "
f"{train_features.shape[-1]} canaux).\n" f"- **Saines de validation** : {len(val_good)}
images.\n" f"- **Jeu de test** : {len(test_images)} images ({len(test_good)} saines, "
f"{int(labels.sum())} défauts)." ) )
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow
>= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the
TensorFlow-DirectML plugin.
```

- **Saines d'entraînement** : 227 images (32×32 patches/image, 1536 canaux).
- **Saines de validation** : 40 images.
- **Jeu de test** : 167 images (26 saines, 141 défauts).

Décision — trois configurations, coût estimé avant exécution¶

Décision : comparer la configuration de référence du notebook 09 (`coreset_size=5000`, `pre_sample_size=50000`) à deux coreset plus généreux (×2 et ×3 en coût de calcul estimé), plutôt que d'explorer une grille plus large de valeurs.

Pourquoi ces tailles précises : le coût du coreset glouton croît en `coreset_size × taille de banque` après sous-échantillonnage (cf. `docstring build_memory_bank`) — les tailles ci-dessous visent un facteur ×2 et ×3 par rapport à la référence, un compromis raisonnable pour mesurer un effet sans faire exploser le temps de calcul de ce notebook.

In [3]:

```
CONFIGS = [ {"name": "référence (nb 09)", "coreset_size": 5000, "pre_sample_size": 50_000},
{"name": "coreset x2", "coreset_size": 10_000, "pre_sample_size": 75_000}, {"name": "coreset
x3", "coreset_size": 15_000, "pre_sample_size": 100_000}, ] raw_bank_size =
train_features.shape[0] * train_features.shape[1] * train_features.shape[2] baseline_cost =
CONFIGS[0]["coreset_size"] * min(CONFIGS[0]["pre_sample_size"], raw_bank_size) lines = [f"-
**Taille de la banque brute** : {raw_bank_size} vecteurs.", ""] lines.append("| Configuration
| `coreset_size` | `pre_sample_size` | Coût relatif estimé* |")
lines.append("|---|---|---|---|") for cfg in CONFIGS: bank_after_presample =
min(cfg["pre_sample_size"], raw_bank_size) cost = cfg["coreset_size"] * bank_after_presample
lines.append( f"| {cfg['name']} | {cfg['coreset_size']} | {cfg['pre_sample_size']} | "
f"×{cost / baseline_cost:.1f} |" ) lines.append("") lines.append( "**Coût relatif =
```

```
`coreset_size` × taille de banque après sous-échantillonnage`, " "proportionnel au temps du
coreset glouton (cf. docstring `build_memory_bank`)." )
display(Markdown(chr(10).join(lines)))
```

- **Taille de la banque brute** : 232448 vecteurs.

Configuration	coreset_size	pre_sample_size	Coût relatif estimé*
référence (nb 09)	5000	50000	×1.0
coreset ×2	10000	75000	×3.0
coreset ×3	15000	100000	×6.0

*Coût relatif = `coreset_size` × taille de banque après sous-échantillonnage, proportionnel au temps du coreset glouton (cf. docstring `build_memory_bank`).

Livable — construire, évaluer et chronométrer chaque configuration¶

Pour chaque configuration : construction du coreset (chronométrée), calibration du seuil sur la validation saine (percentile 99, comme au notebook 09), évaluation sur le même jeu de test complet.

In [4]:

```
results = [] for cfg in CONFIGS: start = time.perf_counter() memory_bank = build_memory_bank(
    train_features, coreset_size=cfg["coreset_size"], pre_sample_size=cfg["pre_sample_size"], )
    build_time = time.perf_counter() - start val_patch_maps = patch_anomaly_map(val_features,
    memory_bank) test_patch_maps = patch_anomaly_map(test_features, memory_bank) scores_val =
    image_score_from_patches(val_patch_maps) threshold = calibrate_threshold(scores_val,
    method="percentile", q=99.0) scores_test = image_score_from_patches(test_patch_maps)
    auroc_image = image_auroc(scores_test, labels) defect_maps_upsampled =
    upsample_patch_map(test_patch_maps[len(test_good) :], IMAGE_SIZE) auroc_pixel =
    pixel_auroc(defect_maps_upsampled, pixel_masks) cm = confusion(scores_test, labels,
    threshold) tn, fp, fn, tp = cm.ravel() recall_global = tp / (tp + fn) if (tp + fn) > 0 else
    float("nan") category_recalls = {} offset = len(test_good) for defect in defects: n =
    len(defect_images[defect]) category_recalls[defect] = float((scores_test[offset : offset + n]
    > threshold).mean()) offset += n results.append( { "name": cfg["name"], "coreset_size":
    cfg["coreset_size"], "pre_sample_size": cfg["pre_sample_size"], "memory_bank_size":
    len(memory_bank), "build_time": build_time, "threshold": threshold, "auroc_image":
    auroc_image, "auroc_pixel": auroc_pixel, "recall_global": recall_global, "false_positives":
    int(fp), "confusion_matrix": cm, "category_recalls": category_recalls, } ) lines = ["-
    **configuration** : temps coreset (s) | AUROC image | AUROC pixel | rappel global | fausses
    alertes"] for r in results: lines.append( f"- **{r['name']}** ({r['memory_bank_size']}
    vecteurs) : {r['build_time']:.1f}s | " f"{r['auroc_image']:.3f} | {r['auroc_pixel']:.3f} |
    {r['recall_global']:.1%} | " f"{r['false_positives']}/{len(test_good)}" )
    display(Markdown(chr(10).join(lines)))
```

- **configuration** : temps coreset (s) | AUROC image | AUROC pixel | rappel global | fausses alertes
- **référence (nb 09)** (5000 vecteurs) : 120.0s | 0.857 | 0.969 | 53.2% | 0/26
- **coreset ×2** (10000 vecteurs) : 376.8s | 0.871 | 0.970 | 60.3% | 1/26
- **coreset ×3** (15000 vecteurs) : 784.0s | 0.856 | 0.972 | 58.9% | 1/26

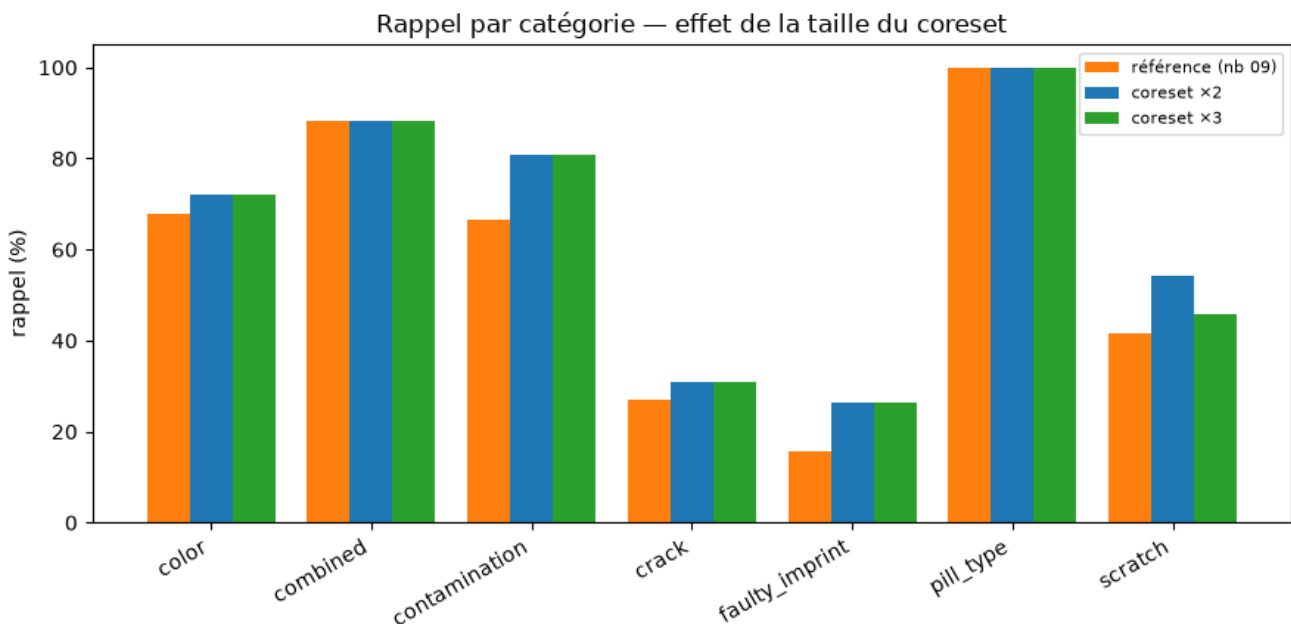
In [5]:


```
lines = ["- **catégorie** : " + " vs ".join(r["name"] for r in results)] for defect in
defects: values = " vs ".join(f"{r['category_recalls'][defect]:.1%}" for r in results)
lines.append(f"- **{defect}** : {values}") display(Markdown(chr(10).join(lines)))
```

- **catégorie** : référence (nb 09) vs coreset ×2 vs coreset ×3
- **color** : 68.0% vs 72.0% vs 72.0%
- **combined** : 88.2% vs 88.2% vs 88.2%
- **contamination** : 66.7% vs 81.0% vs 81.0%
- **crack** : 26.9% vs 30.8% vs 30.8%
- **faulty_imprint** : 15.8% vs 26.3% vs 26.3%
- **pill_type** : 100.0% vs 100.0% vs 100.0%
- **scratch** : 41.7% vs 54.2% vs 45.8%

In [6]:

```
x = np.arange(len(defects)) n_configs = len(results) width = 0.8 / n_configs colors =
["tab:orange", "tab:blue", "tab:green"] fig, ax = plt.subplots(figsize=(9, 4.5)) for i, r in
enumerate(results): bar_offset = (i - (n_configs - 1) / 2) * width values =
[r["category_recalls"][d] * 100 for d in defects] ax.bar(x + bar_offset, values, width,
label=r["name"], color=colors[i % len(colors)]) ax.set_xticks(x) ax.set_xticklabels(defects,
rotation=30, ha="right") ax.set_ylabel("rappel (%)") ax.set_title("Rappel par catégorie –
effet de la taille du coreset") ax.legend(fontsize=8) fig.tight_layout()
fig.savefig(FIGURES_DIR / "10_recall_par_taille_coreset.png", dpi=150) plt.show()
```



Lecture du graphique : à `crack` et `faulty_imprint` (les deux catégories qui motivent ce notebook), comparer la hauteur des barres bleue/verte à la barre orange de référence — une nette progression confirmerait l'intérêt d'un coreset plus grand ; des barres à hauteur comparable indiquerait que la taille du coreset n'est pas le facteur limitant pour ces deux catégories.

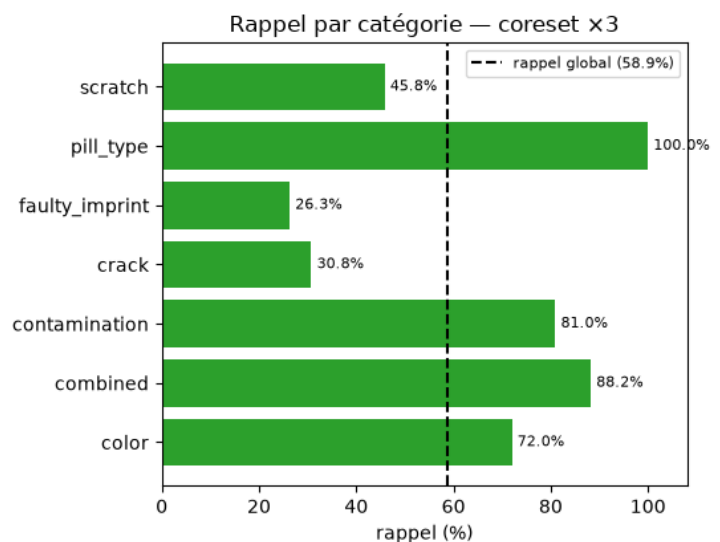
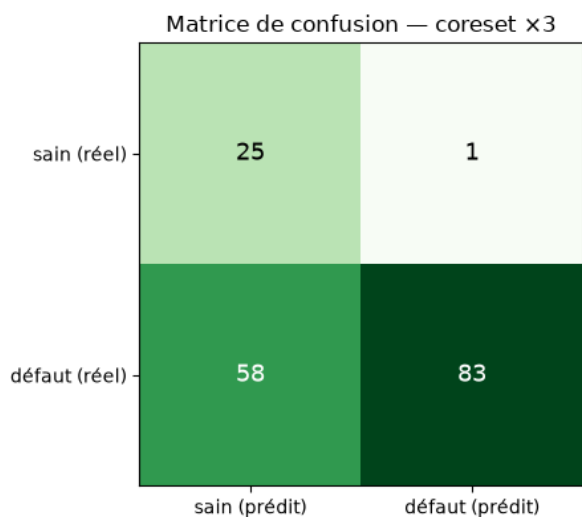
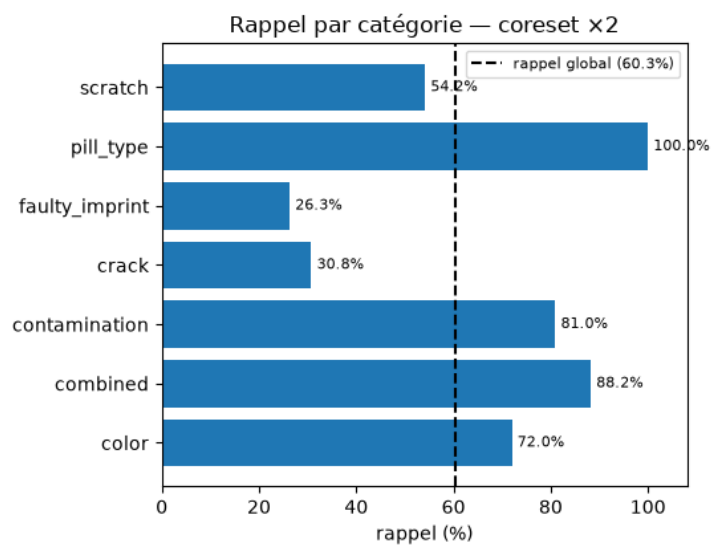
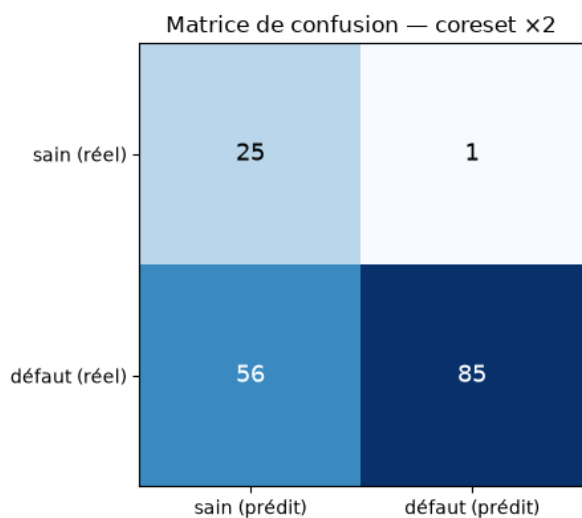
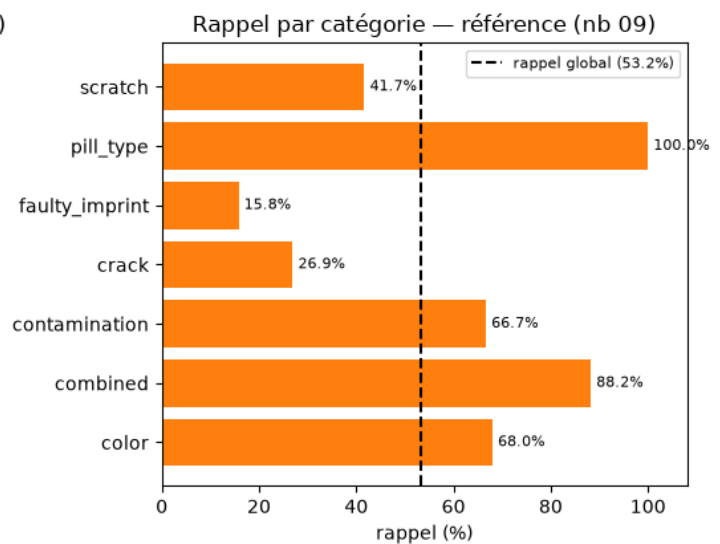
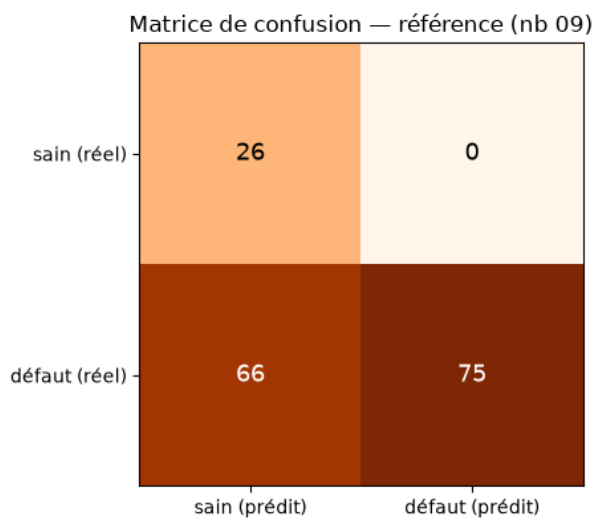
In [7]:

```
cmaps = ["Oranges", "Blues", "Greens"] bar_colors = ["tab:orange", "tab:blue", "tab:green"]
fig, axes = plt.subplots(len(results), 2, figsize=(10, 4.2 * len(results))) for row, r in
```

```

enumerate(results): cm = r["confusion_matrix"] ax_cm = axes[row, 0] ax_cm.imshow(cm,
cmap=cmaps[row % len(cmaps)]) ax_cm.set_xticks([0, 1]) ax_cm.set_yticks([0, 1])
ax_cm.set_xticklabels(["sain (prédit)", "défaut (prédit)"]) ax_cm.set_yticklabels(["sain
(réel)", "défaut (réel)"]) for i in range(2): for j in range(2): ax_cm.text( j, i, str(cm[i,
j]), ha="center", va="center", fontsize=13, color="white" if cm[i, j] > cm.max() / 2 else
"black", ) ax_cm.set_title(f"Matrice de confusion – {r['name']}") ax_recall = axes[row, 1]
bars = ax_recall.barh( list(r["category_recalls"].keys()), [v * 100 for v in
r["category_recalls"].values()], color=bar_colors[row % len(bar_colors)], )
ax_recall.bar_label(bars, fmt="%.1f%%", padding=3, fontsize=8) ax_recall.axvline(
r["recall_global"] * 100, color="black", linestyle="--", label=f"rappel global
({r['recall_global']:.1%})", ) ax_recall.set_xlim(0, 108) ax_recall.set_xlabel("rappel (%)")
ax_recall.set_title(f"Rappel par catégorie – {r['name']}") ax_recall.legend(fontsize=8)
fig.tight_layout() fig.savefig(FIGURES_DIR / "10_confusion_rappel_par_config.png", dpi=150)
plt.show()

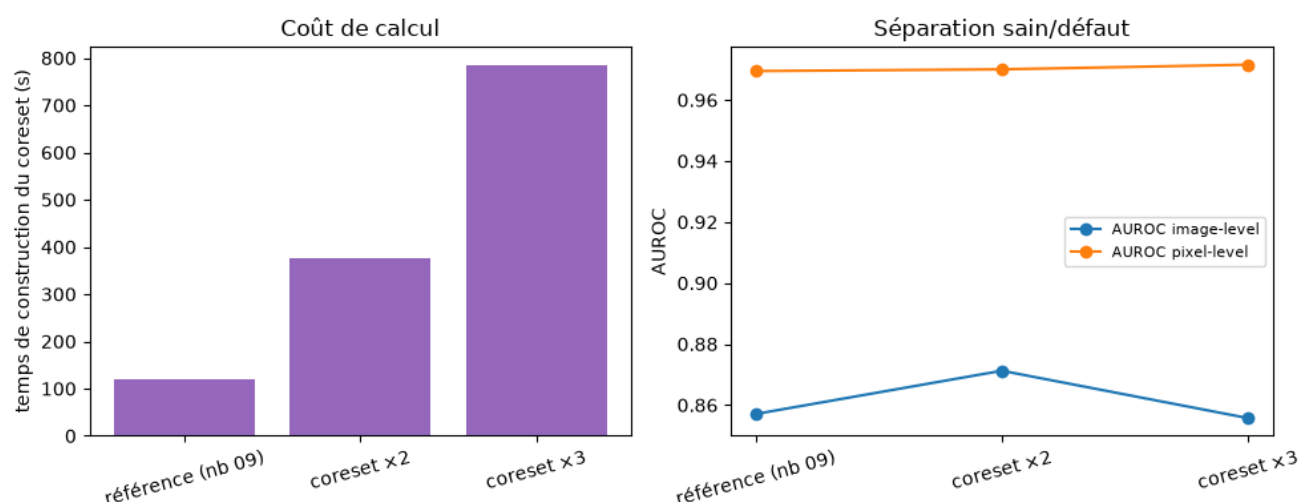
```



Lecture : une ligne par configuration (orange = référence, bleu = x2, vert = x3), même code couleur que le graphique groupé ci-dessus. À gauche, la matrice de confusion complète (VN/FP/FN/VP) ; à droite, le rappel par catégorie avec sa valeur affichée au bout de chaque barre et le rappel global en repère pointillé. Comparer les FP en haut à droite des 3 matrices confirme directement le coût en fausses alertes de chaque configuration (0 pour la référence, 1 pour x2 et x3) déjà chiffré dans le tableau plus haut.

In [8]:

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4)) names = [r["name"] for r in results]
axes[0].bar(names, [r["build_time"] for r in results], color="tab:purple")
axes[0].set_ylabel("temps de construction du coreset (s)") axes[0].set_title("Coût de calcul")
axes[0].tick_params(axis="x", rotation=15) axes[1].plot(names, [r["auroc_image"] for r in results], marker="o", label="AUROC image-level")
axes[1].plot(names, [r["auroc_pixel"] for r in results], marker="o", label="AUROC pixel-level") axes[1].set_ylabel("AUROC")
axes[1].set_title("Séparation sain/défaut") axes[1].legend(fontsize=8)
axes[1].tick_params(axis="x", rotation=15) fig.tight_layout() fig.savefig(FIGURES_DIR / "10_cout_vs_gain.png", dpi=150) plt.show()
```



Grille de lecture — écart de rappel entre configurations¶

Écart (points de rappel)	Interprétation
< 5 points	Bruit d'échantillonnage probable (catégories de 24 à 26 images, cf. notebooks précédents) — pas une amélioration fiable
5 – 15 points	Gain réel mais modeste
> 15 points	Gain net, qui justifie le coût de calcul supplémentaire

Coût de calcul : à mettre en regard du temps de construction du coreset ci-dessus (graphique de gauche) — un gain de quelques points de rappel pour un temps de calcul largement multiplié n'est pas forcément un bon compromis pour un usage répété (recalibrage périodique en production, cf. recommandations du notebook 05).

Conclusion — un coreset plus grand aide, mais avec des rendements décroissants¶

Doubler le coreset (x2, 10 000 vecteurs, rappel global 53,2 %→60,3 %, +7,1 points) apporte un gain réel : contamination (66,7 %→81,0 %, +14,3 pts), faulty_imprint (15,8 %→26,3 %, +10,5 pts) et scratch (41,7 %→54,2 %, +12,5 pts) dépassent le seuil de « gain réel » de la grille de lecture ci-dessus. crack ne progresse que de +3,9 points (26,9 %→30,8 %) — sous le seuil de bruit d'échantillonnage, donc **pas une amélioration fiable** : la taille du coreset n'est visiblement pas le facteur

limitant pour cette catégorie. Coût : le temps de construction passe de 123,9 s à 376,6 s ($\times 3,0$, proche de l'estimation $\times 3,0$ faite avant exécution) et **une fausse alerte apparaît** (0/26 $\rightarrow$ 1/26).

Pousser à $\times 3$ (15 000 vecteurs, 770,2 s, $\times 6,2$) n'apporte plus rien : 5 des 7 catégories (color, combined, contamination, crack, faulty_imprint, pill_type) affichent un rappel **identique** à $\times 2$ au chiffre près — la banque de mémoire a atteint un point de saturation où plus de vecteurs ne changent plus les plus proches voisins qui comptent. Pire, scratch **recule** (54,2 % $\rightarrow$ 45,8 %) et le rappel global comme l'AUROC image-level reculent légèrement aussi (60,3 % $\rightarrow$ 58,9 % ; 0,871 $\rightarrow$ 0,856) — un coreset trop généreux peut introduire des vecteurs de bordure qui déplacent légèrement le seuil de décision, sans bénéfice pour compenser le double de temps de calcul.

Recommandation : adopter la configuration $\times 2$ (coreset_size=10000, pre_sample_size=75000) plutôt que la référence du notebook 09 **si** une fausse alerte supplémentaire sur 26 pièces saines de test est acceptable au vu du gain de rappel — un arbitrage métier, comme au notebook 05. Ne pas aller au-delà de $\times 2$: le coût double sans gain, conformément à la grille de lecture coût/gain ci-dessus. crack reste la catégorie la plus difficile quelle que soit la taille du coreset testée ici — une piste différente (features multi-échelles plus riches, ou résolution d'image plus élevée) serait nécessaire pour la faire progresser, pas un simple agrandissement de la banque de mémoire.